

Local Truth in SessionStream

A Software Engineer's Working Textbook on Categories, Presheaves, Sheaves, Toposes,
and Cohomological Diagnostics

Custom study edition

Edition 0.1 - 15 August 2026

Contents

Preface	1
What this book is for	1
Your entry point: Goldblatt Chapter 3, limits	1
Three readings of every construction	2
A twelve-week route	2
Notation	3
The recurring SessionStream pipeline	3
Source and scope note	4
1 The System We Will Keep Rebuilding	5
1.1 Canonical history and derived views	5
1.2 The family of cursors	5
1.3 Hydration as a local-to-global protocol	6
1.4 The six law families	6
1.4.1 Per-session serializability	6
1.4.2 Consistent-cut snapshots	6
1.4.3 Stable retry identity	6
1.4.4 Atomic projection progress	7
1.4.5 Deterministic replay	7
1.4.6 Snapshot-plus-suffix completeness	7
1.5 The first context vocabulary	7
1.6 Study discipline: classify the failure first	8
1.7 Exercises	8
2 Categories as Composable Semantics	9
2.1 Definition and engineering reading	9
2.2 A prefix category	9
2.3 Commutative diagrams as route independence	10
2.4 Isomorphism and representation independence	10
2.5 Monic and epic as cancellation properties	11
2.6 Duality	11
2.7 A category of SessionStream contexts	11
2.8 Exercises	12
3 Limits: The Best Joint Witness	13
3.1 Diagrams and cones	13
3.2 Familiar limits	13
3.2.1 Terminal object	13
3.2.2 Product	13
3.2.3 Equalizer	14
3.2.4 Pullback	14
3.3 Product versus pullback in APIs	14
3.4 Equalizers as invariant subsets	14
3.5 A consistent snapshot as a limit problem	15
3.6 Finite completeness and system composition	15
3.7 Preview: the sheaf condition as an equalizer	16
3.8 Universal-property proof pattern	16
3.9 Exercises	16

4	Pullbacks, Cuts, and Transactional Witnesses	18
4.1	Definition	18
4.2	Pullback as a typed join	18
4.3	The SQLite snapshot study	19
4.4	Projection progress as a pullback of claims	19
4.5	Pulling back predicates	20
4.6	Base change	20
4.7	Pullback-stable coverage preview	20
4.8	A proof obligation for snapshot-plus-live	21
4.9	Exercises	21
5	Colimits: Variants, Quotients, and Amalgamation	23
5.1	Cocones and colimits	23
5.2	Coproducts and event variants	23
5.3	Coequalizers and identification	23
5.4	Stable retry identity as a quotient question	24
5.5	Quotients and well-defined projections	24
5.6	Pushouts and amalgamation	25
5.7	Event-history branching	25
5.8	Colimits and architecture boundaries	25
5.9	Exercises	26
6	Exponentials: Behavior as an Object	27
6.1	Definition	27
6.2	Handlers as points of a function object	27
6.3	Projections and declared dependencies	28
6.4	Evaluation and test generation	28
6.5	Exponentials are context-sensitive	28
6.6	Cartesian closure and composition of programs	29
6.7	From exponentials to topoi	29
6.8	Sufficient parameters as factorization through an exponential	29
6.9	Exercises	30
7	Functors: Structure-Preserving Interpretations	31
7.1	Definition	31
7.2	A history category	31
7.3	Projection is not automatically a functor	32
7.4	Encoding as a functor	32
7.5	Forgetful and summary functors	33
7.6	Composite interpretations	33
7.7	Faithful, full, and essentially surjective	33
7.8	Functoriality as executable law	34
7.9	Exercises	34
8	Natural Transformations: Coherent Change Everywhere	35
8.1	Definition	35
8.2	Schema migration as naturality	35
8.3	Replay implementation replacement	36
8.4	Natural transformations between presheaves	36
8.5	Ad hoc conversion versus naturality	36
8.6	Vertical and horizontal composition	36
8.7	Functor categories	37
8.8	A naturality review template	37
8.9	Exercises	37
9	Presheaves: Data Indexed by Context	39
9.1	Definition	39

9.2	What is the base category?	39
9.2.1	Prefix contexts	39
9.2.2	Aspect contexts	40
9.2.3	Product contexts	40
9.2.4	Trace regions	40
9.2.5	API-information contexts	40
9.3	A presheaf of locally lawful observations	40
9.4	Restriction is not arbitrary projection	40
9.5	Several useful SessionStream presheaves	41
9.5.1	Raw trace presheaf Trace	41
9.5.2	State presheaf State	41
9.5.3	Schema presheaf Schema	41
9.5.4	Predicate presheaf Law	41
9.5.5	Completion presheaf Comp	41
9.6	Parameter sufficiency via fibers	41
9.7	Missing coordinates	42
9.8	Presheaves and distributed knowledge	42
9.9	Implementing a presheaf interface	42
9.10	Exercises	43
10	Sheaves: Compatible Local Data That Glues	44
10.1	Covers	44
10.2	Matching families	44
10.3	The sheaf condition	44
10.4	The limit/equalizer form	45
10.5	A basic example: functions	45
10.6	A failing presheaf: bounded data	45
10.7	A SessionStream matching family	46
10.8	Existence, uniqueness, and protocol failures	46
10.8.1	No amalgamation	46
10.8.2	Multiple amalgamations	46
10.9	Sheaf condition versus eventual consistency	47
10.10	Sheaf thinking as an architecture review	47
10.11	Exercises	47
11	Hydration as a Gluing Theorem	48
11.1	Canonical prefixes and reducers	48
11.2	Snapshot soundness	48
11.3	The temporal cover	49
11.4	Gluing operation	49
11.5	Snapshot-plus-suffix theorem	49
11.6	Mapping the theorem to the WebSocket adapter	50
11.7	Why buffer overflow is mathematically honest	50
11.8	Duplicate suppression and the overlap	51
11.9	Trace evidence as a section	51
11.10	Failure catalogue	51
11.11	Exercises	52
12	Sites: Declaring What Counts as Local Coverage	53
12.1	Pretopology axioms	53
12.2	Coverage is a design claim	53
12.3	A SessionStream context category	54
12.4	Temporal covers	54
12.5	Aspect covers	54
12.6	API parameter covers	55
12.7	Pullback stability as least-privilege sanity	55
12.8	Grothendieck topologies and sieves	55

12.9 Site design workflow	56
12.10 Exercises	56
13 Repairing Local-to-Global Failure	57
13.1 The universal property	57
13.2 Intuition from germs	57
13.3 Software repairs that resemble sheafification	57
13.3.1 Canonicalization	57
13.3.2 Completion	58
13.3.3 Local enrichment	58
13.3.4 Joint witnessing	58
13.3.5 Coverage correction	58
13.4 Repair taxonomy for SessionStream	58
13.4.1 Snapshot inconsistency	58
13.4.2 Stable retry identity	58
13.4.3 Hidden projector input	58
13.4.4 Projection cursor/materialization gap	58
13.4.5 Hydration buffer overflow	59
13.5 Separated reflection versus existence repair	59
13.6 Local equivalence and observability	59
13.7 A repair decision tree	59
13.8 Sheafification and APIs	60
13.9 Do not repair by hiding the failure	60
13.10 Exercises	60
14 Toposes: A Universe of Context-Dependent Sets	62
14.1 Elementary topos	62
14.2 Subobjects as predicates	62
14.3 The subobject classifier	63
14.4 Truth values in a presheaf topos	63
14.5 A SessionStream sieve	63
14.6 Power objects	64
14.7 Presheaf topos versus sheaf topos	64
14.8 Yoneda interlude: know an object by its probes	64
14.9 Internal objects are not global structs	65
14.10 What a topos does not give automatically	65
14.11 Exercises	65
15 Local Truth and Internal Reasoning	67
15.1 Stages and generalized elements	67
15.2 Forcing notation	67
15.3 Logical connectives, operationally	68
15.3.1 Conjunction	68
15.3.2 Implication	68
15.3.3 Disjunction	68
15.3.4 Existential quantification	68
15.3.5 Universal quantification	68
15.4 Why excluded middle can fail	68
15.5 Truth of snapshot consistency	69
15.6 Local existence versus global identity	69
15.7 Contextual authorization	69
15.8 Geometric reasoning	69
15.9 Internal language as specification notation	70
15.10 A local-truth worksheet	70
15.11 Exercises	70
16 Adjunctions and Change of Context	71

16.1	Definition by hom-set bijection	71
16.2	Unit and counit	71
16.3	Free structure examples	71
16.4	Event folds as universal interpretation	72
16.5	Sheafification as an adjunction	72
16.6	Substitution and quantifiers	72
16.7	Geometric morphisms	73
16.8	Software change-of-base reading	73
16.9	Geometric formulas and portability	74
16.10	Adjunction design questions	74
16.11	Exercises	74
17	Nerves: Turning Overlap into Shape	76
17.1	Simplicial complexes	76
17.2	Nerve of a cover	76
17.3	The SessionStream architecture complex	77
17.4	Pairwise contracts versus joint witnesses	77
17.5	Product dimensions	78
17.6	The Čech nerve	78
17.7	Good covers and inference from the nerve	78
17.8	Two ways a hole can disappear	79
17.9	Building a nerve from traces	79
17.10	Exercises	79
18	Cohomology: Measuring Persistent Incompatibility	81
18.1	Why coefficients must be additive	81
18.2	Cochains	81
18.3	The coboundary	82
18.4	Cocycles, coboundaries, and cohomology	82
18.5	Cursor calibration example	82
18.6	Exact versus closed	83
18.7	Incidence matrices	83
18.8	Graph cycle rank	84
18.9	Cellular sheaf version	84
18.10	A small calculation	84
18.11	What nonzero and zero mean	85
18.12	Computation workflow	85
18.13	Exercises	85
19	A SessionStream Obstruction Catalogue	87
19.1	Diagnostic matrix	87
19.2	Cursor holonomy	87
19.3	Projection progress and the missing face	88
19.4	Consistent-cut failure	88
19.5	Retry identity is usually a base problem	89
19.6	Deterministic replay as functoriality	89
19.7	Migration curvature	89
19.8	Hydration gaps	90
19.9	Observer evidence and epistemic failure	90
19.10	Local truth of failure suspicion	90
19.11	Constraint sheaves before linearization	91
19.12	Investigation workflow	91
19.13	Exercises	92
20	Laboratory: Build ss-sheafcheck	93
20.1	Scope	93
20.2	Package structure	93

20.3	Core data model	94
20.4	Model file	94
20.5	Trace extraction	95
20.6	Local validators	95
20.7	Restriction engine	96
20.8	Matching checker	96
20.9	Global-section search	97
20.10	Simplicial complex and cochains	97
20.11	Cursor-cycle example	97
20.12	Hydration example	98
20.13	Property and mutation tests	98
20.14	Four capstone projects	99
20.14.1	Capstone A: Hydration descent checker	99
20.14.2	Capstone B: Cursor coordinate atlas	99
20.14.3	Capstone C: Projection semantic presheaf	99
20.14.4	Capstone D: API sufficiency analyzer	99
20.15	Research notebook template	99
20.16	Final synthesis	99
20.17	Exercises	100
Appendix A: Categorical and Sheaf-Theoretic Cheat Sheet		101
A.1	Core category language	101
A.2	Universal constructions	101
A.2.1	Limit	101
A.2.2	Colimit	101
A.2.3	Standard cases	101
A.2.4	Pullback	102
A.2.5	Equalizer	102
A.2.6	Coequalizer	102
A.3	Functorial language	102
A.3.1	Functor	102
A.3.2	Natural transformation	102
A.3.3	Exponential	102
A.3.4	Adjunction	102
A.4	Presheaves and sheaves	103
A.4.1	Presheaf	103
A.4.2	Matching family	103
A.4.3	Sheaf	103
A.4.4	Site	103
A.5	Topos and local logic	103
A.5.1	Elementary topos	103
A.5.2	Subobject classifier	104
A.5.3	Presheaf truth values	104
A.5.4	Local truth	104
A.5.5	Geometric morphism	104
A.6	Nerves and cohomology	104
A.6.1	Nerve	104
A.6.2	Cochains	104
A.6.3	Cohomology	105
A.7	Diagnostic order	105
Appendix B: Goldblatt Route Map		106
B.1	Pedagogical features retained	107
B.2	Material intentionally omitted	107
B.3	Suggested parallel reading	107
Appendix C: SessionStream Source Map		109

C.1 Core model	109
C.2 Persistence and replay	109
C.3 Hydration transport	109
C.4 Example application	110
C.5 Open study obligations	110
C.6 Suggested repository annotations	110
Appendix D: Hints and Selected Solution Sketches	112
Chapter 1	112
1.3 Prefix law	112
1.5 Missing coordinate	112
Chapter 2	112
2.5 Parameter factorization	112
2.6 Duality	113
Chapter 3	113
3.4 Limits are unique up to unique isomorphism	113
3.7 Two-piece sheaf equalizer	113
Chapter 4	113
4.3 Snapshot race	113
4.7 Atomic witness choices	113
Chapter 5	114
5.3 Retry equivalence	114
5.5 Quotient factorization	114
Chapter 6	114
6.4 Constant on fibers iff factorization	114
6.6 Context-unstable callback	114
Chapter 7	114
7.1 Replay functor	114
7.6 Composite functors	114
Chapter 8	115
8.1 Projection migration square	115
8.6 Pointwise products	115
Chapter 9	115
9.2 Prefix restriction	115
9.5 Fiber ambiguity	115
Chapter 10	115
10.2 Equalizer derivation	115
10.7 Unique reconstruction	115
Chapter 11	116
11.1 Induction detail	116
11.3 Overtaking schedule	116
Chapter 12	116
12.1 Interval pretopology	116
12.6 Generated sieve	116
Chapter 13	116
13.1 Sheafification triangle	116
13.7 Retry is a two-stage repair	116
Chapter 14	117
14.1 Classifier in Set	117
14.6 Representable on cuts	117
Chapter 15	117
15.3 Refinement implication	117
15.5 Local retry witnesses	117
Chapter 16	117
16.1 Lists as free monoid	117
16.4 Fiber quantifiers in Set	117

Chapter 17	118
17.1 Boundary versus filled face	118
17.3 Atomic face	118
Chapter 18	118
18.2 Triangle boundary cohomology	118
18.6 Zero circulation	118
Chapter 19	118
19.3 Nonjoint sampling	118
19.5 A temporal gap is not automatically H^1	118
Chapter 20	119
20.2 Truncation restriction	119
20.3 Completion fibers	119

Appendix E: Glossary **120**

Appendix F: Further Reading and Study Continuation	123
F.1 Primary source for this custom route	123
F.2 Category theory foundations	123
F.3 Sheaves and topoi	123
F.4 Applied sheaves and computation	123
F.5 Contextuality and obstruction methods	123
F.6 Software-adjacent directions	124
F.7 Next mathematical milestones	124
F.8 Completion checklist	124

Preface

This is a custom study text for a software developer approaching abstract mathematics through a concrete system: **SessionStream**, a Go framework in which session-scoped commands produce canonical events, projections derive live and durable views, and reconnecting clients hydrate from a snapshot before receiving later live events.

The mathematical route is guided by Robert Goldblatt's *Topoi: The Categorical Analysis of Logic*, especially the transition from universal constructions and limits in Chapter 3 to topoi, functors, presheaves, sheaves, sites, adjunctions, and local truth in later chapters. The exposition, examples, diagrams, exercises, and software models here are original. Definitions from Goldblatt are paraphrased and reorganized around SessionStream rather than reproduced as a substitute for the source book.

The SessionStream study is anchored to repository commit d62dca9f5efa2e3094d6c62e5ead5ed0c88fd35c (13 August 2026), together with the Architecture Garden research note supplied in the conversation. The implementation will evolve. When code and book diverge, treat the code as the current artifact and this book as a mathematical model of a frozen study version.

What this book is for

The goal is not to decorate software with category-theory vocabulary. It is to develop a disciplined way to ask:

1. What are the *contexts* in which information is available?
2. What counts as a locally valid observation in each context?
3. How can a richer observation be restricted to a poorer one?
4. Which families of local observations are compatible?
5. When do compatible observations determine a unique global execution?
6. When does no global execution exist, and what shape does the obstruction have?
7. Which design changes add missing information, add a joint witness, or repair a bad local-to-global interface?

These questions lead naturally from limits to presheaves, from presheaves to sheaves, from sheaves to topoi and local logic, and finally to cohomological diagnostics.

Your entry point: Goldblatt Chapter 3, limits

This text assumes that the following ideas are at least familiar, even if they are not yet automatic:

- a category has objects, arrows, identities, and associative composition;
- an isomorphism is an arrow with a two-sided inverse;
- monic and epic arrows generalize injective and surjective functions;
- initial and terminal objects are characterized by unique arrows;
- products and equalizers are defined by universal properties;
- dual statements are obtained by reversing arrows.

You are currently at the point where Goldblatt unifies products, equalizers, and terminal objects as **limits of diagrams**. That is the correct place to begin the sheaf story. The sheaf gluing axiom can itself be expressed as a limit condition. The path is therefore not

category theory -> unrelated topology -> sheaves

but rather

limits -> compatible families -> gluing -> presheaves/sheaves -> local truth

Three readings of every construction

Most chapters use three layers.

Categorical layer. A precise definition in terms of objects, arrows, diagrams, universal properties, functors, or adjunctions.

SessionStream layer. A model using events, projections, cursors, snapshots, live suffixes, clients, schemas, and execution traces.

Engineering layer. A test, invariant, type boundary, transaction, protocol state machine, or analysis tool that could be implemented.

Do not collapse these layers. An analogy can motivate a definition without being a theorem. A formal model can expose a design problem without proving that the production implementation has it. An executable check can validate a finite trace without settling every possible execution.

Throughout the text, the following labels keep the distinction visible:

Definition. A mathematical definition used in the book.

SessionStream model. A deliberate formalization of part of SessionStream.

Engineering translation. A practical reading that may guide design or testing.

Caution. A boundary where the analogy or model should not be overstated.

A twelve-week route

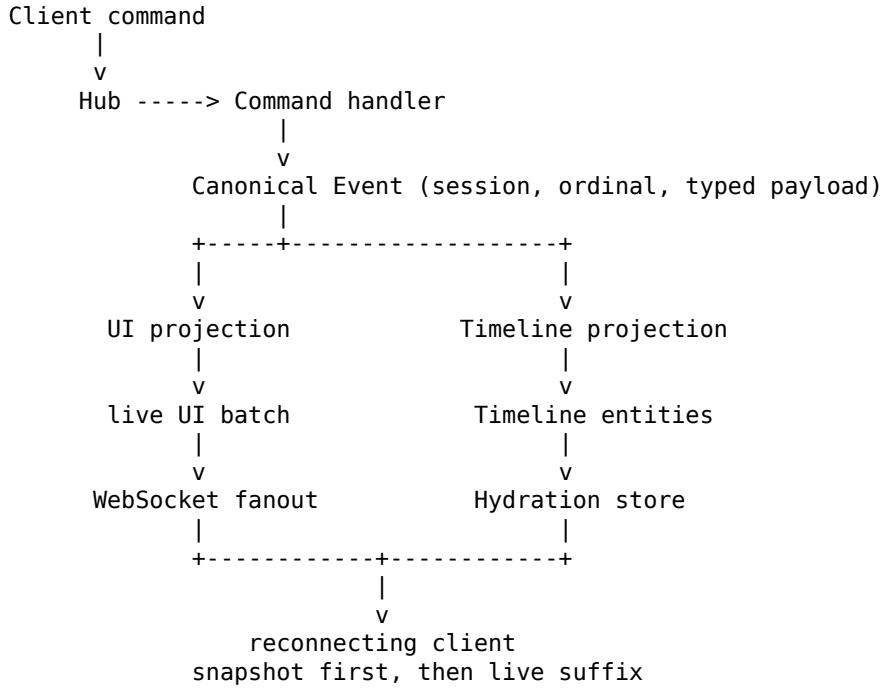
	Week	Main chapters	Deliverable
	1	1-2	Draw the SessionStream diagram and name every arrow
	2	3-4	Specify one universal property and one pullback test
	3	5-6	Model replay and projection as functorial behavior
	4	7-8	Define a category of observation contexts and a presheaf
	5	9	Write restriction laws and property tests
	6	10-11	Prove a snapshot-plus-suffix gluing proposition
	7	12-13	Design a SessionStream coverage policy and repair one failure
	8	14-15	Model contextual truth with sieves and subobjects
	9	16	Find an adjunction in a real API boundary
	10	17	Build the nerve of a small architecture cover
	11	18-19	Compute a small H^0 and H^1 example
	12	20	Implement a trace checker or design a capstone specification

A productive session is 60-90 minutes: read one subsection, redraw its diagrams, do two short exercises, and write one paragraph connecting the idea to an actual function or test in the repository.

Notation

- Categories are written $\mathcal{C}, \mathcal{D}, \mathcal{E}$.
- Objects are A, B, C ; arrows are $f : A \rightarrow B$.
- Composition is $g \circ f$, meaning first f , then g .
- A presheaf is usually $F : \mathcal{C}^{op} \rightarrow \mathbf{Set}$.
- For $V \subseteq U$, restriction is $\rho_V^U : F(U) \rightarrow F(V)$, often written $s \mapsto s|_V$.
- E_n is an event prefix through ordinal n .
- T_n is durable timeline state through ordinal n .
- S_n is a snapshot representing a cut at ordinal n .
- $L_{(n,m]}$ is a live suffix after n through m .
- $\Gamma(U, F)$ or simply $F(U)$ denotes sections over context U .
- H^k denotes cohomology only after an additive coefficient system has been specified.

The recurring SessionStream pipeline



The same event history is not copied unchanged into every component. It is interpreted through different projections and observed at different cuts. This family of partial, overlapping views is the main mathematical object of study.

Source and scope note

The Goldblatt-guided spine used here is:

- Chapter 3: universal constructions, limits, colimits, pullbacks, pushouts, finite completeness, exponentials;
- Chapter 4: subobjects, subobject classifiers, topoi, bundles, and sheaves;
- Chapter 9: functors, natural transformations, and functor categories;
- Chapter 14: presheaves (called stacks there), sheaves, sites, Grothendieck topoi, and local truth;
- Chapter 15: adjunctions and quantifier-like constructions;
- Chapter 16: geometric morphisms, internal logic, and geometric theories.

The chapters on nerves, cochains, and cohomology are an applied extension beyond Goldblatt’s main route. They are included because they make the user’s geometric intuition operational, but they are explicitly separated from the source book’s logical-topos development.

The SessionStream source map appears in Appendix C. Paths cited in the text refer to the frozen commit named above.

The System We Will Keep Rebuilding

A mathematical example becomes useful when it survives repeated changes of viewpoint. `SessionStream` will be treated not merely as an illustration but as a **study object** that we repeatedly reconstruct: first as a category of transformations, then as a diagram with limits, then as a presheaf of local observations, then as a sheaf-gluing problem, then as a site with contextual logic, and finally as a cellular object with cohomological diagnostics.

1.1 Canonical history and derived views

The central design move is that a command handler publishes a canonical backend event rather than returning a fully formed UI state. For a fixed session s , write the ordered event history as

$$E(s) = e_1 e_2 e_3 \cdots,$$

where each event has a session identifier, a typed name and payload, and an ordinal. In the current core type, an event has the conceptual shape

```
type Event struct {
    Name      string
    Payload   proto.Message
    SessionId SessionId
    Ordinal   uint64
}
```

The same event can be observed through multiple derived views:

$$P_{UI}(e_n, T_{n-1}) = \text{zero or more live UI events},$$

$$P_T(e_n, T_{n-1}) = \text{zero or more durable timeline entities}.$$

The projections receive the event and a read-only pre-event timeline view. Their codomains differ: live UI events are transport-facing and ephemeral, while timeline entities are durable projected state.

SessionStream model 1.1. For a session s , a projection step is a transition

$$(E_{n-1}, T_{n-1}) \xrightarrow{e_n} (E_n, T_n, U_n),$$

where E_n extends the canonical log, T_n is the new durable view, and U_n is the live UI output attributed to ordinal n .

This notation does not claim that all effects occur atomically in the implementation. It names the *global semantic step* whose realizability we will later examine.

1.2 The family of cursors

Several numbers can be called a cursor, but they express different claims:

- event cursor: the greatest canonical event ordinal present;
- timeline or snapshot cursor: the greatest ordinal represented in durable timeline state;
- projection cursor: the greatest ordinal a named projector claims to have processed;
- entity LastEventOrdinal: the greatest event reflected in one entity;
- live event ordinal: the event that produced one UI batch;
- client cut: the greatest ordinal the client has integrated.

Treating these as one untyped integer erases semantic distinctions. Later we will model each as a coordinate chart on a common execution and ask whether the transition maps between charts are coherent.

Engineering translation. A type alias such as `uint64` records representation. A categorical model records *meaning*: what assertion is made when this number is n ?

1.3 Hydration as a local-to-global protocol

A reconnecting client cannot simply subscribe to future events. It needs past durable state and future live updates without a gap or duplicate. SessionStream’s WebSocket adapter therefore uses a hydrating state:

1. register the subscription as hydrating;
2. begin loading a snapshot;
3. buffer UI batches arriving concurrently;
4. send the snapshot at cut n ;
5. discard buffered batches at or before n ;
6. send buffered batches after n in ordinal order;
7. flush any late buffer and transition to live delivery.

The intended semantic equation is

$$\text{reduce}(S_n, L_{(n,m]}) = C_m,$$

where C_m is the client-visible state through m .

This is our first local-to-global problem. The snapshot describes one region of history, the live suffix describes another, and their boundary is the cut n . We want them to glue into one coherent client execution.

1.4 The six law families

The Architecture Garden study identifies several obligations that will recur throughout the book.

1.4.1 Per-session serializability

Events within one session should have a coherent order, and state transitions should behave as if applied in that order.

1.4.2 Consistent-cut snapshots

Every entity returned in a snapshot at cut n should represent no event later than n :

$$\forall x \in S_n. \quad \text{last}(x) \leq n.$$

1.4.3 Stable retry identity

A redelivery of the same logical event should not be mistaken for a distinct event merely because it receives a new transport or storage coordinate.

1.4.4 Atomic projection progress

A projection checkpoint at n should not be visible unless the promised materialization through n is visible under the same commit semantics.

1.4.5 Deterministic replay

Replaying the same canonical prefix from the same initial state, under the same declared schema and projector version, should produce the same durable view.

1.4.6 Snapshot-plus-suffix completeness

A reconnect should cover every relevant event exactly according to the protocol's semantics: represented in the snapshot, delivered in the suffix, or intentionally projected away - never silently lost between regions.

These laws differ. Some are ordering properties, some are atomicity properties, some are identity properties, and some are determinism properties. A useful mathematical language should help distinguish them rather than rename them all "consistency."

1.5 The first context vocabulary

We will use the following aspect symbols:

Symbol	Aspect
E	canonical event log
T	materialized timeline view
P	projection progress/cursor
S	snapshot representation
U	live UI output
C	client-integrated state
R	schema and registry information
X	runtime trace or observer evidence

A preliminary observation context is a triple

$$(s, n, K),$$

where s is a session, n is an ordinal cut, and $K \subseteq \{E, T, P, S, U, C, R, X\}$ lists which aspects are visible.

Examples:

$(s, 42, \{E\})$ means event history for session s through 42,

$(s, 42, \{S, C\})$ means the snapshot/client boundary at cut 42,

$(s, 42, \{E, T, P\})$ means a joint event/materialization/progress context.

The definition is intentionally provisional. Later we will decide which contexts exist, which arrows connect them, and which families count as covers. Those choices are part of the model, not consequences of the source code.

1.6 Study discipline: classify the failure first

When an invariant fails, ask which of the following occurred.

1. **Malformed local section.** One component's state violates its own local rules.
2. **Restriction failure.** Forgetting information is inconsistent or non-functorial.
3. **Mismatch on overlap.** Two locally valid observations disagree about shared facts.
4. **Ambiguous gluing.** Local data is compatible but admits multiple global completions.
5. **Impossible gluing.** Local data passes some checks but no global completion exists.
6. **Missing coordinate.** The model omits information needed to distinguish executions.
7. **Missing joint context.** The architecture never observes or commits a needed combination atomically.
8. **Linearized obstruction only.** A cohomological diagnostic finds a discrepancy in a chosen additive summary, not necessarily the entire nonlinear system.

This taxonomy prevents premature claims that every distributed bug is a “topological hole.”

1.7 Exercises

1.1. Pipeline reconstruction. Without looking at the diagram in the preface, draw the command-to-client pipeline. Label each arrow as one of: routing, publication, projection, persistence, fanout, hydration, or reduction.

1.2. Coordinate semantics. For each cursor listed in Section 1.2, write a sentence of the form: “value n means that ...”. Identify two pairs that could differ during a partial failure.

1.3. Prefix law. Let E_n denote the event prefix through n . State three algebraic laws a prefix operation should satisfy. One should express idempotence and one should express nesting.

1.4. Snapshot counterexample. Construct a snapshot with `SnapshotOrdinal = 10` and two entities that is locally well-typed but violates consistent-cut semantics. Explain why typing alone cannot reject it.

1.5. Missing coordinate. Give an example in which replaying the same event prefix gives different output because the projector reads an undeclared input. Is this primarily a gluing obstruction or a modeling omission?

1.6. Repository trace. In `pkg/sessionstream/hub.go`, list the externally visible effect boundaries in event processing. Mark which pairs would need joint atomicity to make “projection cursor n ” a strong materialization claim.

1.7. Model criticism. The context triple (s, n, K) assumes an ordinal cut is enough to describe time. Name one concurrency model in which this is too weak and propose a richer coordinate.

Categories as Composable Semantics

A category is not a bag of types and functions. It is a discipline for talking about transformations through their domains, codomains, identities, and composition. This is exactly the information needed to state that two software routes have the same externally meaningful result.

2.1 Definition and engineering reading

Definition 2.1 (Category). A category $\mathcal{C}$ consists of:

- a collection of objects;
- for each pair A, B , a collection $\mathcal{C}(A, B)$ of arrows $f : A \rightarrow B$;
- an identity arrow $1_A : A \rightarrow A$ for every object;
- a composite $g \circ f : A \rightarrow C$ whenever $f : A \rightarrow B$ and $g : B \rightarrow C$;

satisfying associativity and the left and right identity laws.

The software temptation is to say “objects are types and arrows are functions.” That gives the category of types and total functions, but it is only one model. For SessionStream, useful categories may have objects such as event prefixes, schema versions, observation contexts, protocol states, database snapshots, or sets of valid executions.

An arrow should be read as a transformation whose composability matters. Examples include:

- reduce an event prefix to a timeline view;
- restrict a rich observation to a smaller aspect set;
- truncate an observation from cut n to cut $m \leq n$;
- encode a typed event as wire data;
- migrate a schema- v_1 value to schema v_2 ;
- interpret a trace as a set of invariant facts.

2.2 A prefix category

Fix a session s . Let $\mathbf{Cut}_s$ have natural-number cuts as objects. Put one arrow

$$m \longrightarrow n$$

exactly when $m \leq n$.

This is a category because $m \leq m$ gives identities and transitivity gives composition. It is also a **poset category**: there is at most one arrow between any pair of objects.

There are two possible conventions, both useful:

- arrows point forward, from a smaller prefix to a larger prefix;
- arrows point backward, from a richer observation to a poorer observation.

The first convention models extension of history. The second models restriction of knowledge. Presheaves will reverse arrows, so choosing the base orientation carefully prevents later confusion.

For this book we normally let the base category express inclusion of contexts:

$$V \hookrightarrow U$$

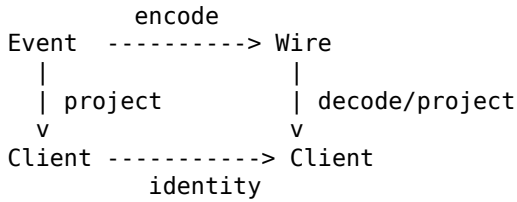
means that V is a smaller region or poorer observation than U . A presheaf then supplies a restriction map

$$F(U) \rightarrow F(V).$$

2.3 Commutative diagrams as route independence

A diagram commutes when every directed route with the same start and finish denotes the same composite arrow.

Consider a canonical event payload e , an encoded wire frame w , and a client value c :



Commutativity says that projecting before transport and projecting after an encode/decode round trip agree, subject to whatever semantic equivalence the category uses.

In ordinary software engineering, this appears as:

- serializer round-trip laws;
- migration compatibility squares;
- cache and source-of-truth agreement;
- replay equivalence;
- API adapter coherence;
- refactoring laws.

Caution. A diagram does not commute because the boxes are connected on a slide. It commutes only after arrows have precise semantics and equality has been specified.

Sometimes equality should be weakened: byte-for-byte equality, protobuf semantic equality, equality after canonicalization, observational equivalence, or isomorphism. That choice determines the category or higher structure in which the diagram lives.

2.4 Isomorphism and representation independence

Definition 2.2 (Isomorphism). An arrow $f : A \rightarrow B$ is an isomorphism when there is $g : B \rightarrow A$ with $g \circ f = 1_A$ and $f \circ g = 1_B$.

Two isomorphic objects are interchangeable with respect to categorical structure, although they need not have identical representations.

A JSON event and an in-memory protobuf event might be isomorphic only under restrictions: known schemas, canonical numeric handling, no unknown-field loss, and a specified encoding convention. If either direction loses information, the relation is not an isomorphism.

This is the first lesson in avoiding diagrammatic overclaiming: the existence of an encoder and decoder does not prove an isomorphism. The triangle equations do.

2.5 Monic and epic as cancellation properties

Definition 2.3. An arrow $m : A \rightarrow B$ is monic when $m \circ f = m \circ g$ implies $f = g$. An arrow $e : A \rightarrow B$ is epic when $f \circ e = g \circ e$ implies $f = g$.

In **Set** these coincide with injective and surjective functions. In other categories they are defined by cancellation, not element-wise behavior.

For software, a monic adapter preserves distinctions visible to upstream arrows. An epic adapter is sufficient to determine downstream behavior. These readings are suggestive, but always return to cancellation equations.

Suppose a request parameter projection

$$r_P : \text{GlobalState} \rightarrow \text{Parameters}$$

is monic. Then parameters distinguish every global state. This is stronger than normally required. To decide one invariant I , it is enough that I factor through r_P :

$$I = \bar{I} \circ r_P.$$

This factorization says that the invariant depends only on supplied parameters, even if the full state does not.

2.6 Duality

The opposite category $\mathcal{C}^{op}$ has the same objects and every arrow reversed. A theorem in $\mathcal{C}$ yields a dual theorem in $\mathcal{C}^{op}$.

Software dualities are rarely perfect implementation recipes, but they sharpen questions:

- producer versus consumer;
- read versus write;
- fanout versus aggregation;
- product versus coproduct;
- validation by equalizing versus quotienting by coequalizing;
- pullback integration versus pushout amalgamation.

Goldblatt repeatedly asks the reader to dualize definitions and proofs. Keep that practice. It trains you to recognize which direction of information flow a construction actually uses.

2.7 A category of SessionStream contexts

We can now refine the preliminary contexts from Chapter 1.

SessionStream model 2.4. Let $\mathcal{O}$ be a category whose objects are observation contexts (s, n, K, v) , where s is a session, n is a cut, K is a set of visible aspects, and v records relevant schema/projector versions. There is an arrow

$$(s, m, J, v') \longrightarrow (s, n, K, v)$$

when the first context is included in the second according to declared rules: typically $m \leq n$, $J \subseteq K$, and v' is the restriction of v .

This category is not “found” in the repository. It is designed. Different choices answer different questions. For example:

- omit versions if studying only one immutable deployment;
- include connection identity for transport-local claims;
- replace scalar ordinals by vector clocks for concurrent producers;
- include authorization scope when visibility is principal-dependent.

The quality of later sheaf reasoning depends on this base category.

2.8 Exercises

2.1. Category check. Show that session cuts with one arrow $m \rightarrow n$ when $m \leq n$ form a category. Identify identities and composition.

2.2. Two orientations. Redraw the cut category with arrows pointing toward less information. Which orientation makes truncation covariant? Which makes extension covariant?

2.3. Commuting square. Design a square involving an event schema migration and a projection. State exactly what equality would make it commute.

2.4. Non-isomorphic codec. Give a realistic protobuf-to-JSON setting in which encode and decode exist but are not inverse isomorphisms.

2.5. Parameter factorization. Let `authorize(command, user, session)` be an invariant. Propose a parameter object P and a factorization showing that the invariant can be decided from P without reconstructing full global state.

2.6. Dualize. Write the dual of: “a terminal object has exactly one arrow into it from every object.” Name a software situation that resembles each direction.

2.7. Context design. Add one coordinate to (s, n, K, v) needed to reason about authorization. Define when an arrow between two enriched contexts should exist.

2.8. Equality choice. List four different equalities that could be used for timeline entities. Which one is appropriate for deterministic replay tests?

Limits: The Best Joint Witness

Goldblatt’s move at this point is to recognize that products, equalizers, and terminal objects have the same pattern. Each is universal among cones over a diagram. This is the key conceptual bridge to sheaves: a compatible family of local sections forms a cone, and a glued section is characterized by a universal property.

3.1 Diagrams and cones

Definition 3.1 (Diagram). A diagram D in $\mathcal{C}$ is a collection of objects and specified arrows between them, with a fixed shape.

Formally, the shape is a small category J and the diagram is a functor $D : J \rightarrow \mathcal{C}$. You do not need that level of abstraction immediately; what matters is that the shape records which objects and arrows must be considered together.

Definition 3.2 (Cone). A cone from an object N to a diagram D is a family of arrows $p_j : N \rightarrow D(j)$ such that every triangle required by an arrow in D commutes.

The object N is the apex. It is a joint witness: from N we can consistently observe every object in the diagram.

Definition 3.3 (Limit). A limit of D is a cone (L, π_j) such that every other cone (N, p_j) factors through it by exactly one arrow $u : N \rightarrow L$:

$$\pi_j \circ u = p_j$$

for all j .

The phrase “best joint witness” is useful if interpreted precisely:

- **joint**: it maps consistently to every object of the diagram;
- **best**: every other joint witness factors through it;
- **not arbitrary**: the factorization is unique.

A limit, when it exists, is unique up to unique isomorphism compatible with its projections.

3.2 Familiar limits

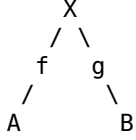
The standard examples are worth internalizing as one pattern.

3.2.1 Terminal object

The limit of the empty diagram is a terminal object. With nothing to observe, the universal cone is an object receiving exactly one arrow from every object.

3.2.2 Product

The limit of two disconnected objects A and B is their product:



Every pair (f, g) factors uniquely through $A \times B$.

3.2.3 Equalizer

For parallel arrows $f, g : A \rightrightarrows B$, the equalizer $e : E \rightarrow A$ is universal among arrows on which f and g agree:

$$f \circ e = g \circ e.$$

In **Set**, E is the subset of inputs where $f(a) = g(a)$.

3.2.4 Pullback

For $f : A \rightarrow C$ and $g : B \rightarrow C$, the pullback is the limit of the cospan:

$$A \text{ ---- } f \text{ ---- } > C < \text{---- } g \text{ ---- } B$$

It consists of compatible pairs whose images in C agree.

3.3 Product versus pullback in APIs

A product $A \times B$ combines arbitrary independent values. A pullback

$$A \times_C B$$

combines only values agreeing under maps to a shared interface C .

For example:

- A : a snapshot record carrying `(sessionId, snapshotOrdinal, entities)`;
- B : a buffered live batch carrying `(sessionId, eventOrdinal, events)`;
- C : a boundary description such as `(sessionId, schemaVersion)`.

The product contains every snapshot/live pair. The pullback contains only pairs that agree on shared boundary facts. If the maps to C also encode cut constraints, the pullback can enforce more.

Engineering translation. A pullback is a typed join with an agreement condition, characterized universally rather than by one particular data representation.

3.4 Equalizers as invariant subsets

Suppose two ways of interpreting a trace produce a claimed projection cursor:

$$\text{storedCursor}, \text{derivedCursor} : X \rightrightarrows \mathbb{N}.$$

Their equalizer selects traces on which the stored and derived claims agree.

Likewise, suppose replay and live execution both map canonical event histories to timeline state:

$$R, L : E^* \rightrightarrows T.$$

The equalizer is the set of histories on which deterministic replay agrees with the live path.

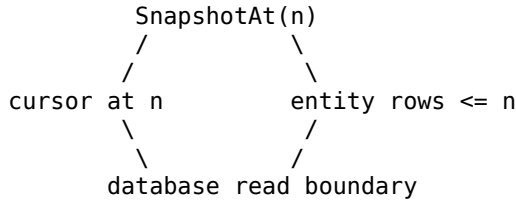
This does not prove that either path is correct with respect to product semantics. It proves agreement between two interpretations. The chosen codomain T and equality on it matter.

3.5 A consistent snapshot as a limit problem

Let Q_n be the database query context “read a session at cut n .” Let

- C_n be the cursor observation;
- V_n be the set of entity versions visible through n ;
- B_n be the shared database snapshot/transaction boundary.

A coherent snapshot should be a joint witness mapping to both cursor and entity observations under one boundary:



If the cursor and rows are read in unrelated database states, we may have two local observations but no cone whose apex is one consistent cut. A read transaction can be understood as constructing a stronger apex from which both reads factor.

Caution. The transaction is not literally a categorical limit in the Go implementation. The limit model specifies the semantic witness we want; a database transaction is one implementation technique for realizing it.

3.6 Finite completeness and system composition

A category is finitely complete when every finite diagram has a limit. A standard result emphasized by Goldblatt is that a terminal object plus pullbacks suffices to construct all finite limits.

This matters conceptually because complex compatibility conditions can be assembled from repeated pullbacks. If each component exposes maps to shared contracts, finite systems of agreement can often be represented as one iterated limit.

For software modeling, ask:

1. What are the local observation objects?
2. What shared codomains express overlap facts?
3. What pullbacks select compatible combinations?
4. What equalizers impose equation-like invariants?
5. Does the intended category actually contain these limits?

A category of raw runtime values may contain them, while a category of realizable transactional states may not. Existence is a substantive claim.

3.7 Preview: the sheaf condition as an equalizer

Let U be covered by contexts U_i . A presheaf F gives local-section sets $F(U_i)$ and overlap sets $F(U_i \cap U_j)$. A matching family is a tuple (s_i) whose two restrictions agree on every overlap.

The set of matching families is the equalizer of two maps:

$$\prod_i F(U_i) \rightrightarrows \prod_{i,j} F(U_i \cap U_j).$$

One map restricts s_i to the overlap; the other restricts s_j . The sheaf condition says that restriction from global sections gives a bijection

$$F(U) \cong \text{Eq} \left(\prod_i F(U_i) \rightrightarrows \prod_{i,j} F(U_i \cap U_j) \right).$$

This is why limits are not a detour. They are the algebraic skeleton of gluing.

3.8 Universal-property proof pattern

When proving that a proposed object L is a limit:

1. specify the projection arrows from L ;
2. prove they form a cone;
3. take an arbitrary cone (N, p_j) ;
4. construct a mediating arrow $u : N \rightarrow L$;
5. prove the required triangles commute;
6. prove uniqueness of u .

In software terms, steps 4-6 are often the hard part: construct one canonical integration function and show that every implementation satisfying the interface must coincide with it.

3.9 Exercises

3.1. Product proof. In **Set**, prove that the Cartesian product with coordinate projections satisfies the universal property. Do not use element-pair notation until after stating the arrow equations.

3.2. Equalizer test. Define two functions from recorded SessionStream traces to cursor summaries. Describe their equalizer and one property-based test that samples it.

3.3. Pullback request. Let A be authorized session subscriptions, B be available snapshots, and C be session identifiers. Interpret $A \times_C B$.

3.4. Unique up to isomorphism. Prove that any two limits of the same diagram are uniquely isomorphic in a way that preserves their cone maps.

3.5. Snapshot cone. Formalize the cursor and entity-row diagram of Section 3.5. State exactly what data belongs at the apex and what the projection arrows return.

3.6. Finite-limit decomposition. Express the set of triples (e, t, p) satisfying

$$\text{session}(e) = \text{session}(t) = \text{session}(p)$$

and

$$\text{ord}(t) = \text{ord}(p)$$

as iterated pullbacks or an equalizer of product maps.

3.7. Sheaf preview. For a two-piece cover $U = U_1 \cup U_2$, write the equalizer diagram for matching pairs. What changes for three pieces?

3.8. Failed existence. Give a category of software artifacts in which a desired pullback does not exist because no artifact can jointly satisfy two contracts. Explain why constructing a struct with two fields does not solve the semantic problem.

Pullbacks, Cuts, and Transactional Witnesses

Pullbacks deserve more attention than a brief example because they are the basic finite-limit operation behind typed joins, change of context, invariant-preserving refinement, and the pullback-stability condition in the definition of a site.

4.1 Definition

Given arrows $f : A \rightarrow C$ and $g : B \rightarrow C$, a pullback is an object P with arrows $p_A : P \rightarrow A$ and $p_B : P \rightarrow B$ such that

$$f \circ p_A = g \circ p_B,$$

and for every X with arrows $x_A : X \rightarrow A$ and $x_B : X \rightarrow B$ satisfying the same equality, there is a unique $u : X \rightarrow P$ with

$$p_A \circ u = x_A, \quad p_B \circ u = x_B.$$

The square

$$\begin{array}{ccc} P & \xrightarrow{p_B} & B \\ | & & | \\ p_A & & g \\ | & & | \\ v & & v \\ A & \xrightarrow{f} & C \end{array}$$

commutes and is universal among commuting squares over the cospan.

In **Set**,

$$A \times_C B = \{(a, b) \in A \times B \mid f(a) = g(b)\}.$$

The equation selects compatible pairs, while the universal property says this representation captures every compatible pairing canonically.

4.2 Pullback as a typed join

A database join resembles a pullback when both tables map to the same key object and the result contains exactly agreeing pairs. The categorical statement is stronger than “execute SQL JOIN”: it specifies the result through maps and a universal property, independent of storage layout.

Let

- A be timeline entities with a `SessionId`;
- B be projection checkpoints with a `SessionId`;
- C be session identities.

Then $A \times_C B$ contains entity/checkpoint pairs belonging to the same session. Add another shared map to ordinals if equality of cut is required.

A common modeling mistake is to overload the shared object C with too little information. Joining only on `SessionId` does not ensure that entity state and cursor state belong to the same transaction or cut. A more accurate boundary object might be

$$C = \text{SessionId} \times \text{Cut} \times \text{CommitEpoch} \times \text{ProjectorVersion}.$$

The pullback can only enforce agreement on coordinates represented in the maps.

4.3 The SQLite snapshot study

The current SQLite store's snapshot path conceptually performs two reads:

1. obtain a snapshot cursor;
2. query current entity rows, or historical versions for an explicit asOf cut.

For current-state snapshots, the cursor and entity rows should correspond to one database state. Let

- R_C be the result of reading the cursor;
- R_E be the result of reading entity rows;
- D be the database-state or read-transaction context;
- $q_C : D \rightarrow R_C$ and $q_E : D \rightarrow R_E$ be the two observations.

The intended snapshot is induced by one $d \in D$:

$$S(d) = (q_C(d), q_E(d)).$$

If cursor and entity reads occur against different database states d_1 and d_2 , the pair may lie in the ordinary product $R_C \times R_E$ but outside the image of any joint observation map from D . This is a precise way to distinguish a **pair of values** from a **coherent pair of observations**.

The law

$$\max_{x \in \text{Entities}(S)} \text{LastEventOrdinal}(x) \leq \text{SnapshotOrdinal}(S)$$

is necessary but may not be sufficient. Two states can satisfy the inequality while still mixing entity values from different database moments. A stronger invariant may include a commit epoch, MVCC snapshot identifier, or proof that both reads were made in one read transaction.

Engineering translation. Use the weakest operational mechanism that realizes the semantic joint witness. In SQLite that may be a read transaction; in another store it may be an MVCC token, versioned query, or snapshot API.

4.4 Projection progress as a pullback of claims

Consider three observations for an event ordinal n :

- E_n : event n is durably appended;
- T_n : timeline changes caused by n are durably applied;
- P_n : projection progress claims completion through n .

The strong global state should contain triples compatible over a shared commit fact. A naive product

$$E \times T \times P$$

contains inconsistent combinations such as an advanced cursor with missing materialization. The desired state resembles an iterated pullback over a commit/witness object K :

$$E \times_K T \times_K P.$$

But what is K ? Possibilities include:

- a database transaction identifier;
- an outbox record;
- a compare-and-swap generation;
- a log position serving as one source of truth;
- a recovery state that explicitly represents partial progress.

The mathematical model forces this design question. If there is no shared witness object and no maps into it, the intended pullback has not been made operational.

4.5 Pulling back predicates

A predicate on C can be represented as a subobject $P \hookrightarrow C$. Given $f : A \rightarrow C$, pulling the subobject back along f gives the values of A satisfying the predicate after translation through f :

$$\begin{array}{ccc} f^*P & \xrightarrow{\quad} & P \\ | & & | \\ \downarrow & & \downarrow \\ A & \xrightarrow{\quad f \quad} & C \end{array}$$

For example, let $P \hookrightarrow \text{CursorPairs}$ be the predicate “event cursor equals projection cursor.” Pulling it back along a trace-summary function selects traces satisfying cursor agreement.

This is the categorical core of preconditions, refinements, database constraints, and authorization checks: a condition defined in one context is transported to another by pullback.

4.6 Base change

Pullback is often called **base change**. If $A \rightarrow C$ is a family of objects indexed by C , and $B \rightarrow C$ chooses a new indexing context, then $A \times_C B \rightarrow B$ is the same family viewed over B .

SessionStream examples include:

- restrict all-session observations to one session;
- restrict a global schema registry to event types used by one application;
- restrict a transport trace to one connection;
- restrict a projection law to one projector version;
- restrict a timeline view to one cut.

This viewpoint becomes central in sites and geometric morphisms: local information should remain local and lawful after a change of context.

4.7 Pullback-stable coverage preview

A coverage declaration says that local contexts $\{U_i \rightarrow U\}$ jointly cover U . A site requires coverage to be stable under pullback. Given any $V \rightarrow U$, the pulled-back family

$$\{V \times_U U_i \rightarrow V\}$$

must cover V .

Engineering reading:

If a family of observations is sufficient globally, then after restricting to a smaller session, principal, cut, tenant, or subsystem, the correspondingly restricted observations should still be sufficient there.

This can fail. A monitoring dashboard may cover system behavior globally only because it aggregates facts unavailable within one tenant boundary. Whether that should count as a site depends on the semantics being modeled.

4.8 A proof obligation for snapshot-plus-live

Let S be snapshots, L live suffixes, and B boundary descriptors. Suppose

$$b_S : S \rightarrow B, \quad b_L : L \rightarrow B.$$

A compatible snapshot/live pair is an element of $S \times_B L$. A reconstruction function is an arrow

$$r : S \times_B L \rightarrow C$$

to client states.

For unique reconstruction, prove:

1. every legal reconnect trace determines a compatible pair;
2. r is total on compatible pairs;
3. r respects ordinal order and projection semantics;
4. any client state satisfying the snapshot and suffix observations equals $r(s, l)$.

The fourth clause is the uniqueness portion often omitted from protocol prose.

4.9 Exercises

4.1. Universal proof. Prove the set-theoretic construction $A \times_C B$ satisfies the pullback universal property.

4.2. Under-specified base. Construct two entity/checkpoint pairs that agree on `SessionId` but should not be considered compatible. Add the smallest base coordinate that separates them.

4.3. Snapshot race. Write an interleaving in which the cursor read returns 42 and an entity query returns a row reflecting event 43. Identify the two database states involved.

4.4. Necessary versus sufficient. Give a mixed snapshot satisfying `lastEventOrdinal <= snapshotOrdinal` for every row but still failing to represent one coherent database state.

4.5. Predicate pullback. Define a predicate on live UI batches and pull it back along a function from WebSocket traces to batch summaries.

4.6. Base change. Model “all transport records for one connection” as a pullback. Name all four objects and arrows.

4.7. Atomicity design. Propose three different choices for the shared witness K in the projection-progress model. For each, describe the operational cost.

4.8. Coverage stability. Give an example of a globally sufficient observability cover that ceases to be sufficient after restricting to one tenant. Should the original declaration be rejected, or should the base category change?

Colimits: Variants, Quotients, and Amalgamation

Limits assemble compatible observations by mapping *into* a diagram. Colimits reverse the arrows: they assemble generated pieces by mapping *out of* a diagram. Goldblatt develops the dual notions of cocone and colimit, then treats coequalizers and pushouts. These constructions are especially useful for variant types, identification, normalization, and composition across shared interfaces.

5.1 Cocones and colimits

Definition 5.1 (Cocone). A cocone from a diagram D to an object N is a family of arrows $D(j) \rightarrow N$ compatible with every arrow in D .

Definition 5.2 (Colimit). A colimit is a cocone (L, ι_j) through which every other cocone factors by one unique arrow $L \rightarrow N$.

Products become coproducts, equalizers become coequalizers, pullbacks become pushouts, and terminal objects become initial objects under duality.

5.2 Coproducts and event variants

In **Set**, a coproduct is a disjoint union. Values retain a tag indicating which summand they came from:

$$A + B.$$

This is the categorical shape of a sum type or tagged union.

A SessionStream event payload universe can be modeled as a coproduct of typed schemas:

$$\text{EventPayload} = \text{UserAccepted} + \text{InferenceStarted} + \text{TokensDelta} + \text{InferenceFinished} + \dots$$

The event Name plus schema registry supplies the tag-to-prototype relationship. A top-level arbitrary object weakens the coproduct structure because it obscures which injection introduced the value and which eliminator cases are required.

Engineering translation. Concrete protobuf messages plus registered logical names behave more like an explicit coproduct than an untyped JSON blob.

The universal property of the coproduct is case analysis: to define a function from $A + B$ to X , define one function from A to X and one from B to X .

That is exactly how a projection handles event variants.

5.3 Coequalizers and identification

For parallel arrows $f, g : A \rightrightarrows B$, a coequalizer is an arrow $q : B \rightarrow Q$ satisfying

$$q \circ f = q \circ g$$

and universal among arrows that identify all pairs $f(a)$ and $g(a)$.

In **Set**, Q is obtained by quotienting B by the smallest equivalence relation forcing $f(a) \sim g(a)$.

This is not merely “deduplicate a list.” It is a universal identification: every interpretation that considers the forced pairs equivalent factors uniquely through the quotient.

5.4 Stable retry identity as a quotient question

Suppose D is the set of physical deliveries and L is the set of logical events. A stable identity function

$$q : D \rightarrow L$$

should map retries of the same logical event to one value.

One way to specify this is to define a relation object R of delivery pairs known to be retries of each other, with projections

$$\pi_1, \pi_2 : R \rightrightarrows D.$$

Then q should coequalize those projections:

$$q \circ \pi_1 = q \circ \pi_2.$$

The coequalizer D/R is the most general quotient enforcing retry equivalence.

This formulation exposes a critical implementation question: **what data generates R ?** If an event contains only (`SessionId`, `Ordinal`), a redelivery assigned a new ordinal may not be related to the original. The problem then precedes quotienting: the system lacks a stable coordinate from which the relation can be constructed.

Caution. “Retries should be equal” is not an implementation. You need an equivalence relation with operational evidence, such as a producer event ID, idempotency key, or stable bus message identity.

5.5 Quotients and well-defined projections

If a projection is defined on physical deliveries but should depend only on logical events, it must be invariant under retry equivalence:

$$d_1 \sim d_2 \implies P(d_1) = P(d_2).$$

Then, and only then, there is a unique induced projection

$$\bar{P} : D/R \rightarrow T$$

with

$$P = \bar{P} \circ q.$$

This is the familiar “well-defined on equivalence classes” proof from quotient sets, restated as a universal factorization.

In code review, the corresponding question is:

If I replace this delivery by an equivalent retry, can any downstream observable change?

If yes, the operation does not descend to logical-event semantics.

5.6 Pushouts and amalgamation

Given arrows $f : A \rightarrow B$ and $g : A \rightarrow C$, a pushout is the universal object formed by joining B and C while identifying the two images of A :

$$\begin{array}{ccc}
 A & \xrightarrow{\quad f \quad} & B \\
 | & & | \\
 g & & v \\
 | & & P \\
 v & & \\
 C & \xrightarrow{\quad \quad} & P
 \end{array}$$

In **Set**, construct the disjoint union $B + C$ and identify $f(a)$ with $g(a)$ for each $a \in A$.

Software interpretations include:

- merge two schema extensions sharing a common base;
- combine two configuration fragments that agree on inherited fields;
- integrate two partial traces sharing a common prefix;
- compose independently developed adapters over one interface.

A pushout may force identifications that expose conflict. If both branches attach incompatible meaning to the same shared value, the categorical pushout in the chosen category may not represent a valid business object, or may not exist in a category restricted to valid artifacts.

5.7 Event-history branching

Let E_n be a shared event prefix, and let B and C be two hypothetical continuations. A pushout-shaped operation asks for an amalgamated history receiving both branches while identifying their common prefix.

For a totally ordered canonical event log, two distinct events claiming the same next ordinal may conflict. In a category of arbitrary graphs, a pushout can keep both branches. In a category of valid SessionStream histories with one event per ordinal, the pushout may fail to exist unless a conflict-resolution rule is added.

This example teaches a general lesson:

Universal constructions are relative to a category. Changing what counts as an object or arrow changes which limits and colimits exist.

5.8 Colimits and architecture boundaries

Limits are often associated with consistency and colimits with construction, but the division is not moral. Both can encode correctness.

- A coproduct preserves distinctions between event variants.
- A coequalizer intentionally erases retry distinctions.
- A pushout composes extensions while preserving a common interface.
- A pullback restricts combinations to shared-boundary agreement.

A robust architecture often alternates them:

```
variants --coproduct--> canonical event universe
                        |
                        v
                    projection and restriction
                        |
                        v
retries --coequalizer--> logical effects
                        |
                        v
snapshots/live --pullback/sheaf gluing--> client state
```

5.9 Exercises

5.1. Coproduct eliminator. State the universal property of $A + B$ and translate it into an exhaustive event projection.

5.2. Lost tag. Explain what information is lost when several event variants are decoded into one untyped map without a reliable tag. Which coproduct injection can no longer be recovered?

5.3. Retry relation. Define a relation on physical deliveries using a stable `EventId`. Prove it is an equivalence relation under reasonable assumptions.

5.4. Well-definedness. Let a metric increment once per delivery. Does it descend to the logical-event quotient? Modify the metric semantics so that it does.

5.5. Coequalizer factorization. Given a retry-invariant timeline projection $P : D \rightarrow T$, construct the induced $\bar{P} : D/R \rightarrow T$ and prove uniqueness.

5.6. Pushout conflict. Construct two event histories with a common prefix whose pushout does not exist in the category of single-valued ordinal histories.

5.7. Schema pushout. Model two protobuf schema extensions sharing a base message. What should the common object and inclusion arrows be? Name one compatibility condition the pure set-theoretic pushout would ignore.

5.8. Dualize. Dualize the universal-property proof pattern for a limit into one for a colimit.

Exponentials: Behavior as an Object

Topos theory requires more than finite limits. It needs a way to treat maps themselves as objects. Goldblatt introduces exponentiation after products and limits; later, a topos is characterized by finite limits, exponentials, and a subobject classifier. For software developers, exponentials connect function spaces, configuration, currying, and internal reasoning about behavior.

6.1 Definition

Let A and B be objects in a category with products.

Definition 6.1 (Exponential). An exponential B^A is an object equipped with an evaluation arrow

$$\text{ev} : B^A \times A \rightarrow B$$

such that for every $f : X \times A \rightarrow B$, there is a unique transpose

$$\lambda f : X \rightarrow B^A$$

satisfying

$$\text{ev} \circ (\lambda f \times 1_A) = f.$$

In **Set**, B^A is the set of all functions $A \rightarrow B$. The transpose is currying.

The universal bijection is

$$\mathcal{C}(X \times A, B) \cong \mathcal{C}(X, B^A),$$

natural in X .

A category with finite products and exponentials is cartesian closed.

6.2 Handlers as points of a function object

A command handler conceptually maps

$$\text{Command} \times \text{Session} \times \text{PublisherContext} \rightarrow \text{EffectResult}.$$

Currying separates configuration from invocation. An engine configuration $x \in X$ can choose a handler:

$$X \rightarrow \text{EffectResult}^{\text{Input}}.$$

Evaluation then applies that selected behavior to an input.

In Go, closures and interface values provide one implementation of this pattern. The categorical content is not that “functions are values”; it is the universal relationship between maps out of a product and maps into a function object.

6.3 Projections and declared dependencies

A deterministic timeline projection can be modeled as a point of

$$T^{E \times V \times M},$$

where

- E is a canonical event;
- V is the pre-event timeline view;
- M is declared metadata such as schema/projector version;
- T is a list of timeline-entity updates.

If the actual implementation also reads wall-clock time W , randomness R , or a mutable service Q , then its true type is closer to

$$T^{E \times V \times M \times W \times R \times Q}.$$

Pretending it belongs to the smaller exponential hides coordinates. Replay can then fail because the function being replayed was never actually a function of the declared inputs alone.

Engineering translation. Determinism is a typing claim about dependency closure: the output factors through the declared input object.

6.4 Evaluation and test generation

The evaluation map gives a systematic testing pattern. Let P be a set of candidate projectors and I a set of inputs. Then

$$\text{ev} : P \times I \rightarrow O$$

runs each projector on each input. Laws can be predicates on the evaluation result:

- same input twice gives equal output;
- equivalent retries give equal logical effect;
- extending a prefix by an irrelevant event leaves a projection unchanged;
- replay and live evaluation coincide.

Property-based testing samples points of products and checks equations between composites.

6.5 Exponentials are context-sensitive

In a presheaf topos, an exponential is not generally computed pointwise as “all raw functions at each context.” A section of B^A over context U must act coherently in every refinement $V \rightarrow U$. It is a **context-stable transformation**, not just a function on current local values.

This is crucial for software intuition. A locally available callback that works at one session/cut may fail to be natural under restriction. The internal function object contains behavior that remains meaningful as context changes.

We will revisit this when interpreting APIs and policies internally to a topos.

6.6 Cartesian closure and composition of programs

Cartesian closure supports a typed lambda calculus internally:

- products represent paired inputs;
- exponentials represent function types;
- evaluation represents application;
- transposition represents abstraction.

This is one reason topoi connect geometry and logic. They have enough categorical structure to interpret variables, substitution, conjunction-like products, and function formation, while a subobject classifier supplies a generalized object of truth values.

For SessionStream, this means the eventual topos viewpoint can contain not only states but also context-dependent transformations between states.

6.7 From exponentials to topoi

A practical modern definition is:

Definition 6.2 (Elementary topos, preview). An elementary topos is a category with finite limits, exponentials, and a subobject classifier.

Goldblatt initially presents finite completeness, finite cocompleteness, exponentiation, and a subobject classifier, then notes that finite cocompleteness follows from the other structure.

The three ingredients have distinct software readings:

- **finite limits:** combine and compare compatible local data;
- **exponentials:** treat coherent behavior as data internal to the category;
- **subobject classifier:** represent contextual predicates and truth values.

A topos is not simply “a nice category of software objects.” The axioms are strong, and the payoff is an internal higher-order intuitionistic logic.

6.8 Sufficient parameters as factorization through an exponential

Return to the question: are request parameters P sufficient to decide an invariant $I : X \rightarrow \Omega$ on global states?

Let $r : X \rightarrow P$ forget everything except supplied parameters. Sufficiency means there exists

$$\bar{I} : P \rightarrow \Omega$$

such that

$$I = \bar{I} \circ r.$$

By exponential transposition, a family of invariants parameterized by Y can be viewed either as

$$Y \times X \rightarrow \Omega$$

or as

$$Y \rightarrow \Omega^X.$$

This turns “which policy are we applying?” into a point of an internal predicate/function object.

6.9 Exercises

6.1. Currying. Write the set-theoretic bijection between functions $X \times A \rightarrow B$ and functions $X \rightarrow B^A$. Prove the two constructions are inverse.

6.2. Handler dependencies. List all inputs read by the chat-demo timeline projection. Separate declared parameters from values reachable through global state or services.

6.3. Hidden clock. Show how a call to `time.Now()` changes the mathematical domain of a projection. Give two ways to restore deterministic replay.

6.4. Factorization. For an authorization predicate $I : X \rightarrow \{0, 1\}$ and parameter projection $r : X \rightarrow P$, prove that I is constant on every fiber of r iff I factors through r .

6.5. Projector object. Define a finite set P of toy projectors and an evaluation function. Write one equation expressing replay/live agreement.

6.6. Context stability. Give a callback that is valid at cut 100 but cannot be restricted meaningfully to cut 50. Why should it fail to define a section of an exponential presheaf?

6.7. Topos ingredients. For each of finite limits, exponentials, and a subobject classifier, name one software capability it suggests and one reason the analogy may fail.

6.8. API policies. Model a family of request invariants as a map $Y \rightarrow \Omega^X$. What could Y represent in SessionStream?

Functors: Structure-Preserving Interpretations

Categories describe composable transformations. Functors translate one category into another without breaking identity or composition. Goldblatt's presentation emphasizes exactly these preservation laws before moving to natural transformations and functor categories. In software terms, a functor is not just an adapter between data types; it is an interpretation that respects the way transformations compose.

7.1 Definition

Definition 7.1 (Functor). A functor $F : \mathcal{C} \rightarrow \mathcal{D}$ assigns:

- to each object A of $\mathcal{C}$, an object $F(A)$ of $\mathcal{D}$;
- to each arrow $f : A \rightarrow B$, an arrow $F(f) : F(A) \rightarrow F(B)$;

such that

$$F(1_A) = 1_{F(A)}$$

and

$$F(g \circ f) = F(g) \circ F(f).$$

The two equations are the entire discipline. They say that doing nothing remains doing nothing, and staged work remains staged work after translation.

A contravariant functor reverses arrows. Equivalently it is an ordinary functor

$$F : \mathcal{C}^{op} \rightarrow \mathcal{D}.$$

Presheaves are contravariant set-valued functors.

7.2 A history category

Fix a session and consider a category $\mathcal{H}$:

- objects are valid event prefixes E_n ;
- an arrow $E_m \rightarrow E_n$ exists when E_m is a prefix of E_n ;
- composition concatenates compatible suffix extensions.

Now consider a category $\mathcal{T}$:

- objects are timeline states;
- arrows are valid state transitions induced by event suffixes.

A deterministic replay semantics may define a functor

$$R : \mathcal{H} \rightarrow \mathcal{T}.$$

Functoriality becomes two laws:

1. replaying an empty suffix leaves the view unchanged;
2. replaying suffix a and then suffix b equals replaying their concatenation.

Written algebraically, if fold applies a suffix to a view,

$$\text{fold}(T, \epsilon) = T,$$

$$\text{fold}(\text{fold}(T, a), b) = \text{fold}(T, ab).$$

This is stronger than saying `timelineProjection` is a Go function. It says the induced interpretation of history extension respects categorical composition.

7.3 Projection is not automatically a functor

The core projection interface consumes one event and a current view. Several things can prevent it from inducing a functor on histories:

- output depends on wall-clock time;
- output depends on external mutable state;
- the same event is interpreted differently after an undeclared deployment change;
- error policy skips a transition without making that choice part of the target category;
- retry behavior applies an effect twice;
- transition composition is order-sensitive but the source category identifies reorderings.

The lesson is methodological:

A type signature proposes object and arrow assignments. Functor laws are additional proof obligations.

A property-based replay test is a practical approximation:

```
choose prefix p and compatible suffixes a, b
left  = replay(replay(view(p), a), b)
right = replay(view(p), a ++ b)
assert observationallyEqual(left, right)
```

7.4 Encoding as a functor

Let $\mathcal{E}$ contain typed event values and schema-respecting transformations. Let $\mathcal{W}$ contain wire representations and wire-level transformations. An encoder can be functorial when it preserves identity transformations and composition of migrations or wrappers.

For a simpler one-object category, take event transformations as a monoid under composition. A representation map is then a monoid homomorphism precisely when it is a functor between the associated one-object categories.

This connects familiar engineering laws to category theory:

- middleware composition;
- parser combinator composition;
- schema migration chains;
- endomorphism pipelines;
- reducer composition.

7.5 Forgetful and summary functors

Many analyses deliberately forget structure.

A trace may be mapped to:

- its maximum event ordinal;
- a multiset of error kinds;
- a happens-before graph;
- a cursor vector;
- a Boolean invariant result.

To be functorial, the summary must map trace composition to the corresponding composition in the summary category. For example, maximum ordinal does not compose by addition; it composes by maximum under suitable monotone trace concatenation. Choosing the wrong target category makes a lawful summary appear non-functorial.

Engineering translation. When an aggregation law looks awkward, reconsider the algebra in the codomain. Counts compose by addition, latest timestamps by maximum, sets by union, and ordered traces by concatenation.

7.6 Composite interpretations

If

$$F : \mathcal{C} \rightarrow \mathcal{D} \quad \text{and} \quad G : \mathcal{D} \rightarrow \mathcal{E},$$

then $G \circ F$ is a functor.

A SessionStream interpretation may compose:

```
canonical event history
      | replay
      v
timeline entity history
      | snapshot serialization
      v
wire snapshot frames
      | client reducer
      v
client model
```

If each stage is functorial with respect to the chosen arrows and equalities, the composite is. This is the mathematical form of modular correctness: local preservation laws compose.

But if two stages use incompatible notions of identity or order, the composite model is invalid even when each implementation passes isolated unit tests.

7.7 Faithful, full, and essentially surjective

A functor is:

- **faithful** if it does not identify distinct arrows;
- **full** if every arrow between image objects comes from a source arrow;
- **essentially surjective** if every target object is isomorphic to one in the image.

These notions help characterize adapters.

A lossy event-to-UI projection is normally not faithful: many backend distinctions can produce the same UI behavior. That may be intentional. A serialization layer intended for exact replay should be much closer to faithful on the chosen event category.

Do not treat “not faithful” as a defect without considering the abstraction boundary.

7.8 Functoriality as executable law

A useful law-test harness needs:

1. generators for objects and composable arrows in the source category;
2. an implementation of F on both;
3. equality or observational equivalence in the target;
4. tests for identity and composition.

For history replay, objects are valid prefixes and arrows are valid suffixes. For context restriction, objects are contexts and arrows are inclusions. For schema migration, objects are versions and arrows are migrations.

This testing style exposes a common anti-pattern: generators produce arbitrary values but not *composable arrows*. Category-aware property testing should generate paths.

7.9 Exercises

7.1. Replay functor. Define the source and target categories needed for deterministic replay. State the two functor laws in code-like form.

7.2. Counterexample. Construct a projection that has the correct Go interface but violates the composition law for history extensions.

7.3. Codomain algebra. Define a functor that maps a trace to its set of error kinds. What is composition in the target category?

7.4. Lossy projection. Give two distinct backend events intentionally mapped to the same UI event. Explain why non-faithfulness is appropriate.

7.5. Serialization. State conditions under which protobuf JSON encoding could be faithful for a restricted category of messages. Name two features that threaten faithfulness.

7.6. Composite law. Prove that a composite of functors preserves identities and composition.

7.7. Path generator. Sketch a property-based generator for composable event-prefix extensions rather than arbitrary event lists.

7.8. Wrong category. Give a summary operation that appears to violate composition until the target category’s composition is changed.

Natural Transformations: Coherent Change Everywhere

A functor is one interpretation of a category. A natural transformation is a coherent way to move between two interpretations. This is a stronger idea than “a conversion function for every type”: all those conversions must commute with every relevant arrow.

8.1 Definition

Let $F, G : \mathcal{C} \rightarrow \mathcal{D}$ be functors.

Definition 8.1 (Natural transformation). A natural transformation $\eta : F \Rightarrow G$ assigns to every object A an arrow

$$\eta_A : F(A) \rightarrow G(A)$$

such that for every arrow $f : A \rightarrow B$ in $\mathcal{C}$, the naturality square commutes:

$$G(f) \circ \eta_A = \eta_B \circ F(f).$$

Diagrammatically:

$$\begin{array}{ccc} F(A) & \xrightarrow{F(f)} & F(B) \\ \eta_A \downarrow & & \downarrow \eta_B \\ G(A) & \xrightarrow{G(f)} & G(B) \end{array}$$

You may migrate first and then transform, or transform first and then migrate; the result agrees.

8.2 Schema migration as naturality

Suppose F interprets event histories using schema/projector version v_1 and G uses v_2 . A component η_{E_n} migrates the derived state for each prefix E_n .

Naturality requires migration to commute with every history extension $a : E_m \rightarrow E_n$:

$$G(a) \circ \eta_{E_m} = \eta_{E_n} \circ F(a).$$

Engineering reading:

$$\begin{array}{ccccc} \text{old state at } m & \xrightarrow{\text{migrate}} & \text{new state at } m & \xrightarrow{\text{apply new suffix}} & \text{new state at } n \\ \downarrow \text{apply old suffix} & & & & \downarrow \\ \text{old state at } n & \xrightarrow{\text{migrate}} & & & \end{array}$$

A one-time data migration that passes spot checks may still fail naturality if future updates interact differently with migrated state.

8.3 Replay implementation replacement

Let F be an in-memory hydration interpretation and G a SQLite-backed interpretation of the same abstract history category. A natural isomorphism

$$\eta : F \cong G$$

would mean every prefix state can be translated back and forth, and translation commutes with extension. This is stronger than obtaining equal snapshots after one test. It expresses backend substitutability across all modeled transitions.

In practice, exact isomorphism may be too strong because one backend stores tombstones or audit meta-data the other omits. Then choose a target category of observable client states and compare after a forgetful functor.

8.4 Natural transformations between presheaves

For presheaves $F, G : \mathcal{C}^{op} \rightarrow \mathbf{Set}$, naturality says that transformation commutes with restriction:

$$G(U \rightarrow V)(\eta_U(s)) = \eta_V(F(U \rightarrow V)(s)).$$

Read this as:

Convert a rich observation and then forget information, or first forget information and then convert; the local result is the same.

This is the coherence law required of context-aware migrations, redactions, canonicalizers, validators, and view adapters.

A redaction transformation, for example, should not reveal a value after restriction that it removed before restriction.

8.5 Ad hoc conversion versus naturality

Suppose every message type has a manually written migration. Why might the family fail to be natural?

- nested objects use a different version rule;
- list filtering occurs before migration in one path and after migration in another;
- truncating to an earlier cut changes how defaults are synthesized;
- field omission is interpreted differently across transports;
- schema registry lookup depends on ambient global state.

Naturality is a whole-family law. It catches incoherence that no component signature reveals.

8.6 Vertical and horizontal composition

Natural transformations compose vertically:

$$F \xRightarrow{\eta} G \xRightarrow{\theta} H$$

gives $\theta \circ \eta : F \Rightarrow H$ componentwise.

They also interact with functor composition horizontally. This supports reasoning about layered migrations and adapters without expanding every square.

For software versioning, a chain

$$v_1 \Rightarrow v_2 \Rightarrow v_3$$

should compose into the same migration semantics as a declared direct path, when both are supported. That is another commuting diagram rather than merely a version-number convention.

8.7 Functor categories

Functors $\mathcal{C} \rightarrow \mathcal{D}$ can themselves be objects of a category $[\mathcal{C}, \mathcal{D}]$, with natural transformations as arrows. Presheaves form the functor category

$$[\mathcal{C}^{op}, \mathbf{Set}].$$

This is already a topos for small $\mathcal{C}$. That fact is one of the main reasons presheaves are mathematically powerful: local-data assignments and coherent transformations between them live in a category with rich logical and universal structure.

Limits and colimits in a presheaf category are computed pointwise. This means, for example, that the product presheaf satisfies

$$(F \times G)(U) = F(U) \times G(U),$$

with restrictions applied componentwise.

Exponentials are subtler and not merely pointwise function sets.

8.8 A naturality review template

For an adapter η between two context-dependent representations:

1. list the base contexts U ;
2. list restriction arrows $V \rightarrow U$;
3. define $F(U)$ and $G(U)$;
4. define each component η_U ;
5. choose generators for sections $s \in F(U)$;
6. test

$$\eta_V(s|_V) = \eta_U(s)|_V.$$

This is a reusable architecture review for migrations, redaction, projection upgrades, and transport conversions.

8.9 Exercises

8.1. Naturality square. Write the naturality square for migrating a timeline state before or after applying event e_n .

8.2. Bad default. Construct a migration that fills a missing field with the current time. Show how this can violate naturality under prefix restriction.

8.3. Backend equivalence. Choose an observable codomain in which in-memory and SQLite hydration stores might be naturally isomorphic. State what information is intentionally forgotten.

8.4. Redaction. Define presheaves of full and redacted observations. State a naturality law for redaction.

8.5. Version chain. Draw a diagram comparing direct migration $v_1 \rightarrow v_3$ with the composite $v_1 \rightarrow v_2 \rightarrow v_3$. What equality should hold?

8.6. Pointwise product. Prove that products in a presheaf category can be computed pointwise.

8.7. Non-pointwise exponential. Explain why an arbitrary function $F(U) \rightarrow G(U)$ need not define a section of G^F over U .

8.8. Test harness. Write pseudocode for checking naturality of a snapshot canonicalizer across random cut restrictions.

Presheaves: Data Indexed by Context

A presheaf packages two ideas:

1. every context has its own collection of possible local data;
2. data can be restricted coherently when context is reduced.

Goldblatt uses the term “stack” or “pre-sheaf” for a contravariant functor from the open sets of a topological space to sets. Modern usage usually reserves “stack” for a richer object and calls the set-valued construction a presheaf. We will use **presheaf**.

9.1 Definition

Definition 9.1 (Presheaf). A presheaf of sets on a category $\mathcal{C}$ is a functor

$$F : \mathcal{C}^{op} \rightarrow \mathbf{Set}.$$

For each context U , $F(U)$ is the set of sections over U . For each arrow $i : V \rightarrow U$, there is a restriction function

$$F(i) : F(U) \rightarrow F(V).$$

We write $s|_V$ when the arrow is understood.

Functoriality becomes the restriction laws:

$$s|_U = s,$$

$$(s|_V)|_W = s|_W \quad \text{whenever } W \rightarrow V \rightarrow U.$$

A presheaf does **not** yet promise that compatible local data glues globally.

9.2 What is the base category?

For topological sheaves, objects are open sets and arrows are inclusions. In software, the base can be any category of contexts appropriate to the problem.

Candidate bases for SessionStream include:

9.2.1 Prefix contexts

Objects are cuts n . An inclusion $m \rightarrow n$ means $m \leq n$. Restriction truncates a trace or reconstructs a historical view.

9.2.2 Aspect contexts

Objects are subsets of observable aspects K . An arrow $J \rightarrow K$ means $J \subseteq K$. Restriction forgets fields or subsystems.

9.2.3 Product contexts

Objects are (s, n, K, v, p) containing session, cut, aspect set, version data, and perhaps principal. Arrows can reduce any coordinate according to declared rules.

9.2.4 Trace regions

Objects are finite subgraphs or intervals of a distributed execution. Restriction selects a subtrace.

9.2.5 API-information contexts

Objects are parameter sets or query scopes. Restriction forgets parameters.

There is no unique “topology of the software.” The base is part of the research question.

9.3 A presheaf of locally lawful observations

SessionStream model 9.2. For each observation context U , let

$$\text{Obs}(U)$$

be the set of assignments to all facts visible in U that satisfy the laws checkable entirely within U .

A section might record:

```
sessionId
cut
schema/projector version
event cursor
projection cursor
snapshot ordinal
entity summaries
live batch ordinals
client digest
trace evidence
```

Restriction may:

- discard aspects not in the smaller context;
- truncate event and live sequences to an earlier cut;
- filter records to one session or connection;
- erase fields unavailable to a principal;
- map a detailed entity to a summary digest.

The phrase “locally lawful” is essential. If every arbitrary assignment is allowed, gluing may detect only syntactic equality. Local validation encodes the constraints visible in each region.

9.4 Restriction is not arbitrary projection

A proposed restriction must satisfy identity and composition. Consider truncating a timeline snapshot from cut n to cut $m < n$.

If the store retains historical entity versions, restriction can select the latest version at or before m . If only current entity rows remain, true truncation may be impossible. Simply relabeling `SnapshotOrdinal` from n to m violates semantics.

Thus the existence of a restriction map is an architectural claim:

Can every section over the richer context be meaningfully viewed in the poorer context?

If not, either:

- the presheaf is defined on a smaller class of arrows;
- sections carry more history;
- restriction is partial, requiring a different categorical setting;
- or the proposed base category is wrong.

9.5 Several useful SessionStream presheaves

9.5.1 Raw trace presheaf Trace

$\text{Trace}(U)$ contains transport and projection records observable in U . Restriction filters records.

9.5.2 State presheaf State

$\text{State}(U)$ contains event, timeline, cursor, and client states satisfying local invariants.

9.5.3 Schema presheaf Schema

$\text{Schema}(U)$ contains the concrete protobuf contracts available in context U . Restriction forgets unavailable event kinds or fields.

9.5.4 Predicate presheaf Law

$\text{Law}(U)$ contains predicates meaningful using only information in U . Restriction or reindexing transports predicates along context arrows.

9.5.5 Completion presheaf Comp

$\text{Comp}(U)$ contains global executions compatible with the observation in U . This is useful for parameter sufficiency but may be large or noncomputable.

Different presheaves answer different questions on the same base.

9.6 Parameter sufficiency via fibers

Let X be a global context and $P \rightarrow X$ a parameter context. Restriction gives

$$r_P : F(X) \rightarrow F(P).$$

For a request observation $p \in F(P)$, its fiber is

$$r_P^{-1}(p) = \{x \in F(X) \mid x|_P = p\}.$$

Interpretation:

- empty fiber: the parameters are inconsistent with every global state;
- singleton fiber: they determine the full modeled state;
- multiple points: they leave global state underdetermined.

For an invariant $I : F(X) \rightarrow V$, full determination is unnecessary. Parameters are sufficient for I when I is constant on the fiber:

$$\forall x, y \in r_P^{-1}(p), \quad I(x) = I(y).$$

Equivalently, when this holds for every p , I factors through r_P .

This formalizes “are these API parameters enough?” without yet using sheaf cohomology.

9.7 Missing coordinates

Suppose two projector executions have identical declared context but different outputs. Before invoking topology, test whether the base omitted a relevant coordinate:

- wall-clock time;
- random seed;
- service response;
- deployment version;
- feature flags;
- authorization principal;
- producer identity;
- logical event ID.

If adding a coordinate makes restriction and determinism well-defined, the main problem was model under-specification.

A presheaf is only as honest as its base category.

9.8 Presheaves and distributed knowledge

A component’s local state is not necessarily a restriction of one existing global state. During races and failures, several local sections may coexist that cannot be globally reconciled. Presheaf language accommodates this: it defines what local sections and restrictions mean without assuming a global section exists.

This is why presheaves are more faithful than a single global struct. A struct starts by assuming that all fields coexist. A presheaf starts with context-indexed possibilities and asks whether coexistence can be justified.

9.9 Implementing a presheaf interface

A minimal executable model might use:

```
type Context struct {
    Session string
    Cut      uint64
    Aspects  AspectSet
    Version  string
}

type Section struct {
    Facts map[FactKey]Value
}
```

```

type Presheaf interface {
    Sections(Context) []Section
    Restrict(from, to Context, s Section) (Section, error)
}

```

Then test:

```

restrict(U, U, s) == s
restrict(V, W, restrict(U, V, s)) == restrict(U, W, s)

```

For real systems, enumerating all sections is impossible. Replace it with symbolic constraints, trace-derived sections, database queries, or generators.

9.10 Exercises

9.1. Base design. Define a poset of contexts containing session, cut, and aspect set. State precisely when $V \rightarrow U$ exists.

9.2. Restriction laws. Define restriction for event-prefix traces and prove identity and composition.

9.3. Impossible truncation. Explain why a store retaining only current entity rows may fail to define restriction to an earlier cut.

9.4. Two presheaves. On the same context category, define a raw-trace presheaf and a locally-valid-state presheaf. How do their section sets differ?

9.5. Fiber analysis. For a charge API with parameters (`orderId`, `amount`), construct two global completions giving different currency semantics. Add one parameter that makes the invariant constant on the fiber.

9.6. Hidden dependency. Model a feature flag as a missing coordinate. Show how adding it restores a well-defined projection function.

9.7. Property testing. Sketch generators for chains $W \rightarrow V \rightarrow U$ and sections $s \in F(U)$ to test restriction composition.

9.8. Partial restriction. Give a software example where restriction is partial. Propose either a smaller base category or a totalized result type.

Sheaves: Compatible Local Data That Glues

A presheaf tells us how to restrict. A sheaf tells us when local pieces are exactly the restrictions of one global piece. Goldblatt formulates this as a compatibility-and-amalgamation condition and later shows that the global section object is a limit of the overlap diagram. This chapter makes that statement the center of the software bridge.

10.1 Covers

For topological spaces, an open cover of U is a family $\{U_i \subseteq U\}$ whose union is U . On a general site, a cover is a declared family of arrows

$$\{U_i \rightarrow U\}$$

satisfying coverage axioms discussed in Chapter 12.

A software cover should mean:

The family of contexts jointly contains enough local information, according to the model, to account for the target context.

This is not the same as every field appearing somewhere. Coverage is semantic and must be justified.

10.2 Matching families

Let F be a presheaf and $\{U_i \rightarrow U\}$ a cover.

Definition 10.1 (Matching family). A family of sections $s_i \in F(U_i)$ is matching when, for every pair i, j , their restrictions to the overlap agree:

$$s_i|_{U_i \times_U U_j} = s_j|_{U_i \times_U U_j}.$$

For open sets, the pullback is the intersection $U_i \cap U_j$.

Pairwise agreement is not just equality of shared field names. The overlap object decides what is semantically shared.

10.3 The sheaf condition

Definition 10.2 (Sheaf). A presheaf F is a sheaf for a coverage when every matching family over every cover has a unique amalgamation $s \in F(U)$ satisfying

$$s|_{U_i} = s_i$$

for all i .

The condition has two parts:

- **existence:** compatible local sections have a global glue;
- **uniqueness:** two global sections with the same local restrictions are equal.

A presheaf satisfying only uniqueness is often called separated. In software language, local observations distinguish global state, but they may not be jointly realizable.

10.4 The limit/equalizer form

Define two arrows

$$\alpha, \beta : \prod_i F(U_i) \rightrightarrows \prod_{i,j} F(U_i \times_U U_j)$$

by

$$\alpha((s_i))_{ij} = s_i|_{U_i \times_U U_j},$$

$$\beta((s_i))_{ij} = s_j|_{U_i \times_U U_j}.$$

The equalizer is the set of matching families. Restriction gives

$$\rho : F(U) \rightarrow \text{Eq}(\alpha, \beta).$$

The sheaf condition is that ρ is a bijection.

Equivalently, $F(U)$ is the limit of the diagram of local section sets and overlap restrictions. This is the direct continuation of Goldblatt's Chapter 3 limit machinery.

10.5 A basic example: functions

Let $F(U)$ be the set of functions from a region U to a set Y , with restriction by domain restriction. Compatible local functions glue uniquely by defining

$$f(x) = f_i(x)$$

for any i with $x \in U_i$. Compatibility ensures well-definedness; coverage ensures existence at every point; uniqueness is immediate.

This example is simple but structurally complete. Every software gluing proof should identify analogues of:

- where the global value is defined;
- why a local piece exists there;
- why choosing another local piece gives the same result;
- why no other global value can have the same restrictions.

10.6 A failing presheaf: bounded data

Let $F(U)$ be the set of bounded real-valued functions on U . Restriction is valid, so F is a presheaf. On an infinite cover, locally bounded functions may glue to an unbounded global function. The matching family has no section in $F(U)$.

Software analogue: each shard, page, or batch satisfies a local resource bound, but the union violates a global bound. Local validity and overlap agreement do not imply global validity when the global predicate is not local for the chosen coverage.

10.7 A SessionStream matching family

Consider a target client execution through cut m , covered by:

- U_S : durable snapshot state through n ;
- U_L : live UI output over $(n, m]$;
- U_R : schema/projector/reducer contract valid across the reconnect;
- optionally U_X : trace evidence that buffering and ordering followed protocol.

A section over U_S might contain

```
session = s
snapshotOrdinal = n
entities = [...]
schemaVersion = v
```

A section over U_L might contain

```
session = s
ordinals = n+1 ... m, subject to projection output
schemaVersion = v
```

The overlap includes at least session, boundary cut, and semantic version. Matching requires agreement there.

But pairwise agreement is not enough unless the cover and section definitions encode completeness. A suffix could omit event $n+3$ while still agreeing with the snapshot on the cut. To make gluing meaningful, either:

- the live section records a contiguous canonical range or explicit projection witness;
- trace evidence proves every relevant batch was represented;
- the target section permits projected-away events under a declared rule.

The sheaf model forces “enough information” into the cover rather than assuming it.

10.8 Existence, uniqueness, and protocol failures

10.8.1 No amalgamation

Examples:

- snapshot says cut 42, but a returned entity reflects event 43;
- snapshot and suffix use incompatible projector versions;
- the suffix begins at 44 with an unaccounted event 43;
- two local components claim different payloads for the same logical event;
- a projection checkpoint claims n while materialization remains at $n - 1$.

10.8.2 Multiple amalgamations

Examples:

- parameters omit currency, allowing multiple valid transactions;
- a snapshot contains only a digest insufficient to determine client ordering;
- logical retry identity is absent, so two deliveries may represent one event or two;

- schema defaults admit distinct global interpretations with the same local fields.

No amalgamation is inconsistency. Multiple amalgamations are ambiguity. Both are local-to-global failures, but they require different repairs.

10.9 Sheaf condition versus eventual consistency

A sheaf condition is not the same as eventual consistency.

- A sheaf is a mathematical property of a chosen context system, coverage, and section assignment.
- Eventual consistency is a temporal convergence guarantee under operational assumptions.

You may model eventually consistent replicas with a sheaf over sufficiently late or causally closed contexts, but the equivalence is not automatic.

Likewise, a database transaction can help realize a sheaf-like gluing law, but “uses transactions” does not prove the sheaf condition.

10.10 Sheaf thinking as an architecture review

For an invariant-sensitive workflow:

1. choose the target context U ;
2. list the local contexts U_i , actually observed or committed;
3. justify why they cover U ;
4. define sections and restriction maps;
5. write matching equations on every overlap;
6. construct the global amalgamation;
7. prove or test uniqueness;
8. classify failures as local invalidity, mismatch, nonexistence, or ambiguity.

This is already valuable without calculating cohomology.

10.11 Exercises

10.1. Functions sheaf. Prove that set-valued functions form a sheaf on a topological space.

10.2. Equalizer derivation. Derive the equalizer formula for a two-piece cover and generalize it to a finite family.

10.3. Separated but not sheaf. Find or construct a presheaf in which global sections are determined by restrictions but some matching family has no global section.

10.4. Snapshot overlap. Define the overlap object between a snapshot and a live suffix. Include enough coordinates to reject a projector-version change during reconnect.

10.5. Missing event. Build a matching family that passes a weak pairwise boundary check but omits an event. Modify the section or coverage definition so it no longer matches.

10.6. Ambiguous completion. Give an API parameter family with two global amalgamations that differ on a transactional invariant.

10.7. Unique reconstruction. State and prove a small theorem: given deterministic reducer r , a snapshot at n , and a complete ordered suffix through m , the reconstructed client state is unique.

10.8. Architecture review. Apply the eight-step review in Section 10.10 to projection cursor advancement.

Hydration as a Gluing Theorem

The snapshot-before-live protocol is the strongest SessionStream example because it contains every ingredient of sheaf reasoning: a target global execution, two temporal regions, an overlap boundary, matching conditions, a gluing operation, uniqueness, and explicit failure modes.

11.1 Canonical prefixes and reducers

Fix a session s . Let

$$E_n = (e_1, \dots, e_n)$$

be the canonical event prefix through ordinal n . Let

$$\text{fold}_T(T_0, E_n) = T_n$$

be deterministic timeline reduction, and let

$$\text{fold}_C(C_0, U_{1..n}) = C_n$$

be deterministic client reduction over projected UI events.

The UI event sequence need not contain one event per backend ordinal. A backend event may project to zero, one, or several UI events. Therefore completeness must be stated relative to projection semantics, not by requiring consecutive UI frames.

Let P_U map each backend transition to its UI batch:

$$P_U(e_k, T_{k-1}) = U_k.$$

Then the complete UI suffix after cut n through m is

$$U_{(n,m]} = U_{n+1} \cdots U_m,$$

including empty batches as semantic steps even when no wire frame is sent.

11.2 Snapshot soundness

A snapshot S_n is sound for cut n when it represents the durable state intended after processing E_n under a declared semantic environment v :

$$\text{decodeSnapshot}(S_n) = \text{fold}_T(T_0, E_n; v).$$

At minimum it should satisfy:

1. `SessionId` is s ;
2. `SnapshotOrdinal` is n ;
3. every entity's creation and last-event coordinates are at or before n ;
4. entity payloads decode under the declared schema;
5. the projector/reducer version used to interpret the snapshot is compatible with the live suffix;
6. tombstone and ordering semantics are represented consistently.

The first three are visible in current core types. Version coordinates may be implicit in deployment state; the mathematical model makes them explicit because gluing depends on them.

11.3 The temporal cover

Let the target region be the execution interval $[0, m]$. Cover it with

$$U_S = [0, n] \quad \text{and} \quad U_L = [n, m].$$

The overlap is the boundary $\{n\}$. In a richer site, the live region may be $(n, m]$ while a separate boundary object carries the cut and semantic environment.

Sections:

- $s_S \in F(U_S)$ is a sound snapshot section;
- $s_L \in F(U_L)$ is a complete ordered live section;
- restrictions to the boundary expose session, cut, schema version, projector version, and reducer contract.

Matching means these boundary descriptions agree.

11.4 Gluing operation

Define

$$\text{glue}(S_n, U_{(n,m]}) = \text{fold}_C(\text{clientize}(S_n), U_{(n,m]}).$$

Here `clientize` converts durable snapshot entities into the client's initial model. It is itself part of the semantic contract and should be versioned or proven compatible with live UI reduction.

The reconstruction equation is

$$C_m = \text{glue}(S_n, U_{(n,m]}).$$

11.5 Snapshot-plus-suffix theorem

Theorem 11.1 (Unique hydration reconstruction). Fix a session s , cuts $n \leq m$, and semantic environment v . Assume:

1. **snapshot soundness:** S_n represents the durable/client base state through n ;
2. **boundary agreement:** snapshot and suffix agree on s, n, v ;
3. **suffix completeness:** for every backend event e_k with $n < k \leq m$, the suffix contains exactly the UI output prescribed by $P_U(e_k, T_{k-1}; v)$, in backend-ordinal order;
4. **determinism:** snapshot decoding, projection, and client reduction are functions of their declared inputs;

5. **stable identity/idempotence policy:** duplicate physical deliveries are either absent or reduced according to a declared logical identity rule.

Then the matching pair $(S_n, U_{(n,m]})$ has a unique client-state amalgamation through m .

Proof sketch. Snapshot soundness fixes the client base C_n . Order and completeness provide a uniquely specified sequence of projected updates for ordinals $n + 1$ through m . Deterministic reduction yields one state after the first update, then one after the second, and so on; induction gives one C_m . Any other amalgamation with the same restrictions must have base C_n and perform the same declared transitions in the same order, so it equals C_m . $\square$

The theorem is modest but useful. It makes every hidden assumption visible. Remove one assumption and a counterexample appears.

11.6 Mapping the theorem to the WebSocket adapter

The current adapter realizes a careful operational sequence:

```

register subscription as hydrating
    |
    +-- concurrent UI batches are buffered
    |
load snapshot at n
send snapshot
filter buffered batches to ordinal > n
sort by ordinal
send buffered batches
while holding the subscription lock:
    flush late buffered batches
    mark subscription live

```

Each step supports a theorem assumption:

Implementation action	Semantic role
register before loading	prevent a gap during snapshot read
hydration buffer	retain local suffix evidence
filter ordinal > n	prevent snapshot/suffix double coverage
stable ordinal sort	impose canonical suffix order
locked final flush	prevent newer live batches overtaking late buffered ones
explicit overflow error	refuse to fabricate a global section after evidence loss

The implementation is not a proof of all assumptions. For example, snapshot soundness depends on the hydration store, and UI completeness depends on upstream event/projection/fanout behavior.

11.7 Why buffer overflow is mathematically honest

Suppose the hydration buffer reaches its bound and drops a batch. The system then has:

- a snapshot through n ;
- some later batches;
- no evidence that the missing batch is represented elsewhere.

Returning a “successful” client state would assert an amalgamation without a complete matching family. Closing the connection and requiring fresh hydration is a conservative response: abandon the failed gluing attempt and construct a new cover at a later cut.

This is a good example of mathematics informing failure policy. A bounded resource does not violate the model if exhaustion is surfaced as loss of the premises required to glue.

11.8 Duplicate suppression and the overlap

Filtering batches with ordinal $\leq n$ assumes backend ordinal is a valid identity boundary between snapshot and live paths. This prevents duplicate *coverage by cut*, but it does not solve stable logical retry identity when the same logical event can receive a new ordinal.

Thus two equivalence relations are involved:

1. temporal coverage equivalence: already represented by snapshot versus after-cut suffix;
2. logical event equivalence: distinct physical deliveries representing one event.

Conflating them creates subtle duplicates.

11.9 Trace evidence as a section

The transport observer can record stages such as subscription registration, snapshot load, snapshot send, buffered batches, flushes, and live transition. A recorded trace section can support an executable gluing checker.

For one subscribe attempt, derive:

```
sid
snapshotOrdinal
buffered ordinals
sent ordinals
transition-to-live position
overflow/error status
```

Check:

- subscription was registered before snapshot load began;
- snapshot was sent before any after-cut batch for that subscription;
- every buffered ordinal at or before the cut was excluded;
- every retained after-cut ordinal was sent in nondecreasing order;
- no direct live send overtook a late buffered send;
- overflow prevented successful subscription completion.

This validates a finite trace. It does not prove all possible schedules, but it turns the sheaf model into a concrete oracle.

11.10 Failure catalogue

Removed assumption	Possible result
sound snapshot	reconstructed state starts from fiction
boundary agreement	snapshot and suffix describe different semantics
complete suffix	no global section through the claimed cut
deterministic reducer	several amalgamations
stable identity rule	duplicate logical effects
ordering	different reductions from same batches
atomic live transition	race-dependent gaps or overtaking

These are better diagnostics than a generic “WebSocket synchronization bug.”

11.11 Exercises

11.1. Proof completion. Expand Theorem 11.1 into a formal induction on $m - n$.

11.2. Empty projection. Modify suffix completeness to handle backend events that project to zero UI frames. What trace evidence is needed to distinguish “projected away” from “lost”?

11.3. Overtaking schedule. Construct a concurrency schedule in which marking the subscription live before flushing the late buffer allows ordinal 45 to be sent before 44.

11.4. Fresh hydration. Explain why reconnecting after overflow can repair the cover. Which new cut changes?

11.5. Duplicate semantics. Give a case where filtering `ordinal <= snapshotOrdinal` prevents one duplicate but stable `EventId` is still needed for another.

11.6. Trace checker. Design a state machine that consumes observer records and accepts exactly the successful hydration traces satisfying the listed local laws.

11.7. Version mismatch. Construct a snapshot produced by projector v_1 and a suffix interpreted by v_2 that agree on session and cut but admit no coherent client state.

11.8. Alternative cover. Replace snapshot-plus-suffix with checkpoint-plus-event-replay. Define the cover, overlap, and gluing operation.

Sites: Declaring What Counts as Local Coverage

A sheaf needs a notion of cover. Topological open covers are one example, but software contexts rarely form literal open subsets of a physical space. A **site** supplies an abstract category of contexts together with coverage rules. Goldblatt presents these through a pretopology: identity, composition of covers, and pullback stability.

12.1 Pretopology axioms

Definition 12.1 (Coverage basis / pretopology). A pretopology on a category $\mathcal{C}$ assigns to each object U certain covering families $\{U_i \rightarrow U\}$ such that:

1. the identity family $\{1_U : U \rightarrow U\}$ covers U ;
2. if $\{U_i \rightarrow U\}$ covers U and each $\{V_{ij} \rightarrow U_i\}$ covers U_i , then the composites $\{V_{ij} \rightarrow U\}$ cover U ;
3. if $\{U_i \rightarrow U\}$ covers U and $V \rightarrow U$ is any arrow, then the pullback family $\{V \times_U U_i \rightarrow V\}$ exists and covers V .

A category equipped with such coverage is a site.

These axioms say:

- a context covers itself;
- local coverage can be refined in stages;
- coverage survives change of base.

12.2 Coverage is a design claim

In software, declaring $\{U_i \rightarrow U\}$ a cover asserts that the local contexts are jointly sufficient for the kind of global section being modeled.

Possible coverage policies:

- **temporal:** a prefix and a suffix cover a full execution interval;
- **aspect:** event log, timeline state, and projection cursor cover one materialization step;
- **replica:** quorum observations cover a replicated register under a stated consistency protocol;
- **API:** parameter groups and authoritative lookups cover the facts needed to decide a transaction;
- **observability:** logs, metrics, traces, and store snapshots cover a failure analysis;
- **authorization:** principal claims and resource policy context cover an access decision.

Coverage is relative to a presheaf. A family sufficient for a cursor-summary sheaf may be insufficient for a full-payload sheaf.

12.3 A SessionStream context category

A useful base object can be written

$$U = (s, I, K, v, p),$$

where

- s is a session or session family;
- $I = [a, b]$ is an ordinal interval;
- K is an aspect set;
- v is a semantic-version environment;
- p is a principal/visibility scope.

An arrow $V \rightarrow U$ means that V is a valid refinement or subcontext:

- narrower session set;
- subinterval;
- fewer visible aspects;
- compatible version restriction;
- no greater principal authority.

This orientation makes presheaf restriction run from U to V .

The exact category should be small enough to reason about and rich enough to contain required pullbacks. For an executable prototype, use a finite subcategory extracted from one trace.

12.4 Temporal covers

For $a \leq n \leq b$, declare

$$[a, n] \rightarrow [a, b], \quad [n, b] \rightarrow [a, b]$$

to be a cover when the overlap at n carries enough boundary data.

Refining the left interval at m gives

$$[a, m], [m, n], [n, b]$$

as a composite cover, satisfying transitivity.

Pullback along a subinterval $[c, d] \rightarrow [a, b]$ yields intersections such as

$$[c, d] \cap [a, n], \quad [c, d] \cap [n, b],$$

which should cover $[c, d]$.

This is the interval version of ordinary topology and gives an intuitive first site.

12.5 Aspect covers

For a projection step, declare contexts

$$U_E = \{E, R\}, \quad U_T = \{T, R\}, \quad U_P = \{P, R\}, \quad U_X = \{X, R\}$$

to cover

$$U = \{E, T, P, X, R\}$$

only if the overlap structure includes the facts needed to correlate them: session, ordinal, logical event identity, projector version, and commit evidence.

A weak cover that overlaps only on session and ordinal may falsely classify incompatible sections as matching. Strengthening overlaps or adding a joint context changes the site.

12.6 API parameter covers

Suppose an operation needs facts

$$X = \{\text{order, price version, quantity, currency, account authorization}\}.$$

A request supplies context P , while authoritative services supply contexts A_1, A_2 . Declaring

$$\{P, A_1, A_2\} \rightarrow X$$

a cover says these sources jointly suffice to reconstruct or decide the modeled global facts.

This turns API design into a coverage question:

- Which information must be client supplied?
- Which may be looked up?
- On what keys do contexts overlap?
- Are those keys versioned and stable?
- Does coverage remain valid under tenant or authorization restriction?

12.7 Pullback stability as least-privilege sanity

Suppose an observability cover works for all sessions, but after pulling back to one authorized principal it loses the event-log context while retaining only metrics. Then the family is not coverage-stable in the principal-sensitive site.

Possible responses:

1. do not declare the global family a cover;
2. alter the target sheaf to expose less detail;
3. add a principal-safe local source;
4. change the base so the global and restricted analyses are distinct objects without the problematic arrow.

This shows why site design and security design interact.

12.8 Grothendieck topologies and sieves

A pretopology lists covering families. A Grothendieck topology can instead declare certain **sieves** covering.

A sieve S on U is a collection of arrows into U closed under precomposition: if $V \rightarrow U$ is in S and $W \rightarrow V$ is any refinement, then $W \rightarrow U$ is also in S .

A sieve represents a downward-stable body of evidence or access paths. The sieve generated by a cover contains every arrow that factors through one of its members.

Sieves become the truth values of a presheaf topos. We will use them in Chapter 15.

12.9 Site design workflow

For one SessionStream law:

1. state the global claim;
2. choose target contexts at which the claim is meaningful;
3. define arrows as valid restrictions/refinements;
4. list proposed covers;
5. verify identity coverage;
6. verify transitive refinement;
7. verify pullback stability;
8. define local sections and overlap maps;
9. test the sheaf condition or record the failure.

Do not start with cohomology. A poorly designed site produces sophisticated answers to the wrong question.

12.10 Exercises

12.1. Temporal pretopology. Prove that interval subdivision covers satisfy the three pretopology axioms.

12.2. Aspect pullback. Pull an event/timeline/progress cover back to a context exposing only event and progress aspects. What is the resulting family?

12.3. Weak overlap. Construct two projection sections that match on `(session, ordinal)` but differ in projector version. Repair the site.

12.4. API cover. Design a cover for an API invariant involving an immutable order version and payment currency. State every overlap key.

12.5. Security failure. Give a cover that is not stable under restriction to a principal. Choose one of the four repairs in Section 12.7.

12.6. Generated sieve. For a two-member cover $U_1, U_2 \rightarrow U$, describe the generated sieve.

12.7. Coverage relativity. Give one family that covers cursor summaries but not full event payloads.

12.8. Finite prototype. Extract five contexts and their arrows from one SessionStream hydration trace. List all pullbacks needed by your proposed covers.

Repairing Local-to-Global Failure

A presheaf may fail the sheaf condition because compatible pieces do not glue or because gluing is not unique. Mathematics offers **sheafification**, a universal way to map a presheaf into a sheaf with the same local behavior in an appropriate sense. Software repair has several related forms, but they must not all be called literal sheafification.

13.1 The universal property

Let

$$i : \mathbf{Sh}(\mathcal{C}) \hookrightarrow \mathbf{PSh}(\mathcal{C})$$

be the inclusion of sheaves into presheaves on a site.

Definition 13.1 (Sheafification). A sheafification of a presheaf F is a sheaf aF and a natural map

$$\eta_F : F \rightarrow i(aF)$$

universal among maps from F to sheaves. For every sheaf G and natural map $F \rightarrow iG$, there is a unique map $aF \rightarrow G$ making the triangle commute.

Equivalently, sheafification is left adjoint to inclusion:

$$a \dashv i.$$

It makes the minimal universal repair needed to obtain a sheaf, relative to the chosen site.

13.2 Intuition from germs

For ordinary presheaves on a topological space, sheafification can be imagined in two stages:

1. identify sections that agree locally around every point;
2. add sections represented by compatible local data even when the original presheaf lacked a global representative.

The resulting object remembers local behavior while repairing separation and gluing.

The exact construction uses germs or a plus construction. For this book, the universal property matters more than the implementation.

13.3 Software repairs that resemble sheafification

13.3.1 Canonicalization

Identify representations that are locally indistinguishable under the declared observations. Examples: normalize protobuf JSON, sort order-insensitive fields, or quotient physical retries by stable logical identity.

13.3.2 Completion

Add a canonical global object for every compatible family. Examples: construct a client state from a sound snapshot and complete suffix; materialize a composite configuration from matching fragments.

13.3.3 Local enrichment

Add enough overlap information for compatibility to be meaningful: projector version, commit epoch, logical event ID, schema fingerprint.

13.3.4 Joint witnessing

Introduce a transaction, outbox record, MVCC token, or coordination cell that observes several facts together.

13.3.5 Coverage correction

Stop declaring a weak family to be a cover. Some information sets simply do not suffice.

Only the first two are close to the literal mathematical flavor of sheafification. The others change the base category, topology, or presheaf before sheafification.

13.4 Repair taxonomy for SessionStream

13.4.1 Snapshot inconsistency

Symptom: cursor and entity rows can come from different database states.

Likely repair: add a read-transaction/MVCC witness or query historical versions at one explicit cut.

Model change: strengthen the joint context and restrictions. This is not primarily sheafification of unchanged data.

13.4.2 Stable retry identity

Symptom: a redelivery with a new ordinal may be treated as a new logical event.

Likely repair: add stable EventId or idempotency key; construct a retry equivalence relation; quotient physical deliveries.

Model change: add a missing coordinate, then coequalize. Calling the original issue an H^1 obstruction would be premature.

13.4.3 Hidden projector input

Symptom: replay differs from live execution with identical declared prefix.

Likely repair: move the dependency into canonical event data, inject it explicitly, or include it in the semantic environment.

Model change: enlarge the base/input object so the projection is a function.

13.4.4 Projection cursor/materialization gap

Symptom: checkpoint advancement and materialization can be observed separately after failure.

Likely repair: atomic transaction, recoverable intermediate state, or checkpoint derived from authoritative materialization.

Model change: add a higher-order joint witness or change which state is considered globally valid.

13.4.5 Hydration buffer overflow

Symptom: suffix evidence is incomplete.

Likely repair: fail the current gluing attempt and hydrate again at a later cut, rather than inventing completion.

Model change: no permanent sheafification is required; operationally choose a new cover.

13.5 Separated reflection versus existence repair

A presheaf can fail uniqueness because two global sections have identical local restrictions. Repairing this identifies them. It can fail existence because a matching family has no global section. Repairing this adds a glue.

Software analogues:

- **uniqueness repair:** canonicalize equivalent payloads, quotient retries, identify observationally equivalent states;
- **existence repair:** add a reducer, merge procedure, transaction, or synthesized aggregate.

Do not apply existence repair when local data is actually inconsistent. Sheafification adds glue only to compatible local behavior as determined by the site.

13.6 Local equivalence and observability

Two global states may be distinct internally but indistinguishable under every cover in the site. Sheafification can identify them.

This resembles observational equivalence. But the equivalence is relative to the contexts and covers chosen. If the site omits audit context, two states differing only in audit metadata may become locally indistinguishable.

Thus sheafification can intentionally abstract, but it can also erase required evidence if the site is too coarse.

13.7 A repair decision tree

When gluing fails:

```

Is a local section itself invalid?
  yes -> strengthen local validation
  no
Do restrictions satisfy functor laws?
  no -> repair the presheaf/base
  yes
Do sections disagree on an overlap?
  yes -> reject family or enrich overlap semantics
  no
Are there several global completions?
  yes -> add coordinates or quotient by intended equivalence
  no
Is there no completion?
  yes -> add a joint witness, completion operation, or weaken coverage claim
  no

```

Does only a linear summary obstruct?

yes -> investigate; do not infer full inconsistency automatically

13.8 Sheafification and APIs

Given partial API data, a system often performs a completion:

```
request parameters
+ authenticated principal
+ authoritative database lookups
+ defaults/version rules
-----
validated command object
```

This resembles gluing when the inputs form a matching family. A robust API should make the universal choice explicit:

- reject inconsistent families;
- reject ambiguous completions unless ambiguity is intentional;
- produce one canonical validated object;
- ensure downstream invariants factor through it.

The validated command can be viewed as a sheaf-like global section over the operation context.

13.9 Do not repair by hiding the failure

Common anti-repairs include:

- taking the maximum cursor without proving corresponding state exists;
- last-write-wins across semantically incomparable versions;
- assigning a fresh ID to a retry and treating it as new;
- silently dropping a hydration batch;
- coercing unknown schemas into untyped maps;
- defaulting a missing transactional parameter from current mutable state.

These produce a value, but not necessarily an amalgamation of the original local sections.

13.10 Exercises

13.1. Universal triangle. Draw the universal-property diagram for sheafification and explain each arrow in words.

13.2. Two failure modes. Give one presheaf-like software model failing uniqueness and one failing existence.

13.3. Classify repairs. For each SessionStream case in Section 13.4, classify the repair as: add coordinate, strengthen overlap, add joint witness, quotient, completion, or change cover.

13.4. API completion. Design a canonical validated-command object for a payment operation. State when completion is impossible and when it is ambiguous.

13.5. Observational collapse. Give two execution states that are indistinguishable to clients but distinguishable to operators. What happens if operator context is omitted from the site?

13.6. Anti-repair analysis. Choose one anti-repair in Section 13.9 and construct a concrete counterexample.

13.7. Retry quotient. Explain why adding EventId and quotienting retries is a two-stage repair rather than one.

13.8. New cover. Show how fresh hydration after overflow replaces a failed temporal cover with a new one.

Toposes: A Universe of Context-Dependent Sets

A topos is a category that behaves enough like **Set** to support products, function objects, predicates, and an internal logic, while allowing truth and data to vary by context. Goldblatt's book develops this as the meeting point of category theory and logic. For software, a presheaf or sheaf topos provides a disciplined universe in which context-indexed states and coherent transformations are ordinary objects and arrows.

14.1 Elementary topos

Definition 14.1 (Elementary topos). An elementary topos is a category with finite limits, exponentials, and a subobject classifier.

As noted earlier, Goldblatt's initial presentation includes finite cocompleteness and later derives it from the other structure.

The axioms provide:

- terminal objects and pullbacks for finite contextual combination;
- exponentials for internal behavior/function objects;
- a classifier Ω for predicates/subobjects.

Every presheaf category

$$[\mathcal{C}^{op}, \mathbf{Set}]$$

for small $\mathcal{C}$ is a topos. The category of sheaves on a site is a Grothendieck topos.

14.2 Subobjects as predicates

A subobject of A is an equivalence class of monic arrows

$$m : S \hookrightarrow A.$$

In **Set**, this is just a subset $S \subseteq A$. It represents a predicate on elements of A .

SessionStream examples:

- valid events among all decoded payloads;
- sound snapshots among all snapshot-shaped values;
- traces satisfying hydration ordering;
- states in which event and projection cursors agree;
- commands authorized for a principal;
- timeline entities whose `LastEventOrdinal` is at most a cut.

Pulling back a subobject along $f : B \rightarrow A$ gives the predicate on B obtained by substitution through f .

14.3 The subobject classifier

Definition 14.2 (Subobject classifier). A subobject classifier is an object Ω with an arrow

$$\text{true} : 1 \rightarrow \Omega$$

such that every monic $m : S \hookrightarrow A$ occurs, up to isomorphism, as the pullback of true along a unique characteristic arrow

$$\chi_m : A \rightarrow \Omega.$$

Diagram:

$$\begin{array}{ccc} S & \xrightarrow{\quad\quad\quad} & 1 \\ | & & | \\ m & & \text{true} \\ | & & | \\ v & & v \\ A & \xrightarrow{\chi_m} & \Omega \end{array}$$

In **Set**, take $\Omega = \{\text{false}, \text{true}\}$. The characteristic function sends an element to true exactly when it belongs to the subset.

In a general topos, Ω can contain many contextual truth values.

14.4 Truth values in a presheaf topos

For a presheaf topos on $\mathcal{C}$, $\Omega(U)$ is the set of sieves on U . Recall that a sieve is a downward-closed collection of arrows into U .

Why is a sieve a truth value? Given a predicate on a section at stage U , collect every refinement $f : V \rightarrow U$ at which the restricted section satisfies the predicate. This collection is closed under further refinement, so it is a sieve.

Thus truth at U is not necessarily a single Boolean. It can be:

the family of refinements under which the statement becomes verified.

The maximum sieve contains every arrow into U and represents truth at U . The empty sieve represents no supporting refinement.

14.5 A SessionStream sieve

Let U be a broad observation context for a hydration attempt, and consider the predicate

$$\varphi = \text{"the reconnect is known to be gap-free."}$$

Different refinements may expose different evidence:

- $V_1 \rightarrow U$ reveals snapshot and sent ordinals;
- $V_2 \rightarrow U$ also reveals buffered ordinals;
- $V_3 \rightarrow V_2$ reveals observer linearization records;
- $V_4 \rightarrow U$ reveals only connection-level metrics.

The truth value of φ at U can be the sieve of refinements where enough evidence exists and the check passes. If V_2 supports the claim, every further refinement preserving that evidence should support it too.

This is more informative than a global Boolean “healthy.” It distinguishes absence of evidence from evidence of failure.

14.6 Power objects

In **Set**, the power set $\mathcal{P}(A)$ collects subsets of A . In a topos, the power object is

$$\Omega^A.$$

Its generalized elements correspond to subobjects of A . Internally, it is an object of predicates on A .

Software readings include:

- a context-dependent collection of validation policies;
- authorization rules over commands;
- sets of admissible traces;
- invariant suites parameterized by deployment context;
- query predicates stable under restriction.

Again, a section over U must be coherent under every refinement below U ; it is not merely an arbitrary host-language function.

14.7 Presheaf topos versus sheaf topos

A presheaf topos allows arbitrary local-data assignments with coherent restriction. A sheaf topos restricts to those satisfying the chosen gluing law.

Use a presheaf topos when studying:

- inconsistent or partial distributed observations;
- candidate schemas and traces before reconciliation;
- the full space in which gluing can fail.

Use a sheaf topos when studying:

- context-indexed objects that already satisfy descent;
- local definitions intended to determine global behavior;
- internal logic after a coverage policy is accepted.

The inclusion

$$\mathbf{Sh}(\mathcal{C}, J) \hookrightarrow \mathbf{PSh}(\mathcal{C})$$

has sheafification as a left adjoint.

14.8 Yoneda interlude: know an object by its probes

For an object A in $\mathcal{C}$, the representable presheaf

$$yA = \mathcal{C}(-, A)$$

assigns to each context U the set of arrows $U \rightarrow A$ - all ways U can probe or instantiate A .

The Yoneda lemma states

$$\mathbf{Nat}(yA, F) \cong F(A).$$

A section of F at A is equivalent to a coherent rule that turns every probe of A into a local section.

Software intuition:

An object is characterized by all context-respecting ways other objects can interact with it, not by opening its representation.

This resonates with interface-driven design, but Yoneda is stronger and exact: the full web of arrows determines the object up to isomorphism.

For SessionStream, a session context can be studied through commands, snapshots, subscriptions, cursor queries, and trace restrictions that map into it. The representable viewpoint encourages defining semantics by observable probes.

14.9 Internal objects are not global structs

An object F in a presheaf topos is a whole assignment

$$U \mapsto F(U)$$

plus restrictions. Its “elements” at different stages need not come from global elements $1 \rightarrow F$.

A local snapshot section may exist at one session/cut even when no global all-sessions state is available. This is not incomplete bookkeeping; it is the native ontology of the topos.

That is the conceptual payoff:

Context dependence is not metadata attached to otherwise global values. It is built into what an object is.

14.10 What a topos does not give automatically

A topos does not automatically provide:

- a correct choice of contexts or covers;
- temporal fairness or liveness;
- resource bounds;
- Byzantine fault tolerance;
- probability or quantitative latency;
- decidable equality or classical logic;
- a complete cohomological diagnosis;
- a production architecture.

It gives a powerful semantic environment. The engineering value comes from choosing the site and objects honestly.

14.11 Exercises

14.1. Classifier in Set. Prove that $\{0, 1\}$ with τ true classifies subsets in **Set**.

14.2. Sound snapshots. Define the subobject of sound snapshots inside all decoded snapshots. Describe its characteristic map in a set-based model.

14.3. Sieve closure. Prove that refinements supporting a stable predicate form a sieve.

14.4. Contextual truth. For a hydration trace, list four refinements and decide which belong to the sieve supporting “gap-free reconnect.”

14.5. Power object. Interpret a section of Ω^{Trace} as an invariant suite. What coherence under restriction is required?

- 14.6. Yoneda.** For a small poset category of cuts, compute the representable presheaf $y(n)$ explicitly.
- 14.7. Global versus local element.** Give a presheaf with a section at one context but no global section.
- 14.8. Scope check.** Choose one SessionStream property not naturally handled by elementary topos structure alone and identify additional mathematics or operational assumptions needed.

Local Truth and Internal Reasoning

Goldblatt’s later chapters develop truth inside a topos rather than only truth about it from the outside. The key shift is that propositions are interpreted as subobjects and evaluated at stages of context. For presheaves and sheaves, truth can be local, stable under refinement, and supported by covers.

15.1 Stages and generalized elements

A global element of an object A is an arrow

$$1 \rightarrow A.$$

A generalized element of A at stage U is an arrow

$$U \rightarrow A.$$

In a presheaf category, this corresponds naturally to context-indexed data. Reasoning “at stage U ” means reasoning with the information available in context U and all coherent refinements beneath it.

For SessionStream, a stage might be:

- one session through one cut;
- one connection during one hydration attempt;
- one projector version;
- one principal’s authorized observation scope;
- one recorded finite execution trace.

15.2 Forcing notation

We write

$$U \Vdash \varphi$$

to mean that formula φ holds at stage U under the internal semantics.

For a predicate represented by subobject $S \hookrightarrow A$ and a local element $a \in A(U)$, the statement $U \Vdash a \in S$ means that a factors locally through S at U .

Truth is monotone under restriction:

$$U \Vdash \varphi \implies V \Vdash \varphi|_V$$

for every $V \rightarrow U$.

This is why stable evidence forms a sieve.

15.3 Logical connectives, operationally

The precise Kripke-Joyal clauses depend on the site, but the engineering intuitions are:

15.3.1 Conjunction

$$U \Vdash \varphi \wedge \psi$$

when both claims hold at U .

15.3.2 Implication

$$U \Vdash \varphi \Rightarrow \psi$$

when at every refinement $V \rightarrow U$, whenever $V \Vdash \varphi$, also $V \Vdash \psi$.

Implication is therefore future-refinement stable, not a one-time truth-table lookup.

15.3.3 Disjunction

In sheaf semantics, $U \Vdash \varphi \vee \psi$ can hold when U has a cover such that on each covering piece one disjunct holds, even if no single disjunct holds uniformly over all of U .

15.3.4 Existential quantification

$$U \Vdash \exists x. \varphi(x)$$

can hold when there is a cover of U on whose pieces local witnesses exist. A single global witness need not be available.

15.3.5 Universal quantification

A universal claim must persist for all refinements and all local values there.

These clauses explain why sheaf logic is naturally intuitionistic and local.

15.4 Why excluded middle can fail

In ordinary Boolean logic,

$$\varphi \vee \neg \varphi$$

always holds. In a general topos it need not.

Suppose φ is “connection c has failed.” During silence shorter than the configured suspicion threshold, the current context may prove neither failure nor nonfailure. A later refinement with a matching pong supports nonfailure; another with timeout evidence supports suspicion. The broad earlier context supports neither disjunct globally.

This matches distributed-systems reality: lack of evidence is not necessarily evidence of the negation.

The SessionStream README carefully treats heartbeat timeout as suspicion under timing assumptions, not proof of remote process crash. That distinction is a good local-truth example.

15.5 Truth of snapshot consistency

Let

$$\varphi(S) = \forall x \in \text{Entities}(S). \text{last}(x) \leq \text{cut}(S).$$

At a context exposing only the snapshot header, $U \not\models \varphi(S)$ may coexist with $U \not\models \neg\varphi(S)$ because entity ordinals are hidden.

After refining to a context exposing all entity metadata, the predicate can be decided for that finite snapshot.

This demonstrates three distinctions:

- the proposition has a global mathematical meaning;
- a stage may lack enough information to force it;
- refinement can make evidence available without changing the underlying snapshot.

15.6 Local existence versus global identity

Suppose each shard of a trace contains a retry-correlation witness, but witnesses use shard-local IDs. A sheaf may validate

$$U \Vdash \text{“locally, every delivery has a correlation identifier”}$$

without producing one global identifier coherent across shards.

Existential witnesses that vary locally do not automatically glue. This is one reason the internal existential is weaker than choosing a global element.

In software, “every component can find some owner” does not imply there is one globally consistent owner.

15.7 Contextual authorization

Let A be commands and $S \hookrightarrow A$ the subobject of authorized commands. The characteristic arrow

$$\chi : A \rightarrow \Omega$$

returns not merely true/false but the sieve of refinements in which authorization can be established.

For a command submitted with incomplete identity context, the truth value may say:

- authorized after refining with a valid principal token and session policy;
- not supported by refinements lacking tenant membership;
- undecidable at the transport-only stage.

This models authorization as contextual evidence rather than a nullable Boolean. Actual systems should still fail closed operationally.

15.8 Geometric reasoning

Geometric logic uses finite conjunctions, arbitrary or suitably bounded disjunctions, and existential quantification in a form preserved by inverse-image functors of geometric morphisms. Goldblatt emphasizes that geometric statements transport well between topoi.

Engineering interpretation:

Prefer specifications constructed from local existence, finite compatibility, and cover-stable alternatives when you want them preserved across context-changing interpretations.

Not every operational property is geometric. Negation, universal quantification, exact cardinality, liveness, and global resource bounds may require additional care.

15.9 Internal language as specification notation

One can speak inside a topos as though objects were types and arrows were functions, using intuitionistic higher-order logic. For a SessionStream sheaf topos, internal statements might look like:

$$\forall S : \text{Snapshot}. \text{Sound}(S) \Rightarrow \forall x \in \text{entities}(S). \text{last}(x) \leq \text{cut}(S).$$

Externally, this formula denotes subobjects and arrows built from finite-limit, exponential, and classifier structure. The internal language compresses diagrams into readable specifications.

The danger is forgetting stage dependence. Internally quantified “all snapshots” means all generalized snapshots in all contexts, not only globally materialized Go values.

15.10 A local-truth worksheet

For a statement φ :

1. At which contexts is φ meaningful?
2. What local data witnesses it?
3. Is support stable under restriction/refinement?
4. What sieve is its truth value?
5. Can φ be decided at the current stage?
6. Can it hold only locally on a cover?
7. Do local witnesses glue to one global witness?
8. Is the operational policy fail-open, fail-closed, retry, or defer?

15.11 Exercises

15.1. Monotonicity. Prove that a sieve-valued truth assignment is stable under precomposition.

15.2. Undecided stage. Construct a context in which snapshot consistency is neither forced nor refuted. Give two refinements deciding it differently.

15.3. Implication. Interpret “if a subscription is live, then its snapshot has been sent” using the refinement-based implication clause.

15.4. Local disjunction. Give a cover on which each piece satisfies either “event projected to UI” or “event intentionally projected away,” without choosing one disjunct globally.

15.5. Local witnesses. Construct shard-local retry IDs that do not glue to a global logical event ID.

15.6. Failure suspicion. Explain why heartbeat timeout should support a proposition such as “suspected failed under policy p ” rather than “remote process crashed.”

15.7. Authorization sieve. Describe the sieve-valued characteristic arrow for one command and principal context.

15.8. Geometricity check. Classify these statements as plausibly geometric or not: “there exists a snapshot covering this prefix”; “every retry eventually succeeds”; “cursor equals either n or $n + 1$ ”; “no error ever occurs.”

Adjunctions and Change of Context

Adjunctions express an optimal relationship between two directions of translation. Goldblatt uses them to organize quantifiers and later geometric morphisms. In software, adjunctions clarify when one operation is the most economical way to add structure and another is the canonical way to forget or aggregate it.

16.1 Definition by hom-set bijection

Definition 16.1 (Adjunction). Functors

$$F : \mathcal{C} \rightarrow \mathcal{D}, \quad G : \mathcal{D} \rightarrow \mathcal{C}$$

form an adjunction $F \dashv G$ when there is a natural bijection

$$\mathcal{D}(F(A), B) \cong \mathcal{C}(A, G(B))$$

for all A, B .

F is the left adjoint and G the right adjoint.

The bijection says that maps out of the freely or optimally constructed $F(A)$ correspond exactly to maps from A into the underlying/context-changed $G(B)$.

16.2 Unit and counit

An adjunction determines natural transformations

$$\eta : 1_{\mathcal{C}} \Rightarrow GF$$

and

$$\varepsilon : FG \Rightarrow 1_{\mathcal{D}},$$

called unit and counit, satisfying triangle identities.

For a free/forgetful adjunction:

- η_A inserts generators into the underlying object of the free construction;
- ε_B evaluates the free structure generated by the underlying data back into B .

The triangle identities say that adding and then evaluating structure does not introduce arbitrary drift.

16.3 Free structure examples

Familiar examples include:

- free monoid on a set, left adjoint to forgetting the monoid structure;

- free group on a set;
- free category on a directed graph;
- free completion under a chosen class of colimits.

A SessionStream-flavored example is only an analogy unless the categories are specified. One might construct a free event history from generators, then interpret it in a concrete reducer. The universal property says any assignment of generators into a target monoid of transitions extends uniquely to a history interpretation.

This is why folds are ubiquitous: a list is a free monoid, and a reducer is the unique monoid homomorphism induced by its action on generators.

16.4 Event folds as universal interpretation

Let E^* be the free monoid of event sequences. Let M be a monoid of state transformations under composition. Assign each event e a transition

$$\tau(e) : T \rightarrow T.$$

The assignment extends uniquely to a monoid homomorphism

$$\bar{\tau} : E^* \rightarrow M.$$

Then replay is evaluation of the composite transition on initial state.

This gives a clean proof obligation:

If canonical event history is intended to determine state, every event constructor must have a declared transition, and sequence composition must map to transition composition.

Hidden effects violate the universal interpretation because the target transformation is not determined by the generator alone.

16.5 Sheafification as an adjunction

As introduced in Chapter 13,

$$a : \mathbf{PSh}(\mathcal{C}) \rightleftarrows \mathbf{Sh}(\mathcal{C}, J) : i$$

with

$$a \dashv i.$$

The left adjoint a makes the universal sheaf receiving a map from a presheaf. The right adjoint i forgets only the fact that the object satisfies gluing; it retains the underlying presheaf.

This is a genuine mathematical form of “repair with no more commitment than necessary.”

16.6 Substitution and quantifiers

A map $f : X \rightarrow Y$ induces reindexing of predicates by pullback:

$$f^* : \text{Sub}(Y) \rightarrow \text{Sub}(X).$$

In suitable categories, reindexing has adjoints:

$$\exists_f \dashv f^* \dashv \forall_f.$$

Interpretation:

- f^* substitutes along f ;
- $\exists_f$ hides an X -coordinate by existentially projecting evidence to Y ;
- $\forall_f$ asserts a predicate for all values in the fiber over Y .

For an API projection that forgets internal state,

$$f : \text{GlobalState} \rightarrow \text{RequestParameters},$$

$\exists_f$ can express “there exists a global completion satisfying the invariant,” while $\forall_f$ can express “every global completion satisfies it.”

This recovers the fiber criterion for parameter sufficiency:

$$\forall_f(I)$$

is true at parameters p precisely when all modeled completions over p satisfy I .

16.7 Geometric morphisms

Definition 16.2 (Geometric morphism). A geometric morphism $f : \mathcal{F} \rightarrow \mathcal{E}$ consists of an adjunction

$$f^* : \mathcal{E} \rightleftarrows \mathcal{F} : f_*$$

with $f^* \dashv f_*$ and f^* preserving finite limits.

The arrow direction is conventionally opposite the inverse-image functor: f points from $\mathcal{F}$ to $\mathcal{E}$, while f^* maps objects from $\mathcal{E}$ to $\mathcal{F}$.

Why preserve finite limits? Because finite-limit structure interprets equality, conjunction, substitution, and compatible tuples. A context change should preserve these geometric relationships.

16.8 Software change-of-base reading

Potential geometric-morphism-like situations include:

- restrict all-session semantics to one tenant or session;
- interpret server-side sheaves in a client observation site;
- move from full operator context to redacted principal context;
- compare a detailed trace site with a summary-observability site;
- transport schemas and invariants along a deployment or version map.

To claim a genuine geometric morphism, one must define two topoi, the adjoint functors, and prove left exactness. Most ordinary adapters will not satisfy this.

Still, the checklist is useful:

1. What does inverse image do to a local object?
2. What does direct image aggregate or forget?
3. Are they adjoint?
4. Does inverse image preserve terminal objects, pullbacks, and equalizers?

5. Which logical formulas survive the translation?

16.9 Geometric formulas and portability

Positive-existential/geometric formulas are built from atomic predicates using finite conjunction, disjunction in the allowed setting, and existential quantification. They are preserved by inverse-image functors of geometric morphisms.

Software implication:

Specifications expressed through finite local compatibility and existence are more likely to transport correctly across context changes than specifications using global negation, exact complements, or unrestricted universal claims.

For example, “locally there exists a snapshot and suffix agreeing on their boundary” is more geometric in flavor than “there is no possible hidden delivery anywhere.”

This is not a reason to avoid nongeometric properties. It tells you which properties require stronger proof when changing semantic environments.

16.10 Adjunction design questions

For a pair of operations that feel opposite - load/save, parse/print, restrict/extend, free/forget, abstract/concretize - ask:

- Is there a natural bijection of maps, not merely two functions?
- Which side makes the least committed construction?
- What are the unit and counit?
- Do triangle identities hold?
- Is one direction fully faithful?
- Which limits or colimits are preserved?

Adjunctions are common, but they should be discovered by universal property, not by naming any two-way conversion an adjunction.

16.11 Exercises

16.1. Free monoid. Prove that lists form the free monoid on a set and derive the unique fold homomorphism.

16.2. Event interpretation. Model event sequences as a free monoid and state conditions under which projection transitions define a monoid homomorphism.

16.3. Hidden effect. Show how an undeclared network call prevents the event-to-transition assignment from determining a unique homomorphism.

16.4. Quantifier fibers. For $f : X \rightarrow P$ and predicate $I \subseteq X$, describe $\exists_f(I)$ and $\forall_f(I)$ element-wise in **Set**.

16.5. Parameter sufficiency. Express “parameters p are sufficient to guarantee invariant I ” using $\forall_f$. Express “at least one valid completion exists” using $\exists_f$.

16.6. Sheafification adjunction. State the hom-set bijection corresponding to $a \dashv i$.

16.7. Change of base. Propose inverse/direct-image operations between an operator-observation site and a client-observation site. Identify one finite limit that inverse image may fail to preserve.

16.8. Geometric specification. Rewrite a SessionStream law in a positive-existential form if possible. Identify what information is lost compared with the original stronger law.

Nerves: Turning Overlap into Shape

The previous chapters treated covers algebraically. The **nerve** turns the overlap pattern of a cover into a simplicial shape. This is the bridge to the multidimensional topological intuition: vertices represent local contexts, edges represent pairwise overlap, triangles represent compatible three-way overlap, tetrahedra represent four-way overlap, and so on.

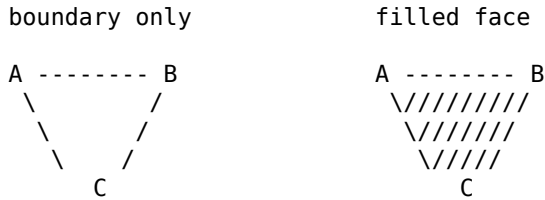
This chapter begins the applied extension beyond Goldblatt’s primary route.

17.1 Simplicial complexes

A finite abstract simplicial complex K consists of a set of vertices together with finite subsets called simplices, closed under taking subsets.

- a 0-simplex is a vertex;
- a 1-simplex is an edge;
- a 2-simplex is a filled triangle;
- a 3-simplex is a tetrahedron;
- a k -simplex has $k + 1$ vertices.

A triangle’s boundary and a filled triangle are different complexes:



The boundary has a one-dimensional loop. The filled triangle does not: its interior supplies a two-dimensional witness filling the loop.

17.2 Nerve of a cover

Let $\mathcal{U} = \{U_i \rightarrow U\}$ be a cover.

Definition 17.1 (Nerve). The nerve $N(\mathcal{U})$ has one vertex for each cover member U_i . A finite set $\{i_0, \dots, i_k\}$ spans a k -simplex when the corresponding contexts have a nonempty or otherwise valid joint overlap

$$U_{i_0} \times_U \cdots \times_U U_{i_k}.$$

For topological open covers, this means nonempty intersection. For a software site, “valid joint overlap” may require an explicit context that can observe or reconcile all members together.

A pairwise graph is only the 1-skeleton of the nerve. Higher simplices carry information not visible in the graph.

17.3 The SessionStream architecture complex

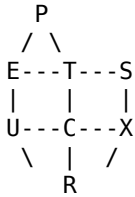
Take vertices:

Vertex	Context
E	event log
P	projection cursor
T	timeline materialization
S	snapshot
U	live UI stream
C	client state
X	observer trace
R	schema/projector contract

Possible edges include:

- $E - P$: event and progress share session/ordinal/projector;
- $E - T$: event and materialization share event identity and state transition;
- $P - T$: checkpoint claims materialization progress;
- $T - S$: snapshot serializes timeline state at a cut;
- $S - C$: client initializes from snapshot;
- $E - U$: UI batches are attributed to backend events;
- $U - C$: client reduces live events;
- X connected to protocol states it observes;
- R connected to every context whose values it interprets.

A partial 1-skeleton might look like:



The diagram alone does not tell us which triangles are filled. Does one transaction jointly witness (E, P, T) ? Does one trace context jointly witness (S, U, C) ? Does one semantic environment jointly bind (E, T, R) ? These are architecture questions.

17.4 Pairwise contracts versus joint witnesses

Suppose the edges $E - P$, $P - T$, and $T - E$ all exist. Pairwise compatibility can still fail to provide one three-way state. Adding the 2-simplex

$$\{E, P, T\}$$

asserts that a joint context exists and that three-way compatibility is meaningful.

Engineering examples of “filling a face” include:

- one database transaction commits event/materialization/checkpoint facts;
- one trace record contains all correlation identifiers;
- one typed aggregate constructs a consistent triple;
- one proof object certifies the three claims;

- one coordinator protocol establishes a joint commit epoch.

Caution. A transaction is not automatically a topological 2-simplex. The simplex is part of a model saying the three contexts have a joint overlap. The transaction is evidence that such an overlap can be realized.

17.5 Product dimensions

SessionStream has several independent axes:

- session identity;
- event time/cut;
- observation aspect;
- schema/projector version;
- principal/visibility;
- replica or process;
- connection and hydration attempt.

A context space therefore resembles a product:

$$\text{Session} \times \text{Time} \times \text{Aspect} \times \text{Version} \times \text{Principal} \times \dots$$

This is the source of the user’s “multidimensional shape” intuition. Dimension is not screen geometry. It is the number of independent coordinates or jointly overlapping contexts needed to describe a fact.

A 2D slice might show time versus observer:

observer	ordinal		
	n	n+1	n+2
event log	E_n	E_{n+1}	E_{n+2}
projection	P_n	P_{n+1}	P_{n+2}
timeline	T_n	T_{n+1}	T_{n+2}
snapshot	S_n	.	S_{n+2}
client	C_n	C_{n+1}	C_{n+2}

Horizontal restrictions change time; vertical maps compare observers at one cut. Squares encode commuting “advance then project” versus “project then advance” laws.

17.6 The Čech nerve

The full Čech nerve of a cover is a simplicial object whose level k contains $(k + 1)$ -fold overlaps:

$$\check{C}_k(\mathcal{U}) = \coprod_{i_0, \dots, i_k} U_{i_0} \times_U \dots \times_U U_{i_k}.$$

Face maps forget one member; degeneracy maps repeat one. Applying a presheaf produces local section data on overlaps of every order.

For a finite engineering model, one usually stores only nondegenerate simplices and explicit restriction maps. But the Čech viewpoint explains why cohomology has cochains in degrees $0, 1, 2, \dots$

17.7 Good covers and inference from the nerve

In topology, a good cover has contractible finite intersections, and the nerve can recover the homotopy type of the covered space under suitable hypotheses.

Do not transfer that theorem casually to software. A software nerve records the overlap pattern you declared. It can reveal loops and missing higher-order witnesses in the model, but it does not automatically recover the behavior of the running system.

The safe claim is:

The nerve is a compact combinatorial representation of the local-context overlap structure used by the analysis.

17.8 Two ways a hole can disappear

A loop in the nerve can disappear by:

1. **adding a higher simplex** - a joint context fills it;
2. **changing the cover/base** - some edges or vertices were not semantically distinct after all.

Software redesign may do either:

- introduce an atomic operation that jointly witnesses facts;
- unify two cursor notions under one authoritative coordinate;
- add stable identity so two deliveries become one quotient object;
- remove a falsely declared overlap that allowed invalid comparisons.

Topological language helps visualize alternatives, but the semantic reason for the redesign comes first.

17.9 Building a nerve from traces

A trace-derived nerve can be constructed as follows:

1. choose context types, such as event append, timeline apply, cursor advance, snapshot read, UI send, client reduce;
2. create a vertex for each relevant context instance or type;
3. add an edge when two contexts share a correlation key and a comparison law;
4. add a higher simplex only when one record, transaction, proof, or declared joint context witnesses all vertices;
5. label simplices with session, ordinal, version, and principal scope;
6. analyze each connected component separately when appropriate.

Instance-level complexes can be large. Type-level complexes are smaller but may hide execution-specific missing cells.

17.10 Exercises

17.1. Boundary versus face. Compute informally the loops in a triangle boundary and explain why adding the filled face removes the one-dimensional hole.

17.2. SessionStream nerve. Choose five vertices from Section 17.3. List all edges you can justify from current interfaces and every triangle for which you can justify a joint context.

17.3. Atomic face. Model event append, entity apply, and cursor advance before and after an atomic transaction. How does the complex change?

17.4. Time-aspect grid. Draw a 3×3 grid for events, timeline, and client at three cuts. Label commuting squares you expect.

17.5. Version dimension. Duplicate one architecture complex for projector versions v_1 and v_2 . Add only the migration edges that are natural. Where can loops appear?

17.6. False simplex. Give an example where three pairwise overlaps exist but no valid triple overlap should be added.

17.7. Trace nerve. Design a correlation-key rule for adding edges between observer records. Explain one false positive and one false negative.

17.8. Model change. Describe one bug fixed by adding a joint witness and one fixed by changing/quotienting the base contexts instead.

Cohomology: Measuring Persistent Incompatibility

Cohomology turns local data on a simplicial shape into algebraic invariants. In the present study it is best understood as a diagnostic layer placed **after** contexts, restrictions, covers, and coefficients have been defined.

The first computation will use an additive sheaf or coefficient system, not arbitrary Go values.

18.1 Why coefficients must be additive

Ordinary sheaf cohomology is naturally defined for sheaves of abelian groups, modules, or similar additive objects. This lets us subtract restrictions and form kernels and quotients.

Useful software summaries with additive structure include:

- cursor offsets in $\mathbb{Z}$;
- parity flags in $\mathbb{F}_2$;
- count differences;
- resource-balance vectors;
- clock skew estimates;
- hashes embedded in a vector space only for specialized analyses;
- signed flow or conservation quantities.

Arbitrary payloads, authorization rules, and state machines are not automatically additive. For them, begin with set-valued sheaves or constraint satisfaction. Linearize only a chosen diagnostic.

18.2 Cochains

Let K be an oriented simplicial complex and use constant coefficients in an abelian group A for the first example.

A 0-cochain assigns a value to every vertex:

$$C^0(K; A) = \prod_{v \in K_0} A.$$

A 1-cochain assigns a value to every oriented edge:

$$C^1(K; A) = \prod_{e \in K_1} A.$$

A 2-cochain assigns a value to every oriented face, and so on.

For a nonconstant cellular sheaf, stalks vary by cell and restriction maps enter the formulas. The constant case is enough to build intuition.

18.3 The coboundary

For a 0-cochain x , define the edge difference

$$(\delta^0 x)(i \rightarrow j) = x_j - x_i.$$

For a 1-cochain r , the coboundary on an oriented triangle (i, j, k) is

$$(\delta^1 r)(i, j, k) = r_{jk} - r_{ik} + r_{ij}.$$

Signs depend on orientation. The key identity is

$$\delta^{k+1} \circ \delta^k = 0.$$

The boundary of a boundary vanishes; correspondingly, differences of vertex potentials have zero circulation around every filled face.

18.4 Cocycles, coboundaries, and cohomology

A k -cochain c is a **cocycle** when

$$\delta^k c = 0.$$

It is a **coboundary** when

$$c = \delta^{k-1} b$$

for some $(k-1)$ -cochain b .

Every coboundary is a cocycle. Cohomology is

$$H^k(K; A) = \ker \delta^k / \text{im } \delta^{k-1}.$$

Interpretation:

- H^0 : globally compatible vertex assignments, often constants on connected components for constant coefficients;
- H^1 : closed edge-discrepancy patterns not explainable as differences of local vertex coordinates;
- H^2 : closed face-level patterns not generated from edge corrections;
- higher degrees: higher-order obstructions or degrees of freedom.

The exact meaning depends on the sheaf and coefficients, not only on the shape.

18.5 Cursor calibration example

Let vertices represent six observer coordinate systems:

$$E \rightarrow P \rightarrow T \rightarrow S \rightarrow C \rightarrow U \rightarrow E.$$

Suppose each vertex has an unknown local calibration $x_v \in \mathbb{Z}$. An edge records the claimed offset

$$r_{ij} = x_j - x_i.$$

If the offsets come from vertex calibrations, they are an exact 1-cochain:

$$r = \delta^0 x.$$

Summing around the cycle gives zero:

$$r_{EP} + r_{PT} + r_{TS} + r_{SC} + r_{CU} + r_{UE} = 0.$$

Now suppose interface conventions imply a total of $+1$. Every edge may appear locally plausible, but no global assignment of coordinate origins realizes all offsets. The edge cochain has nonzero circulation and represents a nontrivial class in H^1 of the cycle.

This is the precise version of an “off-by-one around an architecture loop.”

18.6 Exact versus closed

On a graph with no filled faces, every 1-cochain is automatically closed because $C^2 = 0$. Some are exact, others represent loop circulation.

When a triangle is filled, closure imposes

$$r_{ij} + r_{jk} - r_{ik} = 0.$$

This says the direct offset from i to k agrees with the route through j .

Adding 2-simplices therefore adds higher-order compatibility equations. It can kill 1-dimensional cohomology classes.

Software reading:

A joint three-way witness can make pairwise coordinate drift testable and force route independence.

18.7 Incidence matrices

Order vertices and oriented edges. The coboundary δ^0 is represented by an edge-vertex incidence matrix D_0 .

For the triangle boundary with vertices (A, B, C) and edges (AB, BC, CA) :

$$D_0 = \begin{bmatrix} -1 & 1 & 0 \\ 0 & -1 & 1 \\ 1 & 0 & -1 \end{bmatrix}.$$

If the face (ABC) is filled, δ^1 is represented by

$$D_1 = [1 \quad 1 \quad 1]$$

for a compatible orientation, and

$$D_1 D_0 = 0.$$

Over a field,

$$\dim H^0 = \dim \ker D_0,$$

$$\dim H^1 = \dim \ker D_1 - \text{rank } D_0.$$

For an unfilled connected triangle boundary, $\dim H^1 = 1$. For a filled triangle, $\dim H^1 = 0$.

18.8 Graph cycle rank

For a graph with V vertices, E edges, and c connected components, constant-field coefficients give

$$\dim H^1 = E - V + c$$

when there are no 2-cells.

This counts independent cycles, but it says nothing by itself about software inconsistency. A cycle is a place where a closed discrepancy can circulate. Whether one actually does depends on the coefficient data.

Topology gives the possible modes; sections and cochains give the observed values.

18.9 Cellular sheaf version

A cellular sheaf F assigns a vector space or abelian group $F(\sigma)$ to each cell σ and a restriction map from a lower-dimensional cell to a containing cell, under one common convention.

For an architecture graph:

- a vertex stalk can contain local cursor coordinates or state summaries;
- an edge stalk contains shared facts used to compare endpoints;
- restriction maps translate each endpoint's coordinate into edge coordinates.

Then

$$(\delta^0 x)_e = \rho_{v_+ \rightarrow e}(x_{v_+}) - \rho_{v_- \rightarrow e}(x_{v_-}).$$

A global section is a 0-cochain in $\ker \delta^0$: every endpoint agrees after translation to the overlap.

For this reason, H^0 often deserves attention before H^1 . It is the linear global-consistency space.

18.10 A small calculation

Take the cycle $E - P - T - E$ with claimed edge offsets

$$r_{EP} = 0, \quad r_{PT} = 0, \quad r_{TE} = 1.$$

Assume an orientation $E \rightarrow P \rightarrow T \rightarrow E$. The circulation is

$$0 + 0 + 1 = 1.$$

No vertex coordinates x_E, x_P, x_T can satisfy all three equations:

$$x_P - x_E = 0,$$

$$x_T - x_P = 0,$$

$$x_E - x_T = 1.$$

The first two imply $x_E = x_P = x_T$, contradicting the third.

If instead all offsets sum to zero, choose $x_E = 0$, then integrate offsets along a spanning tree to obtain a global calibration. Zero circulation on every independent cycle is exactly the condition for path-independent integration.

18.11 What nonzero and zero mean

For a well-specified cohomological model:

- a nonzero obstruction class can certify that no global additive calibration/section of the chosen kind exists;
- a zero class says this particular obstruction vanishes.

Zero does **not** automatically prove full system correctness. The model may omit coordinates, use an incomplete cover, or linearize away nonlinear constraints. Some cohomological obstruction theories are sufficient but not complete.

18.12 Computation workflow

1. enumerate cells and choose orientations;
2. assign coefficient spaces/stalks;
3. implement restriction matrices;
4. build coboundary matrices D_k ;
5. verify $D_{k+1}D_k = 0$;
6. compute kernels, images, ranks, and quotient dimensions;
7. map basis vectors back to named architecture cells;
8. validate that the class corresponds to a meaningful software discrepancy.

Over $\mathbb{Q}$ or a finite field, standard Gaussian elimination suffices for small models.

18.13 Exercises

18.1. Coboundary identity. Verify directly that $D_1D_0 = 0$ for the filled triangle matrices.

18.2. Triangle boundary. Compute $\dim H^0$ and $\dim H^1$ for an unfilled triangle over $\mathbb{Q}$.

18.3. Filled triangle. Repeat after adding the 2-simplex.

18.4. Two cycles. Draw a connected graph with cycle rank two and verify $E - V + 1 = 2$.

18.5. Cursor contradiction. Solve the equations in Section 18.10 and show exactly where inconsistency appears.

18.6. Zero circulation. Given offsets $(2, -5, 3)$ around a triangle boundary, construct vertex coordinates realizing them.

18.7. Sheaf stalks. Design vertex and edge stalks for comparing event cursor and projection cursor when one uses “last included” and the other “next to consume.”

18.8. Modeling caveat. Give a nonlinear transactional invariant invisible to an integer-offset cohomology model.

A SessionStream Obstruction Catalogue

This chapter classifies recurring SessionStream obligations by the mathematical tool that best fits them. The purpose is diagnostic discipline: use cohomology where it adds information, but use simpler tools when the issue is malformed local data, a missing coordinate, failed functoriality, or a missing transaction.

19.1 Diagnostic matrix

Engineering symptom	First mathematical reading	Likely repair	Cohomology role
cursor conventions	nonexact edge-offset	type/translate coordinates;	direct and useful
compose to off-by-one	cocycle	fill route witness	
snapshot row later than cut	invalid local section or overlap mismatch	consistent read cut	usually unnecessary
cursor advanced without materialization	no joint global section; missing commit witness	atomic commit/recovery state	possible after linearization
retry gets new ordinal and re-applies	missing logical identity; quotient absent	stable EventId; coequalizer	not first-line
replay differs from live	functor law failure or hidden coordinate	explicit dependencies/versioning	optional summary only
migration depends on route	failed naturality square	coherent migration	defect can be measured on cycles
hydration drops buffered batch	incomplete cover/matching family	fail and rehydrate	not needed to reject
observer record dropped	evidence presheaf incomplete	mark proof inconclusive	do not infer system bug
timeout interpreted as crash proof	incorrect global truth claim	contextual suspicion proposition	local logic, not cohomology

19.2 Cursor holonomy

Cursor-like values form multiple coordinate systems:

event store:	last canonical event present
projection cursor:	last event claimed processed
snapshot ordinal:	last event represented durably
client cursor:	last event integrated or acknowledged
bus coordinate:	transport-specific stream position

Suppose adapters provide translations on architecture edges. If translations are affine offsets,

$$x_j = x_i + r_{ij},$$

then route independence requires zero sum around every loop.

A nonzero loop sum is analogous to holonomy: transport a coordinate around a closed path and it returns shifted. This is a strong use case for H^1 because:

- data is additive;
- local translations are edge values;
- global calibration is a vertex potential;
- loops expose path dependence.

Repair choices:

1. give cursor meanings distinct types and explicit conversions;
2. choose one authoritative coordinate;
3. alter an edge convention;
4. add a higher-order route law/face;
5. remove an invalid comparison edge.

The cohomology class locates an incompatibility mode; it does not choose the business semantics.

19.3 Projection progress and the missing face

The current processing sequence has distinct effects: event append, projection, entity apply, projection-cursor advance, and fanout. Consider the triple (E, T, P) .

Pairwise observations can exist:

- event n is present;
- timeline state reflects n ;
- projector claims n .

If no atomic or recoverable joint state binds all three, the nerve may contain the boundary triangle without a face.

This geometry suggests where an independent circulation or incompatibility might live. But the actual invariant is nonlinear and temporal. To calculate cohomology, choose a coefficient sheaf, for example integer lag:

$$\ell_{ET} = \text{eventCursor} - \text{timelineCursor},$$

$$\ell_{TP} = \text{timelineCursor} - \text{projectionCursor},$$

$$\ell_{PE} = \text{projectionCursor} - \text{eventCursor}.$$

These always sum algebraically to zero if read from one global triple. A nonzero observed circulation indicates the values were not jointly sampled or do not share semantics.

That can be useful in telemetry, but the stronger correctness proof still needs transaction/recovery semantics.

19.4 Consistent-cut failure

A snapshot with an entity later than its cut is best treated as a locally invalid section:

$$\exists x \in S. \text{last}(x) > \text{cut}(S).$$

No higher-dimensional theory is necessary to reject it.

A subtler mixed-cut snapshot may satisfy all scalar inequalities. Then compare:

- cursor read section;
- entity-version section;
- database transaction/MVCC section.

If cursor and rows cannot be restrictions of one database-state section, gluing fails. Again, a direct pull-back/limit model is usually clearer than H^1 .

Use cohomology only if multiple replicas or coordinate transformations create cycles of cut relations.

19.5 Retry identity is usually a base problem

Without stable logical identity, the model cannot tell whether two deliveries should be compared, glued, or quotient-identified. Creating a cycle of ordinal differences will not recover missing identity.

Correct sequence:

1. enrich event/delivery context with EventId or idempotency key;
2. define a retry equivalence relation;
3. prove downstream operations are invariant under it;
4. form or implement the quotient;
5. only then analyze distributed copies of logical identity for higher obstruction.

This is a general rule:

Cohomology can detect obstruction relative to a coefficient system; it cannot manufacture semantic coordinates that the system never records.

19.6 Deterministic replay as functoriality

Replay mismatch is first an equation failure:

$$R(ab) \neq R(b) \circ R(a)$$

or a disagreement between live and replay functors:

$$L(E_n) \neq R(E_n).$$

Investigate:

- hidden clock/random/service inputs;
- schema or projector version drift;
- nondeterministic ordering;
- retry duplication;
- different error policy;
- initial-state mismatch;
- noncanonical payload serialization.

A natural transformation between live and replay interpretations should make every prefix-extension square commute. One can assign error vectors to the edges of version/history diagrams and analyze cycle defects, but only after the basic functor models are sound.

19.7 Migration curvature

Suppose there are schema versions v_1, v_2, v_3 and migration paths:

$$v_1 \rightarrow v_2 \rightarrow v_3$$

and

$$v_1 \rightarrow v_3.$$

For each payload x , define the defect

$$\kappa(x) = M_{13}(x) - M_{23}(M_{12}(x))$$

in an additive summary space. Nonzero defect is path dependence around a version triangle.

If many versions form a graph, defects can be organized as cochains. Filled migration triangles impose coherence. This is a practical cohomological pattern for schema/calibration networks, provided subtraction is meaningful in the chosen summary.

For arbitrary protobuf payloads, compare canonical digests or semantic features rather than pretending payloads form a vector space.

19.8 Hydration gaps

A missing buffered batch causes failure of suffix completeness. The most direct model is:

- the local live section is absent or invalid;
- the family does not cover the target execution;
- no successful amalgamation should be claimed.

Could cohomology detect a gap? An interval has no H^1 hole by itself. A missing event is not automatically a topological loop. One could build a sheaf whose local exactness or conservation law detects it, but that is additional modeling.

This is a useful correction to visual intuition: **a gap in time is not necessarily nontrivial cohomology.**

19.9 Observer evidence and epistemic failure

The observer dispatcher is bounded and best-effort; records may be dropped under backpressure. Therefore the trace presheaf can be incomplete even when the underlying protocol execution is correct.

A checker should produce three results:

```
proved for this trace
refuted by this trace
inconclusive because evidence is incomplete
```

Collapsing inconclusive into refuted confuses a failure of knowledge with a failure of the system.

In topos language, the current stage may force neither the property nor its negation.

19.10 Local truth of failure suspicion

The heartbeat state machine can establish:

$$\text{SuspectedFailed}(c, p, t)$$

under policy p and observed timing evidence t .

It cannot establish the global proposition

RemoteProcessCrashed(c)

without stronger assumptions. Network delay, event-loop suspension, and scheduling delay can produce the same local observation.

Model the classifier honestly: a timeout yields a contextual truth value supported by refinements preserving the timing evidence and assumptions.

19.11 Constraint sheaves before linearization

Many SessionStream laws are better represented as a set-valued constraint presheaf:

$$F(U) = \{\text{assignments on } U \text{ satisfying local laws}\}.$$

A global section is a global satisfying assignment. Search can use:

- SAT/SMT;
- relational joins;
- Datalog;
- finite-domain CSP;
- model checking;
- symbolic execution.

Cohomology can then serve as:

- a fast obstruction test;
- a localization method for inconsistency cycles;
- a dimensional summary of linear degrees of freedom;
- a feature for prioritizing global-section search.

Nonzero can be decisive in a well-formed obstruction theory; zero may still leave a nonlinear CSP unsatisfied.

19.12 Investigation workflow

For a concrete bug or invariant:

1. state the global property without topology vocabulary;
2. identify local contexts and observations;
3. validate local sections;
4. validate restriction functor laws;
5. declare and justify covers;
6. check overlap matching;
7. search for global sections;
8. classify missing or multiple completions;
9. add an additive summary only if meaningful;
10. compute cohomology and map classes back to architecture;
11. propose repairs and re-run the model;
12. add executable repository tests.

19.13 Exercises

19.1. Diagnostic classification. Classify each of the six law families from Chapter 1 using the matrix in Section 19.1.

19.2. Cursor loop. Define three cursor conventions whose translations sum to $+1$ around a cycle. Propose a typed repair.

19.3. Joint sampling. Show why lag differences computed from three different database moments can have nonzero circulation even when every individual cursor is correct.

19.4. Migration defect. Define an additive feature vector for chat-message payloads and compute a path defect across two migration routes.

19.5. Gap correction. Explain why a missing event in a linear interval is not by itself an H^1 class.

19.6. Epistemic result. Design a three-valued result type for a hydration trace checker and state when each case is returned.

19.7. CSP model. Encode snapshot soundness and suffix completeness as finite-domain constraints for a three-event example.

19.8. Investigation plan. Choose one open SessionStream obligation and write the twelve-step workflow for it.

Laboratory: Build ss-sheafcheck

The final part is a concrete research program: a small tool that reads SessionStream traces and a declared context model, checks presheaf and sheaf laws, searches for global sections in finite cases, and optionally computes low-dimensional cohomology for additive summaries.

The tool should be treated as an executable notebook, not a production gate at first.

20.1 Scope

Version 0 should answer:

1. Are local trace-derived sections well formed?
2. Do restriction maps satisfy identity and composition on sampled data?
3. Do sections in a proposed cover match on overlaps?
4. Is a claimed global section compatible with all local sections?
5. For a finite constraint model, are there zero, one, or several global sections?
6. For a finite linear coefficient system, what are H^0 and H^1 ?
7. Which named cells support a nonzero discrepancy class?

Do not attempt general sheaf cohomology over arbitrary rings in the first implementation.

20.2 Package structure

```
cmd/ss-sheafcheck/
  main.go
internal/model/
  context.go
  cover.go
  complex.go
internal/trace/
  sessionstream.go
  sections.go
internal/presheaf/
  restrict.go
  laws.go
internal/glue/
  matching.go
  search.go
internal/cochain/
  matrix.go
  cohomology.go
internal/report/
  markdown.go
  json.go
examples/
  hydration-ok.yaml
  hydration-gap.yaml
  cursor-cycle.yaml
```

Keep the mathematical model independent of SessionStream trace decoding so other software systems can reuse it.

20.3 Core data model

A finite prototype can use:

```
type ContextID string
type CellID string
type FactKey string

type Context struct {
    ID          ContextID
    Session     string
    From        uint64
    Through     uint64
    Aspects     []string
    Version     string
    Principal   string
}

type Section struct {
    Context ContextID
    Facts   map[FactKey]Value
    Source  EvidenceRef
}

type Restriction struct {
    From ContextID
    To   ContextID
    Map  RestrictionSpec
}

type Cover struct {
    Target ContextID
    Members []ContextID
}
```

Values should initially be a small typed algebra rather than any:

- integer
- string
- boolean
- ordered integer sequence
- set of strings
- digest
- record
- unknown/incomplete evidence

Explicit unknown values are needed for the proved/refuted/inconclusive distinction.

20.4 Model file

A YAML-like model might be:

```
contexts:
  reconnect:
    session: s1
    from: 0
    through: 45
    aspects: [snapshot, live, client, trace]
```



```

    version: projector-v3

snapshot:
  session: s1
  from: 0
  through: 42
  aspects: [snapshot]
  version: projector-v3

suffix:
  session: s1
  from: 42
  through: 45
  aspects: [live, trace]
  version: projector-v3

arrows:
  - [snapshot, reconnect]
  - [suffix, reconnect]

covers:
  - target: reconnect
    members: [snapshot, suffix]

overlaps:
  snapshot+suffix:
    facts: [session, cut, version]

```

A real schema should distinguish an inclusion arrow from its induced restriction direction.

20.5 Trace extraction

From observer records and snapshots, derive sections such as:

```

SnapshotSection:
  session
  snapshotOrdinal
  maxEntityLastOrdinal
  entity digest
  schema/projector version if known

```

```

LiveSection:
  session
  buffered ordinals
  sent ordinals
  UI event names/digests
  overflow status

```

```

ProtocolSection:
  registration index
  snapshot sent index
  transition-to-live index
  frame order

```

Every derived fact should retain provenance: source record IDs, file offsets, or trace timestamps. Reports can then map mathematical failures back to evidence.

20.6 Local validators

Examples:

```
snapshot:
  session is nonempty
  maxEntityLastOrdinal <= snapshotOrdinal
  all payload kinds are registered

live suffix:
  ordinals are nondecreasing
  every sent frame belongs to subscribed session
  overflow implies unsuccessful completion

protocol:
  registeredBefore(snapshotLoad)
  snapshotSentBefore(afterCutLive)
  markedLiveAfter(finalBufferedSend)
```

A local failure should stop gluing for that section and be reported directly.

20.7 Restriction engine

Represent restrictions as named functions with law tests:

```
type Restrictor interface {
  Restrict(ctx context.Context, from, to Context, s Section) (Section, error)
}
```

Built-ins:

- truncate ordered events to a subinterval;
- project records to a subset of fact keys;
- filter to a session or connection;
- map detailed values to digests;
- hide fields outside principal scope.

Test on every generated chain $W \rightarrow V \rightarrow U$:

```
restrict(U, U, s) == s
restrict(V, W, restrict(U, V, s)) == restrict(U, W, s)
```

If evidence is insufficient to perform a total restriction, return an explicit unsupported/unknown result and reconsider the base category.

20.8 Matching checker

For each cover and each pair of members:

1. construct or look up the overlap context;
2. restrict both sections to it;
3. compare using typed equality;
4. produce a fact-level mismatch report.

Example:

```
cover reconnect <- {snapshot, suffix}
match failure on overlap boundary:
  snapshot.version = projector-v2
  suffix.version   = projector-v3
provenance:
  snapshot record #18
  live batch record #27
```

Matching is pairwise. Higher-order checks should inspect triple and larger overlaps when the model

includes them.

20.9 Global-section search

For finite domains, global-section search is a CSP.

Variables represent global facts. Local sections constrain their restrictions. Algorithm options:

- brute-force enumeration for tiny examples;
- backtracking with constraint propagation;
- SAT/SMT encoding;
- relational natural joins;
- use an off-the-shelf finite-domain solver.

Return:

```
0 solutions -> inconsistent/no amalgamation
1 solution -> unique global section in the finite model
>1 solutions -> ambiguous/underdetermined
unknown -> model too large or evidence incomplete
```

This implements the parameter-fiber criterion directly.

20.10 Simplicial complex and cochains

Represent cells:

```
type Simplex struct {
    ID          CellID
    Vertices    []ContextID // sorted canonical order
    Degree      int
}
```

Validate downward closure: every face of every simplex must exist.

For constant coefficients over $\mathbb{Q}$ or $\mathbb{F}_p$:

1. order k -simplices;
2. construct signed incidence matrices;
3. transpose boundary matrices to obtain coboundaries under your convention;
4. verify $D_{k+1}D_k = 0$;
5. compute ranks and nullspaces;
6. report Betti dimensions.

For a cellular sheaf, each incidence contributes a block restriction matrix rather than a scalar sign.

20.11 Cursor-cycle example

Model:

```
vertices: [event, projection, timeline]
edges:
  - [event, projection]
  - [projection, timeline]
  - [timeline, event]
edge_offsets:
  event->projection: 0
  projection->timeline: 0
  timeline->event: 1
```

coefficients: integers

Expected report:

```
cycle basis:
  event -> projection -> timeline -> event
circulation: +1
status: no global integer calibration
suggested inspections:
  compare "last included" vs "next expected" semantics
  check whether values were jointly sampled
  inspect timeline->event conversion
```

Do not report only dim $H_1 = 1$; map the class to named edges and semantics.

20.12 Hydration example

For an accepted trace:

```
snapshot ordinal: 42
buffered ordinals: 41, 43, 44
sent after snapshot: 43, 44
late buffered: 45
marked live after 45
```

The checker should verify:

- 41 is correctly excluded as already covered;
- 43,44,45 are sent in order after the snapshot;
- the live transition occurs after 45;
- no overflow invalidates evidence.

For a failing trace, omit 44 or mark live before its send. This is a gluing/protocol failure, not necessarily a cohomology calculation.

20.13 Property and mutation tests

Property tests:

- restriction identities and composition;
- matching is symmetric under pair order;
- a constructed global section restricts to its local family;
- adding a filled face cannot increase constant-coefficient $\dim H^1$ in a fixed finite complex;
- coboundary squares to zero;
- reordering cell enumeration does not change Betti dimensions.

Mutation tests:

- shift one cursor convention by one;
- remove one suffix batch;
- alter one projector version;
- drop one evidence record;
- duplicate one logical event with a fresh ordinal;
- reorder late hydration sends;
- split atomic event/apply/cursor evidence into separate epochs.

Each mutation should produce the mathematically appropriate class of report.

20.14 Four capstone projects

20.14.1 Capstone A: Hydration descent checker

Build a trace checker for snapshot-before-live, with formal assumptions and a state-machine oracle. Produce counterexample traces and a proof-oriented report.

20.14.2 Capstone B: Cursor coordinate atlas

Inventory every ordinal/cursor in SessionStream, define typed coordinate transformations, construct the architecture graph, and compute cycle circulation. Eliminate all unexplained loops.

20.14.3 Capstone C: Projection semantic presheaf

Define contexts by event prefix and projector version. Implement restriction, live/replay functors, and naturality tests for migrations. Search for hidden inputs.

20.14.4 Capstone D: API sufficiency analyzer

Given a finite schema of global facts, dependencies, request parameters, and invariants, compute completion fibers. Report inconsistent, uniquely determined, or invariant-sufficient parameter sets.

20.15 Research notebook template

For every experiment, record:

Question:
 Global property:
 Base category:
 Objects/contexts:
 Arrows/restrictions:
 Covers:
 Presheaf sections:
 Local laws:
 Matching equations:
 Global-section result:
 Coefficient system, if any:
 Cohomology result, if any:
 Counterexample trace:
 Architecture repair:
 Repository test added:
 What the model omits:

The final line is mandatory.

20.16 Final synthesis

The complete bridge is now:

universal constructions $\rightarrow$ functorial semantics $\rightarrow$ context-indexed data $\rightarrow$ presheaves $\rightarrow$ matching and gluing $\rightarrow$

For SessionStream:

- event history supplies a temporal coordinate;

- projections are candidate functorial interpretations;
- stores, cursors, snapshots, transports, and clients are overlapping observers;
- restrictions forget aspects, narrow scope, or move to earlier cuts;
- reconnect is a sheaf gluing problem;
- transaction boundaries provide joint witnesses;
- missing event IDs and hidden dependencies are missing coordinates;
- contextual truth distinguishes evidence, refutation, and uncertainty;
- nerves visualize overlap order;
- H^1 can detect persistent additive path dependence such as cursor holonomy.

The strongest habit to carry forward is not a formula. It is the sequence of questions:

What is local?

What is shared?

How is restriction defined?

What counts as coverage?

Do local pieces match?

Does a global section exist?

Is it unique?

If not, is the issue missing data, missing witness, bad functoriality,
or a genuine obstruction in the chosen coefficient system?

20.17 Exercises

20.1. Minimal model. Write the smallest YAML model for snapshot and suffix contexts with one overlap.

20.2. Restriction implementation. Implement or pseudocode a restriction that truncates an ordered batch sequence and prove the two presheaf laws.

20.3. Global search. For a finite API model with two Boolean hidden facts and one visible parameter, enumerate every completion fiber.

20.4. Matrix code. Implement incidence matrices for a graph and verify the cycle-rank formula on random connected graphs.

20.5. Cellular extension. Replace constant scalar coefficients with two-dimensional cursor/version vectors and edge translation matrices.

20.6. Mutation oracle. Map each mutation in Section 20.13 to the expected diagnostic category.

20.7. Capstone proposal. Choose one capstone and write a two-page specification using the research notebook template.

20.8. Model humility. List five important SessionStream behaviors omitted by your capstone model and explain how each could affect conclusions.

Appendix A: Categorical and Sheaf-Theoretic Cheat Sheet

A.1 Core category language

Concept	Definition pattern	Software question
category	objects, arrows, identity, associative composition	Which transformations compose, and what counts as doing nothing?
isomorphism	f has a two-sided inverse	Is this representation change lossless in both directions?
monic	left-cancellable	Does this map preserve distinctions relevant to upstream arrows?
epic	right-cancellable	Is this map sufficient to determine every downstream arrow?
initial object	exactly one arrow out to each object	What is the canonical empty/free starting object?
terminal object	exactly one arrow in from each object	What is the canonical forget-all or unit observation?
opposite category	reverse every arrow	What is the dual information-flow question?

A.2 Universal constructions

A.2.1 Limit

A cone (L, π_j) over D is limiting when every cone (N, p_j) has a unique mediating arrow $N \rightarrow L$.

other joint witness $\xrightarrow{\text{unique}}$ universal joint witness
 $\quad \quad \quad \downarrow \quad \quad \quad \downarrow$
 $\quad \quad \quad + \text{----- same observations -----} +$

A.2.2 Colimit

A cocone (L, ι_j) is colimiting when every cocone from D receives one unique arrow from L .

A.2.3 Standard cases

Shape	Limit	Colimit
empty diagram	terminal	initial
two disconnected objects	product	coproduct
parallel arrows	equalizer	coequalizer
cospan $A \rightarrow C \leftarrow B$	pullback	-
span $B \leftarrow A \rightarrow C$	-	pushout

A.2.4 Pullback

$$A \times_C B = \{(a, b) \mid f(a) = g(b)\}$$

in **Set**.

Question: Which pairs agree on a shared semantic boundary?

A.2.5 Equalizer

$$\text{Eq}(f, g) = \{a \mid f(a) = g(a)\}.$$

Question: On which inputs do two interpretations agree?

A.2.6 Coequalizer

Quotient by the least equivalence forcing $f(a) \sim g(a)$.

Question: Which representation distinctions should all downstream semantics ignore?

A.3 Functorial language**A.3.1 Functor**

$$F(1_A) = 1_{F(A)}, \quad F(g \circ f) = F(g) \circ F(f).$$

Question: Does this interpretation preserve no-op and staged composition?

A.3.2 Natural transformation

For $\eta : F \Rightarrow G$,

$$G(f) \circ \eta_A = \eta_B \circ F(f).$$

Question: Does conversion commute with every context change or transition?

A.3.3 Exponential

$$\mathcal{C}(X \times A, B) \cong \mathcal{C}(X, B^A).$$

Question: Can context-stable behavior be represented as an object?

A.3.4 Adjunction

$$\mathcal{D}(F(A), B) \cong \mathcal{C}(A, G(B)).$$

Question: Are these opposite-looking operations related by an optimal universal translation?

A.4 Presheaves and sheaves

A.4.1 Presheaf

$$F : \mathcal{C}^{op} \rightarrow \mathbf{Set}.$$

For $V \rightarrow U$:

$$F(U) \rightarrow F(V).$$

Restriction laws:

$$s|_U = s,$$

$$(s|_V)|_W = s|_W.$$

Question: What data is meaningful here, and how can richer data be forgotten coherently?

A.4.2 Matching family

$$s_i|_{U_i \times_U U_j} = s_j|_{U_i \times_U U_j}.$$

Question: Do local pieces agree on every declared overlap?

A.4.3 Sheaf

Every matching family over a cover has a unique global amalgamation.

Equalizer form:

$$F(U) \cong \text{Eq} \left(\prod_i F(U_i) \rightrightarrows \prod_{i,j} F(U_i \times_U U_j) \right).$$

Question: Do compatible local observations determine exactly one global object?

A.4.4 Site

A category plus covers satisfying:

1. identity covers;
2. covers compose/refine;
3. covers are stable under pullback.

Question: What does the architecture declare to be jointly sufficient, and does sufficiency survive restriction?

A.5 Topos and local logic

A.5.1 Elementary topos

Finite limits + exponentials + subobject classifier.

A.5.2 Subobject classifier

Every monic $S \hookrightarrow A$ is the pullback of

$$\text{true} : 1 \rightarrow \Omega$$

along one characteristic arrow $A \rightarrow \Omega$.

A.5.3 Presheaf truth values

$\Omega(U)$ is the set of sieves on U .

A sieve is a downward-closed collection of refinements into U .

Question: Under which refinements is this statement supported?

A.5.4 Local truth

$$U \Vdash \varphi$$

means φ holds at stage U . Truth persists under refinement.

A.5.5 Geometric morphism

$$f^* : \mathcal{E} \rightleftarrows \mathcal{F} : f_*, \quad f^* \dashv f_*$$

with f^* preserving finite limits.

Question: Does this change of context preserve finite compatibility and geometric logic?

A.6 Nerves and cohomology

A.6.1 Nerve

- vertex: local context;
- edge: pairwise overlap;
- triangle: triple overlap/joint witness;
- tetrahedron: four-way overlap;
- higher simplex: higher-order joint context.

A.6.2 Cochains

$$C^0 \xrightarrow{\delta^0} C^1 \xrightarrow{\delta^1} C^2 \rightarrow \dots$$

with

$$\delta^{k+1}\delta^k = 0.$$

A.6.3 Cohomology

$$H^k = \ker \delta^k / \operatorname{im} \delta^{k-1}.$$

- H^0 : compatible global assignments in the additive model;
- H^1 : closed edge discrepancies not explained by vertex recalibration;
- higher H^k : higher-order classes relative to the chosen sheaf.

Question: Does a discrepancy circulate around the overlap shape in a way no local coordinate change removes?

A.7 Diagnostic order

Use this order:

```
local validity
-> restriction laws
-> coverage justification
-> overlap matching
-> global-section existence/uniqueness
-> missing-coordinate analysis
-> additive linearization
-> cohomology
```

Skipping directly to the last step is the most common modeling error.

Appendix B: Goldblatt Route Map

This appendix records how the attached source book guided the custom text. It is a route map, not a summary of all of Goldblatt.

Goldblatt section	Source topic	Use in this text
3.11	Limits and colimits; cones; universal factorization;	Chapters 3-4; sheaf condition as a limit/equalizer
3.12	terminal/product/equalizer as limits	
3.13	Coequalizers; equivalence relations and well-defined quotient operations	Chapter 5; stable retry identity and logical-event quotienting
3.14	Pullback	Chapters 3-4; typed joins, shared cut, predicate reindexing
3.15	Pushouts	Chapter 5; schema/history amalgamation and conflict
3.16	Completeness and finite completeness	Chapters 3 and 14; finite-limit structure of topoi
4.1	Exponentiation	Chapter 6; behavior as an object and cartesian closure
4.2	Subobjects	Chapter 14; invariants as subobjects
4.3	Classifying subobjects	Chapter 14; characteristic maps and contextual predicates
4.4	Definition of topos	Chapters 6 and 14
4.5	First examples	Chapter 14; Set and presheaf-style examples
4.7	Bundles and sheaves	Chapters 9-12; local sections and gluing
4.8	Power objects	Chapter 14; internal policy/predicate objects
9.1	Ω and comprehension	Chapters 14-15; truth values and predicates
9.2	The concept of functor	Chapter 7; replay and semantic translations
9.3	Natural transformations	Chapter 8; coherent migration and restriction
14.1	Functor categories	Chapters 8 and 14; presheaf topoi
14.2	Stacks/presheaves and sheaves; compatibility and amalgamation	Chapters 9-11
14.3	Classifying presheaves and sheaves	Chapters 14-15; sieve-valued truth
14.4	Grothendieck topoi and sites	Chapters 12 and 14
14.6	Elementary sites/topologies	Chapter 12; coverage policy
15.1	Kripke-Joyal semantics	Chapter 15; truth at stages
15.2	Adjunctions	Chapter 16
15.3	Examples of adjoint situations	Chapter 16; free/forgetful and sheafification patterns
15.4	Pullback-related adjoints	Chapter 16; substitution and context change
	Quantifiers	Chapter 16; existential/universal statements over fibers

Goldblatt section	Source topic	Use in this text
16.1	Preservation and reflection	Chapter 16; what translations preserve
16.2	Geometric morphisms	Chapter 16
16.3	Internal logic	Chapter 15
16.4	Geometric logic	Chapters 15-16
16.5	Theories as sites	Suggested future work in API/protocol specification

B.1 Pedagogical features retained

The custom text intentionally follows several features of Goldblatt’s teaching style:

- introduce a construction through its universal property;
- revisit familiar examples as instances of one pattern;
- use duality as an active exercise rather than a footnote;
- place short exercises immediately after definitions;
- distinguish internal categorical statements from set-element intuition;
- build later logical ideas from earlier categorical structure;
- emphasize factorization and uniqueness proofs.

B.2 Material intentionally omitted

This text does not attempt to cover Goldblatt’s full development of:

- classical and intuitionistic propositional calculi;
- Boolean and Heyting algebras in their full detail;
- natural number objects and formal arithmetic;
- set theory internal to a topos;
- completeness theorems and detailed proof theory;
- Deligne and Barr theorems;
- algebraic-geometric applications;
- the full technical construction of sheafification;
- advanced logical modalities.

Those topics remain valuable but are not required for the SessionStream-centered route developed here.

B.3 Suggested parallel reading

When working through this book, read the attached source in this order:

Goldblatt 3.11-3.16
alongside Chapters 3-6 here

Goldblatt 9.1-9.3
alongside Chapters 7-9 here

Goldblatt 4.1-4.5 and 4.7-4.8
alongside Chapters 10 and 14

Goldblatt 14.1-14.6
alongside Chapters 10-15

Goldblatt 15.1-15.4 and 16.1-16.4

alongside Chapter 16

For each source exercise, rewrite at least one object as a SessionStream context and one arrow as a concrete restriction, migration, projection, or reducer.

Appendix C: SessionStream Source Map

Repository studied: github.com/go-go-golems/sessionstream

Frozen commit: [d62dca9f5efa2e3094d6c62e5ead5ed0c88fd35c](https://github.com/go-go-golems/sessionstream/commit/d62dca9f5efa2e3094d6c62e5ead5ed0c88fd35c)

C.1 Core model

Path	Relevant object	Mathematical use
<code>pkg/sessionstream/types.go</code>	<code>SessionId</code> , <code>Command</code> , <code>Event</code> , <code>Session</code>	base coordinates and canonical events
<code>pkg/sessionstream/projection.go</code>	<code>UIEvent</code> , <code>TimelineEntity</code> , <code>TimelineView</code> , projection interfaces	derived views, functor candidates, local sections
<code>pkg/sessionstream/schema.go</code>	typed schema registry	coproduct tags, declared semantic environment
<code>pkg/sessionstream/ordinals.go</code>	ordinal assignment and stream-ID derivation	coordinate systems and translations

C.2 Persistence and replay

Path	Relevant object	Mathematical use
<code>pkg/sessionstream/hydration.go</code>	<code>HydrationStore</code> , <code>EventStore</code> , <code>ProjectionCursorStore</code> , <code>Snapshot</code>	observation contexts and restriction interfaces
<code>pkg/sessionstream/hydration/sqlite.go</code>	<code>liveStore</code> , <code>entity apply</code> , <code>snapshot query</code> , <code>event append</code> , <code>cursors</code>	consistent-cut and atomicity studies
<code>pkg/sessionstream/hub.go</code>	<code>publish</code> , <code>replay</code> , <code>projectAndApply</code>	global semantic step versus separate effect boundaries

The processing path conceptually performs:

```
append event
-> obtain session and pre-event view
-> run UI and timeline projections
-> apply timeline entities
-> advance projection cursor
-> fan out live UI events
```

The mathematical model asks which subsets are jointly atomic, which are recoverable, and which cursors are authoritative.

C.3 Hydration transport

Path	Relevant object	Mathematical use
pkg/sessionstream/transport/ws/shydrating.go	hydrating/live subscription states, buffer, snapshot send, flush	primary sheaf-gluing case
pkg/sessionstream/transport/ws/observed.go	observed/dispatcher	evidence presheaf and inconclusive traces
pkg/sessionstream/transport/ws/observed/trace.go	observed/trace/trace organization tracing	trace-derived sections and finite proofs
pkg/sessionstream/transport/ws/hardhead.go	hardhead/subscription state machine	local truth under timing assumptions

The reconnect protocol registers a hydrating subscription before loading a snapshot, buffers concurrent UI batches, sends the snapshot, filters/sorts after-cut batches, flushes the late buffer, and only then marks the subscription live.

C.4 Example application

Path	Relevant object	Mathematical use
examples/chatdemo/chat.go	commands, canonical chat events, UI and timeline projections	exercises and replay examples
examples/chatdemo/chat_test.go	expected event/snapshot behavior	executable law starting points
proto/sessionstream/v1/transport.proto	protobuf schemas	typed boundary and serialization diagrams

The chat demo provides a manageable event family:

```
UserMessageAccepted
InferenceStarted
TokensDelta
InferenceTrace
InferenceFinished
InferenceStopped
```

and corresponding live/timeline interpretations.

C.5 Open study obligations

The Architecture Garden note motivating this book foregrounds:

- per-session serializability;
- consistent-cut snapshots;
- stable retry identity;
- atomic projection progress;
- deterministic replay;
- snapshot-plus-suffix completeness.

The book does not assert that every obligation is currently violated. It uses them as research questions and law families to formalize and test.

C.6 Suggested repository annotations

As you study, add a separate notebook rather than prematurely changing production types. For each relevant function, record:

source context
codomain context
semantic arrow
identity law
composition law
overlap facts
transaction boundary
failure result
version dependencies
trace evidence

Good starting functions:

- Hub.Publish
- Hub.RebuildTimeline
- Hub.projectAndApply
- HydrationStore.Apply
- HydrationStore.Snapshot
- WebSocket subscribe handling
- deliverUIEvents
- drainHydrationBuffer
- flushLateHydrationBufferAndMarkLive
- OrdinalAssigner.Next

Appendix D: Hints and Selected Solution Sketches

These are deliberately incomplete. A useful rule is to spend at least twenty minutes on an exercise before reading its entry.

Chapter 1

1.3 Prefix law

Let $\text{prefix}_n(E)$ truncate history to ordinals at most n .

Three laws are:

$$\text{prefix}_n(\text{prefix}_n(E)) = \text{prefix}_n(E)$$

(idempotence),

$$m \leq n \implies \text{prefix}_m(\text{prefix}_n(E)) = \text{prefix}_m(E)$$

(nesting), and

$$\text{prefix}_n(E) = E$$

when every ordinal in E is at most n . The nesting law will become presheaf restriction composition.

1.5 Missing coordinate

Let a projection attach `time.Now()` to every timeline entity. Replaying the same events later changes the timestamp. The declared input (`event`, `priorView`) is not the true input; wall-clock observation is missing. This is primarily model under-specification. Repair by recording the timestamp in the canonical event or injecting a deterministic clock value included in the replay environment.

Chapter 2

2.5 Parameter factorization

Let

$$P = (\text{principalId}, \text{sessionId}, \text{commandKind}, \text{policyVersion}).$$

If authorization depends only on these values, define $r : X \rightarrow P$ from complete request state and $\bar{I} : P \rightarrow \{0, 1\}$. Then

$$I = \bar{I} \circ r.$$

To justify the factorization, show any two global states with equal P values receive the same authorization result.

2.6 Duality

The dual statement is: an initial object has exactly one arrow from it to every object. A terminal-like software object may represent “forget all detail to unit.” An initial-like object may represent “empty syntax freely maps into every configured syntax,” though a genuine categorical claim requires specifying the category.

Chapter 3

3.4 Limits are unique up to unique isomorphism

Let (L, π_j) and (L', π'_j) both be limits. Universality of L gives a unique $u : L' \rightarrow L$ preserving projections. Universality of L' gives $v : L \rightarrow L'$. Both $u \circ v$ and 1_L preserve the cone maps from L ; uniqueness forces $u \circ v = 1_L$. Similarly $v \circ u = 1_{L'}$. Compatibility with cone maps also makes this isomorphism unique.

3.7 Two-piece sheaf equalizer

For $U = U_1 \cup U_2$, matching pairs are the equalizer

$$\text{Eq}(F(U_1) \times F(U_2) \rightrightarrows F(U_1 \cap U_2)).$$

The two maps restrict the first and second component. For three pieces, use a product of three local section sets and compare all ordered or unordered pairwise overlaps. Triple intersections matter for higher coherence and for the Čech nerve, even though set-valued sheaf matching is expressed pairwise with functorial restrictions.

Chapter 4

4.3 Snapshot race

One interleaving:

```
reader: read cursor = 42 from state d1
writer: apply event 43 and update entity row; commit state d2
reader: query current entity rows and observe lastEventOrdinal = 43
```

The cursor section is a restriction of d_1 ; the entity section is a restriction of d_2 . Their product pair need not be a restriction of any single database state.

4.7 Atomic witness choices

Possible K :

- one SQL transaction: strong local atomicity, couples state to one database;
- outbox/commit record: decouples consumers, adds recovery and cleanup complexity;
- canonical event log position: derive materialization status rather than separately committing it, but reads may become expensive;
- compare-and-swap generation: supports optimistic concurrency, requires retry logic.

The choice is an engineering tradeoff after the shared witness semantics are stated.

Chapter 5

5.3 Retry equivalence

Define $d_1 \sim d_2$ iff they have the same nonempty stable `EventId` and belong to the same event namespace. Reflexivity, symmetry, and transitivity follow from equality. If IDs can be reused across tenants or producers, namespace must be part of the relation; otherwise transitivity may identify unrelated deliveries operationally.

5.5 Quotient factorization

Define

$$\bar{P}([d]) = P(d).$$

Retry invariance makes this well defined: if $[d] = [d']$, then $d \sim d'$ and $P(d) = P(d')$. Then $P = \bar{P} \circ q$. Any other $Q : D/R \rightarrow T$ satisfying $P = Q \circ q$ must have $Q([d]) = P(d)$, so $Q = \bar{P}$.

Chapter 6

6.4 Constant on fibers iff factorization

If $I = \bar{I} \circ r$ and $r(x) = r(y)$, then $I(x) = \bar{I}(r(x)) = \bar{I}(r(y)) = I(y)$.

Conversely, assume I is constant on every fiber. For p in the image of r , define $\bar{I}(p) = I(x)$ for any x with $r(x) = p$; fiber constancy makes this well defined. Values outside the image require either an arbitrary extension or restricting the codomain to $\text{im}(r)$.

6.6 Context-unstable callback

A callback that fetches entity version “latest at cut 100” may refer to an entity not existing at cut 50. If restriction to 50 is expected, the callback must itself restrict, perhaps by querying historical versions. Without a coherent family of actions on all refinements, it is only a function at one stage, not a section of the internal exponential.

Chapter 7

7.1 Replay functor

Source category: event prefixes as objects; arrows are suffix extensions. Target: timeline states as objects; arrows are state transitions induced by suffixes. Identity is empty suffix; composition is suffix concatenation. Replay is functorial when empty replay is identity and replay of $a \mathrel{++} b$ equals replay by a followed by b .

7.6 Composite functors

For identities:

$$(GF)(1_A) = G(F(1_A)) = G(1_{F(A)}) = 1_{GF(A)}.$$

For composition:

$$(GF)(g \circ f) = G(F(g \circ f)) = G(F(g) \circ F(f)) = G(F(g)) \circ G(F(f)).$$

Chapter 8

8.1 Projection migration square

For an event extension $e : E_{n-1} \rightarrow E_n$:

$$G(e) \circ \eta_{E_{n-1}} = \eta_{E_n} \circ F(e).$$

Left route: migrate old state, then process the event under the new projector. Right route: process under the old projector, then migrate. A failed square identifies a state transition whose semantics changed incompatibly.

8.6 Pointwise products

Define $(F \times G)(U) = F(U) \times G(U)$ and restrictions componentwise. Given natural transformations $H \rightarrow F$ and $H \rightarrow G$, the unique componentwise pairing $H(U) \rightarrow F(U) \times G(U)$ is natural because both component squares commute. Thus the product universal property holds in the functor category.

Chapter 9

9.2 Prefix restriction

For a trace t through n , define $t|_m$ by retaining records with ordinal at most m . Restricting to the same cut changes nothing. If $\ell \leq m \leq n$, filtering first by m and then by ℓ equals filtering directly by ℓ , proving composition.

9.5 Fiber ambiguity

With parameters $(\text{orderId}, \text{amount})$, two completions may associate the same numeric amount with USD and EUR under different order versions. Add immutable `orderVersion` and `currency`, or make `orderId` resolve to an immutable record containing them. The invariant is sufficient exactly when every completion compatible with the request gives the same charge-validity result.

Chapter 10

10.2 Equalizer derivation

A tuple (s_i) belongs to the equalizer exactly when its images under the two restriction maps agree in every overlap component. This equation is the matching-family condition. Restriction from $F(U)$ always lands in the equalizer by presheaf functoriality. The sheaf condition says this map is bijective.

10.7 Unique reconstruction

Let C_n be fixed by a sound snapshot. Induct over the ordered suffix. At each step k , deterministic reduction gives exactly one C_k from C_{k-1} and the prescribed UI batch. Hence there is one C_m . Any other reconstruction agreeing on snapshot and suffix has the same base and transition at every step, so induction makes every intermediate state equal.

Chapter 11

11.1 Induction detail

Base $m = n$: the suffix is empty and the unique amalgamation is `clientize(S_n)`. Inductive step: assume unique state through k . Completeness selects the declared batch for event e_{k+1} ; determinism selects one next state. Any competing amalgamation must restrict to the unique state through k and apply the same batch, so it has the same next state.

11.3 Overtaking schedule

```
hydration worker drains initial buffer through 43
batch 44 enters late buffer
worker marks subscription live
publisher sees live and queues 45 directly
worker later drains late buffer and queues 44
wire order: 45, 44
```

Holding the subscription lock while flushing late batches and changing state prevents the publisher from observing live until 44 is queued.

Chapter 12

12.1 Interval pretopology

Identity subdivision covers an interval by itself. Refining each member of a subdivision produces a finer subdivision whose union is still the target. Pulling a subdivision back along a subinterval yields its intersections with every piece; those intersections cover the subinterval. Empty intersections may be omitted if the site convention allows.

12.6 Generated sieve

The sieve generated by $U_1, U_2 \rightarrow U$ contains every arrow $V \rightarrow U$ that factors through U_1 or U_2 . Closure under precomposition is automatic: if $W \rightarrow V \rightarrow U$ and $V \rightarrow U$ factors through U_i , then so does the composite.

Chapter 13

13.1 Sheafification triangle

For a presheaf F , sheaf G , and map $h : F \rightarrow iG$, universality supplies one $\bar{h} : aF \rightarrow G$ with

$$i(\bar{h}) \circ \eta_F = h.$$

The triangle expresses that every interpretation of F in a sheaf depends only on its universal sheaf repair.

13.7 Retry is a two-stage repair

Before `EventId`, there is no reliable relation saying which deliveries are retries. Adding the coordinate enriches the base/data. Only then can one define the equivalence relation and quotient. A quotient cannot identify semantically equal things if the model has no evidence of equality.

Chapter 14

14.1 Classifier in Set

For subset $S \subseteq A$, define $\chi_S(a) = 1$ iff $a \in S$. The pullback of $\text{true}: 1 \rightarrow \{0, 1\}$ along χ_S consists exactly of elements mapped to 1, hence is S . Any classifying map must take value 1 exactly on S , so it is unique.

14.6 Representable on cuts

In the poset category with arrow $m \rightarrow n$ when $m \leq n$,

$$y(n)(m) = \text{Hom}(m, n)$$

is a singleton when $m \leq n$ and empty otherwise. Restriction along $\ell \rightarrow m$ composes the unique arrow $m \rightarrow n$ with $\ell \rightarrow m$, giving the unique $\ell \rightarrow n$.

Chapter 15

15.3 Refinement implication

At stage U , “live implies snapshot sent” holds when every refinement $V \rightarrow U$ that supports `subscriptionState = live` also supports evidence that snapshot send occurred earlier in the protocol order. A current stage with no live subscription can satisfy the implication without proving that a snapshot has been sent.

15.5 Local retry witnesses

Shard A identifies deliveries by $(A, 17)$; shard B by $(B, 4)$. Each shard proves every delivery has some local correlation ID. If no overlap map relates $(A, 17)$ to $(B, 4)$, there is no global logical ID. Local existential witnesses exist but do not match.

Chapter 16

16.1 Lists as free monoid

The unit inserts a generator as a singleton list. Given any function $f: X \rightarrow M$ into the underlying set of a monoid, define $\bar{f}([x_1, \dots, x_n]) = f(x_1) \cdots f(x_n)$. This is a monoid homomorphism and is unique because every list is generated by singleton lists under concatenation.

16.4 Fiber quantifiers in Set

For $I \subseteq X$ and $f: X \rightarrow P$:

$$\exists_f(I) = \{p \in P \mid \exists x, f(x) = p \wedge x \in I\},$$

$$\forall_f(I) = \{p \in P \mid \forall x, f(x) = p \Rightarrow x \in I\}.$$

The first asks for one valid completion; the second requires every completion to satisfy the predicate.

Chapter 17

17.1 Boundary versus filled face

The boundary has three edges and three vertices, so for a connected graph its cycle rank is $3 - 3 + 1 = 1$. Adding the 2-simplex introduces a face whose boundary is exactly that cycle, making the cycle a boundary and killing the corresponding H^1 class.

17.3 Atomic face

Before atomicity, model pairwise comparison edges among event append, entity apply, and cursor advance, but no triple-overlap context. After one transaction or commit record jointly witnesses all three, add the 2-simplex and its restrictions. The face now enforces route consistency among pairwise coordinates.

Chapter 18

18.2 Triangle boundary cohomology

Over $\mathbb{Q}$, the connected graph has $\dim H^0 = 1$. With three edges, three vertices, and one component,

$$\dim H^1 = 3 - 3 + 1 = 1.$$

Equivalently, D_0 has rank two, C^1 has dimension three, and $D_1 = 0$.

18.6 Zero circulation

Orient edges $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow A$ with offsets $(2, -5, 3)$; the sum is zero. Choose $x_A = 0$, then $x_B = 2$, $x_C = -3$. Check $x_A - x_C = 3$ on $C \rightarrow A$. These vertex potentials realize the edge cochain.

Chapter 19

19.3 Nonjoint sampling

Let event cursor be read before event 10 commits, timeline cursor after materialization 10 commits, and projection cursor during a recovery state. Pairwise lag values combine facts from different global moments. Their algebraic sum need not vanish because they are not differences of one vertex assignment. Nonzero circulation then diagnoses incoherent sampling or semantics, not necessarily incorrect individual stores.

19.5 A temporal gap is not automatically H^1

An interval is contractible and has no one-dimensional cycle. Removing one event from a sequence is missing section data or failure of coverage/completeness. To obtain a cohomological detector, one would need an additional coefficient sheaf or complex whose cocycle equations encode conservation across the gap.

Chapter 20

20.2 Truncation restriction

```
func RestrictBatch(xs []Batch, cut uint64) []Batch {
    out := xs[:0]
    for _, x := range xs {
        if x.Ordinal <= cut {
            out = append(out, x)
        }
    }
    return slices.Clone(out)
}
```

Restricting to the current maximum keeps the sequence. For $a \leq b$, filtering by b and then by a equals filtering directly by a , since the final predicate is `ordinal <= a` in both routes.

20.3 Completion fibers

Let hidden facts be Booleans x, y and visible parameter $p = x \oplus y$. For $p = 0$, completions are $(0, 0)$ and $(1, 1)$; for $p = 1$, completions are $(0, 1)$ and $(1, 0)$. The parameter never determines full state. It is sufficient for invariant $x \oplus y = p$, but not for invariant $x = 1$.

Appendix E: Glossary

Adjunction. A natural correspondence between maps $F(A) \rightarrow B$ and maps $A \rightarrow G(B)$, written $F \dashv G$.

Amalgamation. A global section whose restrictions are a given matching family.

Apex. The object at the tip of a cone or cocone.

Base category. The category of contexts on which a presheaf or sheaf is defined.

Base change. Reindexing a family or object along a map, commonly implemented by pullback.

Boundary. The alternating sum of faces of a simplex; geometrically, the lower-dimensional shell of a cell.

Cartesian closed category. A category with finite products and exponentials.

Cellular sheaf. A sheaf-like assignment of algebraic data to cells of a cell complex, with compatible restriction maps.

Characteristic arrow. The unique arrow $A \rightarrow \Omega$ classifying a subobject of A .

Čech nerve. The simplicial object formed from all multiple overlaps of a cover.

Cocone. A compatible family of arrows from a diagram's objects into one apex.

Cocycle. A cochain in the kernel of the next coboundary map.

Coboundary. A cochain obtained by applying the previous coboundary map; also the map δ itself.

Cofinality. A condition under which a subdiagram computes the same colimit or limit; not developed in this text.

Cohomology. The quotient of cocycles by coboundaries, $H^k = \ker \delta^k / \text{im } \delta^{k-1}$.

Coefficients. The abelian groups, modules, or vector spaces in which cochains take values.

Colimit. A universal cocone; dual to a limit.

Commutative diagram. A diagram in which all directed paths with common endpoints have equal composites.

Complete category. A category containing limits of all diagrams of the specified size; finitely complete means all finite limits exist.

Cone. A compatible family of arrows from one apex into every object of a diagram.

Context. A stage, region, scope, cut, observer set, or information boundary represented as an object in the base category.

Contravariant functor. A functor reversing arrows, equivalently a functor from the opposite category.

Cover. A family of local contexts declared jointly sufficient for a target context.

Covering sieve. A sieve accepted as coverage by a Grothendieck topology.

Currying. The exponential correspondence between maps $X \times A \rightarrow B$ and $X \rightarrow B^A$.

Descent. The general problem of reconstructing global objects from compatible local objects and coherence data.

Diagram. A functor from a shape category into another category; informally, a structured collection of objects and arrows.

Duality. The process of reversing every arrow, converting a statement about $\mathcal{C}$ into one about $\mathcal{C}^{op}$.

Elementary topos. A category with finite limits, exponentials, and a subobject classifier.

Epic arrow. A right-cancellable arrow.

Equalizer. The universal arrow selecting where two parallel arrows agree.

Exact cochain. Another term for a coboundary, especially in degree one.

Exponential. An internal function object B^A characterized by evaluation and currying.

Fiber. For $f : X \rightarrow Y$ and $y \in Y$, the set/object of points of X mapped to y .

Finite limit. A limit of a finite diagram; includes terminal objects, products, equalizers, and pullbacks.

Functor. An object-and-arrow translation preserving identities and composition.

Functor category. A category whose objects are functors and arrows are natural transformations.

Generalized element. An arrow $U \rightarrow A$, interpreted as an element of A at stage U .

Geometric logic. Logic built from structure preserved by inverse-image functors of geometric morphisms, especially finite conjunction and existential/local constructions.

Geometric morphism. An adjunction $f^* \dashv f_*$ between topoi with f^* left exact.

Germ. The local behavior of a section near a point, identifying sections that agree on some neighborhood.

Global section. A section over the largest context, or an arrow $1 \rightarrow F$; in software models, a globally coherent state/execution.

Grothendieck topology. A declaration of covering sieves satisfying maximality, stability, and transitivity.

Grothendieck tops. A category equivalent to sheaves on a site.

Holonomy. Net transformation accumulated by transporting data around a closed loop; used here as intuition for cursor offset circulation.

Identity arrow. The do-nothing arrow $1_A : A \rightarrow A$.

Image. The subobject capturing values reached by an arrow; categorical construction depends on available factorization structure.

Internal logic. The logical language interpreted by the categorical structure of a topos.

Isomorphism. An arrow with a two-sided inverse.

Kripke-Joyal semantics. Stage-based forcing semantics for sheaves/topoi.

Left exact. Preserving finite limits.

Limit. A universal cone; the best joint witness for a diagram.

Local section. A member of $F(U)$ for a context U .

Matching family. Local sections that agree after restriction to every overlap.

Monic arrow. A left-cancellable arrow.

Natural transformation. A componentwise map between functors satisfying every naturality square.

Nerve. A simplicial complex/object recording which members of a cover overlap jointly.

Opposite category. The category obtained by reversing all arrows.

Power object. The topos analogue Ω^A of the power set of A .

Presheaf. A contravariant functor from a context category to sets or another target category.

Pretopology. A coverage basis specified by covering families satisfying identity, transitivity, and pullback stability.

Pullback. A universal compatible-pair object over a common codomain.

Pushout. A universal amalgamation object over a common domain.

Quotient. An object obtained by identifying equivalent values, categorically represented by a coequalizer

in suitable cases.

Refinement. An arrow from a smaller/more specific context to a broader context, depending on the chosen orientation.

Representable presheaf. The presheaf $\mathcal{C}(-, A)$ of probes into A .

Restriction map. The map $F(U) \rightarrow F(V)$ induced by a context arrow $V \rightarrow U$.

Section. A locally defined value in a presheaf or sheaf.

Separated presheaf. A presheaf in which matching local restrictions determine at most one global section.

Sheaf. A presheaf in which every matching family over every cover glues uniquely.

Sheafification. The universal map from a presheaf to a sheaf, left adjoint to inclusion.

Sieve. A precomposition-closed collection of arrows into one object.

Simplex. A combinatorial cell determined by a finite set of vertices.

Site. A category equipped with a Grothendieck topology or coverage basis.

Stalk. The local coefficient object attached to a cell or point, depending on sheaf setting.

Subobject. An equivalence class of monic arrows into an object; the categorical form of a subset/predicate.

Subobject classifier. The object Ω that classifies every subobject by a characteristic arrow.

Topos. A category with set-like categorical and logical structure; elementary and Grothendieck definitions are related but distinct presentations.

Universal property. A characterization by unique factorization among all objects satisfying a specified interface.

Yoneda lemma. The natural bijection $\text{Nat}(\mathcal{C}(-, A), F) \cong F(A)$.

Appendix F: Further Reading and Study Continuation

F.1 Primary source for this custom route

Robert Goldblatt, *Topoi: The Categorical Analysis of Logic*. Use the attached edition and the section route in Appendix B. Its strengths for this project are the systematic universal-property pedagogy, early introduction of topoi, and later return to presheaves, sites, local truth, adjunctions, and geometric morphisms.

F.2 Category theory foundations

The following are outside sources, offered for expansion rather than silently used as replacements for Goldblatt.

- Saunders Mac Lane, *Categories for the Working Mathematician*. The standard broad reference; concise and abstract.
- Emily Riehl, *Category Theory in Context*. Strong on universal constructions, adjunctions, Kan extensions, and proof technique.
- Steve Awodey, *Category Theory*. A compact bridge toward logic and topoi.
- David I. Spivak, *Category Theory for the Sciences*. Application-oriented and useful for database/schema intuition.

F.3 Sheaves and topoi

- Saunders Mac Lane and Ieke Moerdijk, *Sheaves in Geometry and Logic*. A standard systematic route from sites and sheaves to classifying topoi and logic.
- Peter T. Johnstone, *Topos Theory and Sketches of an Elephant*. Advanced references; consult after the core route is stable.
- Glen Bredon, *Sheaf Theory*. Classical sheaf theory with geometric emphasis.

F.4 Applied sheaves and computation

- Justin Curry, *Sheaves, Cosheaves and Applications*. A broad applied treatment with cellular constructions.
- Robert Ghrist, *Elementary Applied Topology*. Accessible computational topology and sheaf-adjacent intuition.
- Michael Robinson, *Topological Signal Processing*. Applied sheaf methods for sensor and information systems.
- Jakob Hansen and Robert Ghrist, work on cellular sheaves and their spectral theory. Useful after Chapter 18 when restriction maps become matrices.

F.5 Contextuality and obstruction methods

- Samson Abramsky and Adam Brandenburger, “The Sheaf-Theoretic Structure of Non-Locality and Contextuality.” This is a major example of global-section obstruction reasoning.

- Later work on cohomological obstructions to contextuality and constraint satisfaction. Read with the caveat emphasized in this book: a chosen obstruction may be sound without being complete.

F.6 Software-adjacent directions

Topics worth connecting next:

- event sourcing and deterministic state-machine replication;
- distributed snapshots and consistent cuts;
- CRDTs as algebraic/local-to-global structures;
- database dependencies, lossless joins, and chase procedures;
- bidirectional transformations and lenses;
- refinement types and dependent types;
- temporal logic and model checking;
- Datalog, SAT/SMT, and CSP global-section search;
- applied category theory for schemas and data migration;
- higher categories for transformations between transformations;
- stacks for local objects with nontrivial automorphisms.

F.7 Next mathematical milestones

After completing the exercises, proceed in this order:

1. Work fluently with pullbacks, equalizers, and adjunction hom-set bijections.
2. Prove the sheaf condition for several ordinary examples.
3. Compute presheaf subobject classifiers as sieves on small posets.
4. Learn the Yoneda lemma well enough to use it without element notation.
5. Study sheafification more formally.
6. Learn chain complexes, homology, and cohomology over fields and integers.
7. Study cellular sheaves and implement block coboundary matrices.
8. Return to geometric morphisms and internal logic.
9. Explore descent, stacks, and higher coherence only after set-valued sheaves feel routine.

F.8 Completion checklist

You have absorbed the core of this book when you can do the following without looking up the answer:

- define a limit by a universal cone and prove uniqueness up to isomorphism;
- distinguish a product from a pullback in an API/data example;
- state functor and naturality laws as executable tests;
- design a context category and justify every restriction arrow;
- define a presheaf of locally lawful SessionStream observations;
- express the sheaf condition as an equalizer/limit;
- prove the snapshot-plus-suffix reconstruction theorem and list its assumptions;
- define a site and test pullback stability of coverage;
- explain why a sieve is a contextual truth value;
- use $\exists_f$ and $\forall_f$ to distinguish one valid completion from invariant-sufficient parameters;
- build a nerve and distinguish an edge loop from a filled face;
- compute H^0 and H^1 for a small complex;
- explain why missing EventId, a hydration gap, and cursor holonomy are three different mathematical problems;
- translate every mathematical diagnosis back into a repository test or architecture decision.